
AI系统工程师培训教程

面向零基础入门的AI系统全面培训教材

Apr 29, 2026

第一阶段：AI基础入门



关于本教程

1.1 教材简介

本教程是专为零基础新人设计的AI系统培训教材，旨在帮助半导体公司算法设计部门的工程师系统掌握AI系统的核心知识。

1.2 目标读者

- 零基础新人（无AI背景）
- 从半导体硬件转向AI软件的工程师
- 希望系统学习AI全栈知识的人员

1.3 学习路径

前置知识要求：

- 基本的编程概念
- 高中数学基础
- 对AI有初步兴趣

1.4 内容结构

阶段	章节	内容	目标字数
第一阶段	Chap01-03	预备知识（计算机/Python/数学）	22万
第二阶段	Chap04-05	AI基础（概述/神经网络）	16万
第二阶段	Chap06	AI芯片体系架构	20万
第二阶段	Chap07	AI编译器原理	20万
第二_phase	Chap08	推理系统与引擎	25万
第二阶段	Chap09-10	AI框架与大模型	30万
第三阶段	Appendix	附录（术语表/实践项目）	5万+
总计			208万+

1.4.1 Chap01 计算机基础

本章学习目标

1. 理解进制与编码的基本概念
2. 掌握CPU与内存的工作原理



Fig. 1.1: AI系统工程师学习路径

3. 了解程序运行的基本原理

前置知识

本章为预备知识第一章，无需特殊前置知识。

内容导航

第一章 进制与编码

学习目标

- 理解进制 (Binary, Octal, Hexadecimal) 的基本概念，掌握不同进制之间的转换方法
- 掌握二进制算术运算和逻辑运算的基本规则
- 理解计算机中整数的表示方式：原码、反码和补码
- 理解浮点数的IEEE 754编码标准
- 了解字符编码的发展历程，掌握ASCII码和UTF-8编码的基本原理
- 理解布尔代数的基本运算和门电路基础

1.1 进制的概念

1.1.1 什么是进制

进制 (Numeral System) 是一种记数的方法，也称为进位计数法。在日常生活中，我们最熟悉的是十进制 (Decimal)。然而，在计算机科学中，二进制 (Binary) 是最重要的进制。计算机是由复杂的电子电路构成的，而电路只有两种基本状态：开和关。除了二进制之外，八进制 (Octal) 和十六进制 (Hexadecimal) 也是计算机领域常用的进制。八进制使用0到7这八个数字，每4个二进制位恰好对应1个十六进制位。

1.1.2 进制的表示方法

当我们写一个数字时，不同进制会用不同的下标来区分。例如：

- 1011_2 表示二进制数 1011
- 273_8 表示八进制数 273
- 159_{10} 或简单的 159 表示十进制数 159
- $3FA_{16}$ 表示十六进制数 3FA

在计算机编程中，为了明确区分不同的进制，通常会有特殊的表示方法。以C语言为例：

- 二进制数通常以 0b 或 0B 开头，例如 0b1011
- 八进制数以 0 开头，例如 0273
- 十六进制数以 0x 或 0X 开头，例如 0x3FA

1.1.3 进制的权值

理解进制的关键是**权值 (Weight)**的概念。在十进制中，每个位上的数字都有不同的权值，从右到左依次是个位、十位、百位、千位等。例如，数字 1234 的权值分别是 $10^0=1$ ，十位的权值是 $10^1=10$ ，百位的权值是 $10^2=100$ ，以此类推。

类似地，在二进制中，每一位的权值是2的幂次方，从右到左依次是 $2^0, 2^1, 2^2, 2^3, \dots$ 。因此，二进制数 1011_2 转换为十进制就是：

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$$

在八进制中，每一位的权值是8的幂次方。所以八进制数 273_8 转换为十进制是：

$$2 \times 8^2 + 7 \times 8^1 + 3 \times 8^0 = 2 \times 64 + 7 \times 8 + 3 \times 1 = 128 + 56 + 3 = 187$$

十六进制的权值是16的幂次方。十六进制数 $3FA_{16}$ 转换为十进制是：

$$3 \times 16^2 + 15 \times 16^1 + 10 \times 16^0 = 3 \times 256 + 15 \times 16 + 10 \times 1 = 768 + 240 + 10 = 1018$$

1.1.4 二进制的特点

二进制在计算机中的广泛应用不是偶然的，而是由其物理特性决定的：

简单性：二进制只需要表示两种状态，这使得电路设计大大简化。在实际电路中，“通电”和“断电”很容易实现和区分，而

可靠性：两种状态之间的区分非常清晰，不容易受到干扰。即使有轻微的电压波动，也能可靠地区分“高电平”和“低电平”

运算简单：二进制的运算规则比十进制简单得多。加法规则只有四条： $0+0=0$ ， $0+1=1$ ， $1+0=1$ ， $1+1=10$ （进位）。乘法

逻辑基础：二进制与布尔代数（Boolean Algebra）完美契合，可以方便地实现各种逻辑运算，为计算机的逻辑判断能力奠

本节小结

- 进制是进位计数法，二进制是计算机的基础，因为它只有两种状态
 - 八进制和十六进制在计算机中也常用，它们与二进制转换方便
 - 每一位的权值等于基数的幂次方，从右到左依次增大
-

1.2 进制转换

1.2.1 十进制转换为其他进制

将十进制数转换为其他进制，除基取余法是最常用的方法。具体步骤是：用目标进制的基数连续去除十进制数，每次取余

十进制转二进制：将十进制数除以2，记录余数，直到商为0。

例如，将十进制数13转换为二进制：

$$13 \div 2 = 6 \cdots \cdots 1 \quad (\text{余数}1)$$

$$6 \div 2 = 3 \cdots \cdots 0 \quad (\text{余数}0)$$

$$3 \div 2 = 1 \cdots \cdots 1 \quad (\text{余数}1)$$

$$1 \div 2 = 0 \cdots \cdots 1 \quad (\text{余数}1)$$

将余数逆序排列，得到 1101_2 。验证： $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$ ，正确。

十进制转十六进制：将十进制数除以16，记录余数。

例如，将十进制数423转换为十六进制：

$$423 \div 16 = 26 \cdots \cdots 7 \quad (\text{余数}7)$$

$$26 \div 16 = 1 \cdots \cdots 10 \quad (\text{余数}A)$$

$$1 \div 16 = 0 \cdots \cdots 1 \quad (\text{余数}1)$$

逆序排列得到 $1A7_{16}$ 。

1.2.2 其他进制转换为十进制

将其他进制转换为十进制的方法是按权展开求和法。将每一位数字乘以该位的权值，然后求和。

二进制转十进制：将二进制数的每一位乘以对应的2的幂次方，然后求和。

例如， 101101_2 转换为十进制：

位位置	5	4	3	2	1	0
位值	1	0	1	1	0	1
权值	2^5	2^4	2^3	2^2	2^1	2^0
贡献	32	0	8	4	0	1

总和： $32 + 0 + 8 + 4 + 0 + 1 = 45$

十六进制转十进制：将十六进制数的每一位乘以16的幂次方。

例如， $2AF_{16}$ 转换为十进制：

$$2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 = 2 \times 256 + 10 \times 16 + 15 \times 1 = 512 + 160 + 15 = 687$$

1.2.3 二进制与八进制、十六进制的转换

由于8和16都是2的幂次方，二进制与八进制、十六进制之间的转换非常容易。

二进制转八进制：将二进制数从右到左每3位分成一组（不足三位时在左边补0），然后将每组转换为对应的八进制数。

例如， 1101011_2 转换为八进制：

分组： $110 \mid 101 \mid 1$ （补0后为001）

- $110 \rightarrow 6$
- $101 \rightarrow 5$
- $001 \rightarrow 1$

结果： 155_8

八进制转二进制：将每位八进制数转换为3位二进制数。

例如， 72_8 转换为二进制：

- $7 \rightarrow 111$
- $2 \rightarrow 010$

结果： 111010_2

二进制转十六进制：将二进制数从右到左每4位分成一组（不足四位时在左边补0），然后将每组转换为对应的十六进制数。

例如， 110101101_2 转换为十六进制：

分组： $1 \mid 1010 \mid 1101$ （注意第一组补0变成0001）

- $0001 \rightarrow 1$
- $1010 \rightarrow A$
- $1101 \rightarrow D$

结果： $1AD_{16}$

十六进制转二进制：将每位十六进制数转换为4位二进制数。

例如， $3B_{16}$ 转换为二进制：

- $3 \rightarrow 0011$
- $B(11) \rightarrow 1011$

结果： 00111011_2

1.2.4 常见进制对照表

下面是一些常见的十进制、二进制、八进制、十六进制数值的对照表：

十进制	二进制	八进制	十六进制
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
15	1111	17	F
16	10000	20	10
31	11111	37	1F
32	100000	40	20
63	111111	77	3F
64	1000000	100	40
127	1111111	177	7F
128	10000000	200	80
255	11111111	377	FF

1.2.5 小数部分的转换

带小数的进制转换稍微复杂一些。对于小数部分，转换方法是从小数部分连续乘以目标基数，每次取整数部分作为新的十进制小数转二进制：将小数部分乘以2，取整数部分，然后继续处理小数部分。

例如，将十进制小数 0.625 转换为二进制：

$$0.625 \times 2 = 1.25 \quad (\text{整数部分}1)$$

$$0.25 \times 2 = 0.5 \quad (\text{整数部分}0)$$

$$0.5 \times 2 = 1.0 \quad (\text{整数部分}1)$$

得到 0.101_2 。

如果十进制小数无法用二进制精确表示（如0.1），则需要按精度要求截断。例如，0.1转换为二进制是无限循环的： $0.0001100110011..._2$

本节小结

- 十进制转其他进制使用“除基取余法”，商为0时逆序取余数
- 其他进制转十进制使用“按权展开求和法”
- 二进制与八进制转换：每3位二进制对应1位八进制
- 二进制与十六进制转换：每4位二进制对应1位十六进制
- 小数部分转换使用“乘基取整法”

1.3 二进制运算

1.3.1 二进制加法

二进制加法是计算机最基本的运算之一，它遵循以下四条规则：

- $0 + 0 = 0$ ，进位为0
- $0 + 1 = 1$ ，进位为0
- $1 + 0 = 1$ ，进位为0
- $1 + 1 = 10$ ，进位为1

这与十进制加法类似，只不过当结果大于等于2时就要进位。在十进制中， $9 + 9 = 18$ 产生进位1；在二进制中， $1 + 1 = 10$ 也产生进位1。

竖式计算示例：计算 $1101_2 + 1011_2$

```

1101
+ 1011
-----
11000

```

计算过程（从右到左）：

- 第0位： $1 + 1 = 10$ ，写0进1
- 第1位： $0 + 1 + 1(\text{进位}) = 10$ ，写0进1
- 第2位： $1 + 0 + 1(\text{进位}) = 10$ ，写0进1
- 第3位： $1 + 1 + 1(\text{进位}) = 11$ ，写1进1
- 第4位：进位1，写1

结果： 11000_2 （等于十进制24）

1.3.2 二进制减法

二进制减法可以看作是加法的逆运算，但计算机通常不直接做减法，而是通过补码运算来实现。为了完整性，我们先介绍

- $0 - 0 = 0$ ，借位为0
- $1 - 0 = 1$ ，借位为0
- $1 - 1 = 0$ ，借位为0
- $0 - 1 = 1$ ，借位为1（在二进制中，0减1需要从高位借1）

借位示例：计算 $1100_2 - 0101_2$

```

1100
- 0101
-----
111

```

计算过程：

- 第0位： $0 - 1$ ，不够，向高位借1， $2 + 0 - 1 = 1$ ，借位1
- 第1位：原本是0，减去借位1后不够，向高位借1， $2 + 0 - 1 = 1$ ，借位1
- 第2位：原本是1，减去借位1后是0
- 第3位：1减去0还是1

结果： 0111_2 （等于十进制7）

1.3.3 二进制乘法

二进制乘法的规则也很简单：

- $0 \times 0 = 0$
- $0 \times 1 = 0$

- $1 \times 0 = 0$
- $1 \times 1 = 1$

乘法过程实际上是被乘数与乘数各位的与运算，然后错位相加。

竖式计算示例：计算 $1101_2 \times 1011_2$

```

  1101
× 1011
-----
  1101   (1101 × 1)
 1101   (1101 × 1, 但左移一位)
 0000   (1101 × 0)
 1101   (1101 × 1, 但左移三位)
-----
10001111

```

验证： $1101_2 = 13_{10}$ ， $1011_2 = 11_{10}$ ， $13 \times 11 = 143_{10}$ ， $10001111_2 = 143_{10}$ ，正确。

1.3.4 二进制除法

二进制除法与十进制除法类似，是乘法的逆运算。我们通过一个例子来说明：

计算 $10011_2 \div 11_2$ （即 $19_{10} \div 3_{10}$ ）

```

  111
-----
11 ) 10011
   11
   --
    101
    11
    --
     11
     11
     --
      0

```

结果：商为 111_2 （7），余数为0。

1.3.5 位运算

位运算（Bitwise Operations）是对二进制数的每一位分别进行操作的运算，这是计算机中非常重要的一类运算。

按位与（AND，&）：对应位都为1时结果为1，否则为0。

例如， $1101_2 \& 1011_2$ ：

```

  1101
& 1011
-----
  1001

```

按位或（OR，|）：对应位只要有一个为1结果就为1，否则为0。

例如， $1101_2 | 1011_2$ ：

```

  1101
| 1011
-----
  1111

```

按位异或 (XOR, ^) : 对应位相异时结果为1, 相同则为0。

例如, $1101_2 \oplus 011_2$:

```
1101
^ 011
-----
0110
```

按位取反 (NOT, ~) : 将每一位由0变1, 由1变0。

例如, $\sim 1101_2 = 0010_2$

1.3.6 移位运算

移位运算包括左移和右移, 是二进制运算中非常高效的操作。

左移 (<<) : 将二进制数的所有位向左移动指定位数, 低位补0, 高位丢弃。

例如, $0001_2 \ll 2 = 0100_2$ (即 $1 \times 4 = 4$)。左移n位相当于乘以 2^n 。

右移 (>>) : 将二进制数的所有位向右移动指定位数, 高位补0 (或符号位), 低位丢弃。

例如, $0100_2 \gg 2 = 0001_2$ (即 $4 \div 4 = 1$)。右移n位相当于除以 2^n (向下取整)。

本节小结

- 二进制加法遵循“逢二进一”的规则, 减法遵循“借一当二”
- 二进制乘法的本质是被乘数与乘数各位的与运算, 然后错位相加
- 位运算包括与 (&)、或 (|)、异或 (^)、取反 (~)
- 左移n位相当于乘以 2^n , 右移n位相当于除以 2^n

1.4 数据编码：整数表示

1.4.1 为什么需要原码、反码、补码

在人类的日常计数中, 负数通常用“-”符号来表示, 比如“-5”。但在计算机中, 由于二进制只能表示0和1, 我们需要用特殊

****原码 (Sign-Magnitude) ****是最简单的表示方法: 用一个专门的位 (通常是最高位) 来表示符号, 0表示正数, 1表示负数。例如, 00000101 , -5表示为 10000101 。

原码的优点是直观, 人类容易理解。但它有一个致命的问题: 0的表示不唯一。在8位原码中, 00000000 和 10000000 都表示0, 分别代表 +0 和 -0。这种不唯一性会给运算带来麻烦。

更重要的是, 原码的加减运算非常复杂。如果用原码计算 $1 + (-1)$, 需要判断符号、比较绝对值、决定是做加法还是减法...

****补码 (Two's Complement) ****的引入完美解决了这些问题。在补码系统中, 0只有一种表示, 符号位可以直接参与运算,

1.4.2 原码

原码的定义很直接: 最高位为符号位, 0表示正, 1表示负, 其余位表示数值的绝对值。

以8位二进制为例:

数值	原码表示
+0	00000000
-0	10000000
+1	00000001
-1	10000001
+127	01111111
-127	11111111

原码的优点：

- 直观，人类容易理解和阅读
- 符合我们对正负数的直觉认识

原码的缺点：

- 0有两种表示，造成资源浪费和逻辑混乱
- 加减运算复杂，需要分别处理符号和绝对值
- 电路实现困难，需要比较绝对值大小来决定是做加法还是减法

1.4.3 反码

****反码 (One's Complement) ****是对原码的一种改进。对于正数，反码与原码相同；对于负数，反码是符号位保持不变，

例如（8位）：

- +5 的反码：00000101（与原码相同）
- -5 的反码：11111010（符号位1不变，数值位取反：00000101 $\rightarrow$ 11111010）

数值	原码	反码
+0	00000000	00000000
-0	10000000	11111111
+1	00000001	00000001
-1	10000001	11111110

反码在一定程度上简化了运算（特别是减法可以通过加反码来实现），但并没有解决0有两种表示的问题。

1.4.4 补码

****补码 (Two's Complement) ****是目前计算机中普遍采用的有符号数表示方法。补码的定义为：

- 正数的补码 = 原码
- 负数的补码 = 反码 + 1（或者说，对原码除符号位外按位取反，然后加1）

例如（8位）：

- +5 的补码：00000101
- -5 的补码：11111010 + 1 = 11111011

让我们验证一下：-5 的补码 11111011，加上 +5 的补码 00000101：

```

11111011
+ 00000101
-----
100000000

```

结果是9位，最高位1溢出丢弃，得到 00000000，结果正确（-5 + 5 = 0）。这说明补码的加法可以直接进行，不需要考虑符号。补码的关键特性：

1. 0的唯一表示：在8位补码中，00000000 表示0，而10000000 不再表示-0，而是-128。这使得补码能够表示-128到127的整数范围（共256个值），而原码和反码只能表示-127到127（因为有+0和-0）。
2. 符号位参与运算：补码的符号位可以参与正常的加减运算，简化了电路设计。
3. 减法变加法： $a - b = a + (-b)$ ，而 $-b$ 的补码可以通过取反加一得到。
4. 模运算性质：补码本质上是模运算。以8位为例，模为256。 -1 的补码是255（11111111），所以 $1 + (-1) = 1 + 255 = 256 \equiv 0 \pmod{256}$ 。

补码的取值范围：

对于n位补码：

- 能表示的最大正数： $2^{n-1} - 1$ （如8位时为127）
- 能表示的最小负数： -2^{n-1} （如8位时为-128）
- 总共能表示 2^n 个不同的值

1.4.5 补码的直观理解

理解补码有一个很好的比喻：钟表模型。

想象一个12小时的钟表，当前时间是11点，如果要减掉3小时，可以：

- 直接往回拨： $11 - 3 = 8$
- 或者往前拨（因为 $12 - 3 = 9$ ）： $11 + 9 = 20 = 8$ （超过12的部分溢出丢弃）

在这个例子中，“往前拨”的操作就是“加补数”。如果我们把钟表看作12进制的补码系统，那么-3的补码就是 $12 - 3 = 9$ 。

对于8位二进制，模是256，-1的补码就是 $256 - 1 = 255$ （二进制11111111）。这就像在一个模256的循环中，-1和255是“等价”的。

1.4.6 整数溢出

当运算结果超出数据类型的表示范围时，就会发生溢出（Overflow）。

以8位有符号整数为例：

- 最大值是127（01111111）
- 最小值是-128（10000000）

如果计算 $127 + 1$ ：

```
01111111 (+127)
+ 00000001 (+1)
-----
10000000 (-128!)
```

结果变成了-128，而不是128。这就是上溢（Overflow）。

如果计算 $-128 - 1$ ：

```
10000000 (-128)
+ 11111111 (-1的补码)
-----
01111111 (+127!)
```

结果变成了+127，这就是下溢（Underflow）。

对于无符号整数，溢出后同样会“回绕”： $255 + 1 = 0$ ， $0 - 1 = 255$ 。

溢出检测：在计算机体系结构中，溢出标志位（Overflow Flag）用于指示最近一次算术运算是否发生了溢出。

本节小结

- 原码是最直观表示法，但0有两种表示，加减运算复杂
- 反码改进了负数表示，但仍有+0和-0的问题
- 补码是现代计算机的标准表示法：正数不变，负数是反码加1
- 补码的优势：0表示唯一，符号位可参与运算，减法变加法
- n位补码的范围是 -2^{n-1} 到 $2^{n-1} - 1$

- 运算结果超出范围会发生溢出

1.5 浮点数编码：IEEE 754

1.5.1 浮点数的由来

整数可以精确表示，但现实世界中还有很多需要表示小数。比如圆的周长是直径的 π 倍（ $\pi \approx 3.14159\dots$ ），或者计算复利时需要的指数运算。这些都不能用整数来精确表达。

****定点数（Fixed-Point Number）****是最早的小数表示方法。假设我们用8位表示小数，约定前4位是整数部分，后4位是小数部分。例如，00010010 就表示 $1.010_2 = 1.625_{10}$ 。

但定点数的问题是范围和精度固定。如果用前4后4的约定，最大值只能是 $11111111_2 = 15.9375_{10}$ ，最小精度是 $1/16 = 0.0625$ 。对于科学计算来说，这个范围和精度往往不够用。

****浮点数（Floating-Point Number）****借鉴了科学计数法（Scientific Notation）的思想。科学计数法中，一个数可以表示为：

$$\pm M \times 10^E$$

其中，M是尾数（Mantissa），E是指数（Exponent）。例如，光速 299792458 m/s 可以写成 2.99792458×10^8 。

类似地，浮点数表示为：

$$\pm M \times 2^E$$

这样既可以表示很大范围的数，也可以表示很小的数。

1.5.2 IEEE 754标准概述

IEEE 754是美国电气电子工程师协会制定的浮点数运算标准，是目前几乎所有现代计算机采用的浮点数表示法。

IEEE 754定义了两种精度的浮点数：

1. 单精度（Single Precision）：32位，也称为float
 - 1位符号位（Sign）
 - 8位指数位（Exponent）
 - 23位尾数位（Fraction/Mantissa）
2. 双精度（Double Precision）：64位，也称为double
 - 1位符号位
 - 11位指数位
 - 52位尾数位

此外还有16位的半精度（用于深度学习等场景）和128位的四精度（用于高精度计算）。

1.5.3 浮点数的结构

IEEE 754浮点数的结构如下：

符号位 S	指数部分 E	尾数部分 M（隐藏位）
1位	k位	n位

符号位（S）：0表示正数，1表示负数。注意，符号位与尾数、指数都独立，单独决定正负。

指数部分（E）：用于表示2的幂次方。采用****偏移码（Biased Exponent）****表示，即指数的真实值 = 指数位表示的值 - 偏移量。对于单精度，偏移量是127；对于双精度，偏移量是1023。

尾数部分 (M)：也称为小数部分或有效数字。IEEE

754的尾数部分采用**隐藏位 (Hidden Bit) **技术：对于规格化数，尾数实际上有24位（单精度），但最高位1是隐藏的，不存储在浮点数中。这相当于二进制 $1.011_2 \times 2^E$ ，所以这个1不需要存储。

1.5.4 规格化浮点数

**规格化 (Normalized) **浮点数是IEEE 754的“正常”表示形式，其特点是：尾数的最高位（即隐藏位）为1，所以尾数的 $[1, 2)$ 范围内。

规格化浮点数的指数部分不能是全0或全1（全0和全1有特殊含义，见下文）。

以32位单精度为例：

$$(-1)^S \times 1.\text{尾数} \times 2^{(E-127)}$$

例如，十进制数 6.5 的二进制是 $110.1 = 1.101 \times 2^2$ ：

- 符号位 $S = 0$ （正数）
- 指数 $= 2 + 127 = 129 = 10000001$
- 尾数 $= 101$ （后续补0）

所以 6.5 的浮点表示为：0 10000001 101000000000000000000000

验证： $(-1)^0 \times 1.101 \times 2^{(129-127)} = 1.101_2 \times 2^2 = 110.1_2 = 6.5_{10}$

1.5.5 特殊值

IEEE 754定义了一些特殊的浮点值：

1. 零 (Zero) 指数部分全0，尾数部分全0。注意，+0和-0是不同的：符号位分别为0和1。
2. 非规格化数 (Denormalized Number) 指数部分全0，但尾数部分非0。这些数的指数是固定的 -126 （单精度），可以表示非常接近0的数，但精度会降低。它们的出现是为了让浮点数能够渐进下溢 (Gradual Underflow)，而不是突然从最小正数跳到零。
3. 无穷大 (Infinity) 指数部分全1，尾数部分全0。符号位决定是正无穷还是负无穷。当除以零或运算结果超出表示范围时。
4. NaN (Not a Number) 指数部分全1，尾数部分非0。表示未定义或无法表示的结果，如 $0/0$ 、 $\sqrt{-1}$ 、 $\infty - \infty$ 等。

1.5.6 浮点数的取值范围和精度

以32位单精度为例：

类别	最小值	最大值
规格化正数	$2^{-126} \approx 1.18 \times 10^{-38}$	$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$
非规格化正数	$2^{-149} \approx 1.4 \times 10^{-45}$	略小于最小规格化正数

精度：单精度浮点数的尾数有23位，但加上隐藏的1位，实际有24位精度，约等于6到7位十进制数字。双精度有53位精度，这意味着：

- 浮点数不能精确表示所有小数，很多是近似值
- 浮点数运算不满足结合律：如 $(a + b) - b$ 不一定等于 a
- 比较浮点数是否相等要格外小心

1.5.7 浮点数的误差

浮点数的误差是一个重要且容易被忽视的问题。

舍入误差 (Rounding Error) : 由于尾数位数有限, 很多小数无法精确表示, 必须舍入到最近的可表示值。例如, 十进制

抵消误差 (Cancellation Error) : 当两个相近的数相减时, 有效数字会大大减少。例如, $1.0000001 - 1.0000000 = 0.0000001$, 原本有7位有效数字, 结果可能只剩下1位。

实例分析:

```
# Python代码演示浮点数误差
a = 0.1 + 0.2
print(a) # 输出 0.30000000000000004, 而不是精确的 0.3
print(a == 0.3) # 输出 False
```

这是因为 0.1、0.2 和 0.3 在二进制浮点数中都无法精确表示。

本节小结

- 浮点数借鉴了科学计数法的思想, 用尾数和指数来表示很大或很小的数
 - IEEE 754是浮点数表示的国际标准, 规定了32位单精度和64位双精度格式
 - 单精度浮点数由1位符号、8位指数、23位尾数组成, 采用偏移码和隐藏位技术
 - 指数全0和全1有特殊含义, 分别表示非规格化数和特殊值 (无穷大、NaN)
 - 浮点数精度有限, 存在舍入误差, 进行浮点运算时要格外小心
-

1.6 字符编码

1.6.1 为什么需要字符编码

计算机内部只有0和1, 那么文字、符号是如何存储的呢? 这就需要字符编码 (Character Encoding) —— 将字符映射为数字, 再将数字映射为二进制。

如果字符编码不统一, 就会出现乱码问题。例如, 一段用UTF-8编码的中文文本, 如果用GBK编码来解析, 就会显示为乱码。

1.6.2 ASCII码

****ASCII (American Standard Code for Information Interchange, 美国信息交换标准代码) ****是最早的字符编码标准, 诞生于1963年。

ASCII使用7位 (后来扩展为8位) 来表示128个字符, 包括:

- 控制字符 (0-31和127) : 如换行 (LF)、回车 (CR)、制表符 (Tab) 等, 不可打印
- 可打印字符 (32-126) : 包括空格、英文大小写字母 (A-Z: 65-90, a-z: 97-122)、数字0-9 (48-57)、标点符号等

常用ASCII码对照:

- 字符'0'的ASCII码是48, 二进制是 00110000
- 字符'A'的ASCII码是65, 二进制是 01000001
- 字符'a'的ASCII码是97, 二进制是 01100001

ASCII码的特点:

- 只用了7位, 最高位为0
- 只包含英文字母、数字和少量符号
- 足以支持英文, 但无法表示中文等其他语言

1.6.3 扩展编码

由于ASCII只能表示128个字符，对于需要更多字符的语言（如欧洲语言使用的带重音字母）来说不够用。

****ISO-8859-1 (Latin-1) ****将ASCII的最高位利用起来，用8位表示256个字符。它覆盖了大多数西欧语言，但仍然无法表示

GB2312/GBK/GB18030是中国制定的汉字编码标准：

- GB2312：1980年发布，收录6763个汉字和682个图形符号
- GBK：GB2312的扩展，收录21003个汉字
- GB18030：最新标准，收录70000多个汉字，还包含了藏文、蒙文等少数民族文字

BIG5是繁体中文编码，主要在台湾、香港、澳门使用。

1.6.4 Unicode与UTF-8

随着互联网的普及，需要一种能够表示世界上所有文字的编码系统。Unicode应运而生。

Unicode的目标是为每一个文字符号分配一个唯一的编码（称为码点，Code Point），无论是什么语言、什么平台。Unicode目前已经收录超过14万个字符。

Unicode的表示方式：U+XXXX，其中XXXX是4到6位的十六进制数。例如：

- ‘A’的码点是 U+0041
- ‘中’的码点是 U+4E2D
- ‘□’的码点是 U+1F600

但Unicode只定义了码点到数值的映射，没有规定如何在计算机中存储这些数值。这就是****UTF (Unicode Transformation Format) ****的作用。

UTF-8是目前互联网上最常用的Unicode编码方式，它是一种变长编码：

码点范围	字节数	编码格式
U+0000 ~ U+007F	1字节	0xxxxxxx
U+0080 ~ U+07FF	2字节	110xxxxx 10xxxxxx
U+0800 ~ U+FFFF	3字节	1110xxxx 10xxxxxx 10xxxxxx
U+10000 ~ U+10FFFF	4字节	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

UTF-8的设计非常巧妙：

- 与ASCII完全兼容：ASCII字符在UTF-8中只需1字节，与ASCII编码完全相同
- 自同步：任何字节的开始位置都可以被识别，不会因为中间某字节损坏而影响后续字符
- 节省空间：英文字母只占1字节，中文字符占3字节（比UTF-16的2字节稍大，但考虑到英文文本的空间节省，总体效果

示例：

- ‘A’ (U+0041)：在第一范围内，编码为 01000001（1字节）
- ‘中’ (U+4E2D)：在第三范围内
 - 4E2D 的二进制：0100 1110 0010 1101
 - 去掉U+的前缀得到：100 111000 101101（补齐12位）
 - 按3字节格式填充：11100100 10111000 10101101
 - 十六进制：E4 B8 AD

验证：“中”字的UTF-8编码是 E4B8AD，在浏览器开发者工具中可以看到。

1.6.5 编码选择与转换

不同的编码适用于不同的场景：

- ASCII：英文为主的文档，不需要中文字符时
- GBK/GB18030：中文Windows系统的默认编码
- UTF-8：互联网内容（网页、邮件等）的标准编码，跨平台、跨语言的首选
- UTF-16：Java、JavaScript、C#等语言内部使用的编码（Windows的API也常用UTF-16）

编码转换的注意事项：

1. 转换时需要知道源编码是什么，否则会乱码
2. 某些字符在目标编码中可能不存在（如将GBK的中文转成ASCII），通常会用'?'或'□'替代
3. 文本文件开头可能有BOM（Byte Order Mark），用于指示是大端序还是小端序，以及编码类型

1.6.6 字符串与编码

在编程中处理字符串时，编码问题经常是bug的来源。

以Python为例：

```
# Python 3默认使用UTF-8
s = "你好，世界"
print(len(s)) # 输出 6（字符数，不是字节数）
print(s.encode('utf-8')) # 输出 b'\xe4\xbd\xa0\xe5\xa5\xbd\xef\xbc\x8c\xe4\xb8\x96\xe7\x95\x8c'
print(len(s.encode('utf-8'))) # 输出 18（UTF-8编码占18字节）
```

以JavaScript为例：

```
// JavaScript字符串是UTF-16编码
let s = "你好，世界";
console.log(s.length); // 输出 6
console.log(new TextEncoder().encode(s)); // 输出 Uint8Array [230, 151, 165, 229, 155, 189, 239, 188, 140, 228,
↪ 188, 155, 231, 149, 140]
```

本节小结

- 字符编码将字符映射为数字，再存储为二进制
- ASCII是最早的编码，用7位表示128个字符，只适用于英文
- 为了支持其他语言，出现了GBK、Big5等地区性编码
- Unicode为世界上所有文字分配了唯一的码点
- UTF-8是互联网的主要编码，变长设计，与ASCII兼容，节省空间
- 处理文本时要注意编码一致性，避免乱码

1.7 布尔代数与逻辑门

1.7.1 布尔代数基础

****布尔代数（Boolean Algebra）****由英国数学家乔治·布尔（George Boole）在19世纪中叶创立，是计算机科学的数学基础。与普通代数使用实数不同，布尔代数使用布尔值（也称为逻辑值）：只有两个可能的值，通常记为真（True）和假（False）。

1.7.2 基本逻辑运算

布尔代数有三种最基本的运算：

1. 与运算（AND，合取）

也称为逻辑乘，用符号 $\cdot$ 或 $\wedge$ 表示。运算规则是：只有当所有操作数都为真时，结果才为真。

$$A \wedge B = \begin{cases} 1 & \text{如果 } A = 1 \text{ 且 } B = 1 \\ 0 & \text{其他情况} \end{cases}$$

2. 或运算（OR，析取）

用符号 $+$ 或 $\vee$ 表示。运算规则是：只要有一个操作数为真，结果就为真。

$$A \vee B = \begin{cases} 1 & \text{如果 } A = 1 \text{ 或 } B = 1 \\ 0 & \text{其他情况} \end{cases}$$

3. 非运算（NOT，否定）

用符号 $\neg$ 或 $\bar{A}$ 表示。运算是取反：真变假，假变真。

$$\neg A = \begin{cases} 1 & \text{如果 } A = 0 \\ 0 & \text{如果 } A = 1 \end{cases}$$

1.7.3 复合逻辑运算

除了三种基本运算，还有几种常见的复合运算：

异或（XOR，排他或）：当两个操作数不同时结果为真，相同时为假。

$$A \oplus B = A \cdot \bar{B} + \bar{A} \cdot B$$

同或（XNOR，等价）：当两个操作数相同时结果为真，不同则为假，是异或的反。

$$A \odot B = \overline{A \oplus B} = A \cdot B + \bar{A} \cdot \bar{B}$$

1.7.4 布尔代数定律

布尔代数遵循一些重要的定律，这些定律可以直接用于逻辑电路的化简：

交换律： $A + B = B + A$ $A \cdot B = B \cdot A$

结合律： $(A + B) + C = A + (B + C)$ $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

分配律： $A \cdot (B + C) = A \cdot B + A \cdot C$ $A + (B \cdot C) = (A + B) \cdot (A + C)$

吸收律： $A + A \cdot B = A$ $A \cdot (A + B) = A$

德·摩根定律（De Morgan's Laws）： $\overline{A + B} = \bar{A} \cdot \bar{B}$ $\overline{A \cdot B} = \bar{A} + \bar{B}$

德·摩根定律是布尔代数中特别重要的定律，它表明“与”和“或”在取反时可以相互转换。

1.7.5 逻辑门基础

逻辑门（Logic Gate）是实现布尔运算的电子元件，是数字电路的基本构建单元。

与门（AND Gate）：实现与运算。符号是一个平顶三角形后面跟着一个圆（表示“与”）。有多个输入，一个输出，只有所有输入都为真时，输出才为真。

或门（OR Gate）：实现或运算。符号是一个圆弧后面跟着一个平顶三角形。有多个输入，一个输出，只要有一个输入为真，输出就为真。

非门（NOT Gate）：实现非运算，也称为反相器（Inverter）。符号是一个三角形后面跟着一个小圆圈。一个输入，一个输出，输出是输入的取反。

与非门（NAND Gate）和或非门（NOR Gate）：分别是与门和非门、或门和非门的组合，输出是AND或OR结果的取反。

异或门（XOR Gate）：实现异或运算。符号与或门相似，但在输入侧多了一条曲线。

1.7.6 组合逻辑电路

由多个逻辑门组成的电路称为组合逻辑电路（Combinational Logic Circuit），其输出仅取决于当前的输入。

****加法器（Adder）****是组合逻辑电路的典型例子。

半加器（Half Adder）：实现两个一位二进制数相加，产生和（Sum）和进位（Carry）。电路由一个异或门和一个与门组成。

- 和 = $A \oplus B$
- 进位 = $A \text{ AND } B$

全加器（Full Adder）：实现三个一位二进制数相加（包含来自低位的进位）。它由两个半加器和一个或门组成。

多位数加法器由多个全加器级联而成，可以实现任意位数的二进制加法。

1.7.7 时序逻辑电路

与组合逻辑电路不同，****时序逻辑电路（Sequential Logic Circuit）****的输出不仅取决于当前输入，还取决于之前的输入（状态）。

****触发器（Flip-Flop）****是时序逻辑电路的基本存储单元。最基本的RS触发器由两个与非门或两个或非门交叉连接而成。

****寄存器（Register）****由多个触发器组成，可以存储多个位的数据。

****计数器（Counter）****是一种特殊的时序逻辑电路，可以对输入脉冲进行计数。

本节小结

- 布尔代数使用真/假（1/0）两种值，有与、或、非三种基本运算
- 异或、同或等复合运算可以由基本运算组合而成
- 布尔代数定律包括交换律、结合律、分配律、吸收律和德·摩根定律
- 逻辑门是实现布尔运算的电子元件，包括与门、或门、非门等
- 组合逻辑电路的输出只取决于当前输入，加法器是其典型应用
- 时序逻辑电路具有记忆功能，触发器是其基本存储单元

本章小结

本章介绍了计算机中数据表示和处理的基础知识，主要包括以下几个部分：

1. 进制与转换：计算机使用二进制（0和1），八进制和十六进制是为了方便表示二进制而引入的。不同进制之间可以相互转换。
2. 二进制运算：包括算术运算（加、减、乘、除）和位运算（与、或、异或、取反、移位）。计算机通过补码实现减法运算。
3. 整数编码：原码、反码、补码是三种有符号数的表示方法。现代计算机采用补码表示有符号整数，因为它解决了0的表示问题。
4. 浮点数编码：IEEE 754标准规定了浮点数的表示方法。单精度浮点数由1位符号、8位指数、23位尾数组成，采用偏移二进制表示。
5. 字符编码：ASCII是最初的字符编码，只支持英文字母。Unicode为世界上所有文字分配了唯一码点，UTF-8是互联网最常用的编码，采用变长设计，与ASCII兼容。
6. 布尔代数与逻辑门：布尔代数是数字电路的数学基础，与、或、非三种基本运算可以组合成任意复杂的逻辑功能。

思考与练习

1. 进制转换
 - 将二进制数 1101011_2 转换为十进制、八进制和十六进制。
 - 将十进制数 1023_{10} 转换为二进制。
2. 补码运算

- 8位补码能表示的整数范围是多少?
 - 计算8位补码 $11111111 + 00000001$ 的结果（用补码解释），并说明发生了什么。
3. 浮点数理解
- 用IEEE 754单精度表示法表示十进制数 3.14159（约等于圆周率）。
 - 解释为什么 $0.1 + 0.2 \neq 0.3$ 在大多数编程语言中成立。
4. 字符编码
- ‘A’和‘a’的ASCII码分别是65和97，计算它们的二进制表示，并解释为什么小写字母的ASCII码比对应的大写字母大32。
 - UTF-8编码中，中文字符通常占3个字节，请说明为什么需要多个字节来表示中文。
5. 逻辑化简
- 使用布尔代数定律化简表达式： $F = A \cdot B + \bar{A} \cdot C + B \cdot C$
 - 给出化简后的结果，并用真值表验证。
6. 溢出分析
- 用8位有符号整数计算 $100 + 50$ ，判断是否溢出；如果溢出，说明原因。
 - 如果改用8位无符号整数计算同样的表达式，结果是多少？

第二章 CPU与内存

学习目标

- 理解冯诺依曼架构的基本原理和计算机五大组成部分
- 掌握CPU的基本结构：算术逻辑单元（ALU）、控制单元（CU）和寄存器
- 理解CPU指令执行的基本流程：取指、解码、执行
- 掌握内存层次结构：寄存器、高速缓存（Cache）、主存、虚拟内存
- 理解缓存的工作原理和局部性原理
- 掌握CPU与内存交互的基本方式：总线和DMA

2.1 冯诺依曼架构

2.1.1 计算机架构的诞生

1945年，美籍匈牙利数学家约翰·冯·诺依曼（John von Neumann）在一份关于EDVAC（Electronic Discrete Variable Automatic Computer，电子离散变量自动计算机）的草案中，提出了一种全新的计算机架构设计。这份草案被称为“冯诺依曼架构”（Von Neumann Architecture）。

冯诺依曼架构的核心思想是**存储程序（Stored-Program）**概念：程序和数据都存储在计算机的内存中，计算机可以像处理数据一样处理程序，程序不再是一成不变的硬件接线，而是可以动态改变的可修改内容。

2.1.2 冯诺依曼架构的五大组成部分

冯诺依曼架构将计算机分为五个核心部分：

1. 运算器（Arithmetic Logic Unit, ALU）负责执行所有的算术运算（如加、减、乘、除）和逻辑运算（如与、或、非、比较）。
2. 控制器（Control Unit, CU）负责协调和控制计算机各部件的工作。它的核心功能是从内存中取指令、分析指令（解码）并控制其他部件执行。
3. 存储器（Memory）存储数据和指令。在现代计算机中，这通常指主存（Main Memory），即我们常说的内存（RAM, Random Access Memory）。存储器是计算机的“记忆”，没有它，计算机就无法保持任何信息。
4. 输入设备（Input Device）将外部信息输入到计算机中。常见的输入设备包括键盘、鼠标、触摸屏、扫描仪、传感器等。
5. 输出设备（Output Device）将计算机处理后的信息输出。常见的输出设备包括显示器、打印机、扬声器、执行器等。

2.1.3 冯诺依曼瓶颈

冯诺依曼架构虽然简单高效，但存在一个固有的问题，称为冯诺依曼瓶颈（Von Neumann Bottleneck）。

问题在于：运算器和存储器是分开的两个部件，数据需要在它们之间传输。CPU的处理速度增长非常快（遵循摩尔定律）。打个比方：这就像一个天才的厨师（CPU），但需要等待食材从远处的仓库（内存）送来。厨师做得再快，如果食材供应不上，整体效率就会受限。现代计算机通过多种技术来缓解这个问题：

- 在CPU和内存之间加入高速缓存（Cache）
- 使用更宽的数据总线
- 采用指令流水线（Pipeline）技术
- 发展专用AI芯片，采用不同的架构（如存算一体）

2.1.4 哈佛架构与改进

与冯诺依曼架构相对的是哈佛架构（Harvard Architecture）。哈佛架构将指令存储器和数据存储器分开，指令总线和数据总线也是分开的。然而，哈佛架构的电路更加复杂，通用性也不如冯诺依曼架构。因此，现代计算机通常采用修正的哈佛架构（Modified Harvard Architecture），在内部保留哈佛架构的特点（独立的指令Cache和数据Cache），但对外呈现统一的内存视图。

2.1.5 计算机的基本工作流程

冯诺依曼架构计算机的基本工作流程是一个不断循环的过程，通常称为执行周期（Instruction Cycle）：

1. 取指（Fetch）：控制器从内存中读取下一条指令到指令寄存器
2. 解码（Decode）：控制器分析指令，确定需要执行什么操作
3. 执行（Execute）：运算器执行指令规定的操作
4. 访存（Memory Access）：如果需要，访问内存读写数据
5. 写回（Write Back）：将执行结果写回寄存器

这个过程不断循环，直到遇到停机指令或发生错误。

本节小结

- 冯诺依曼架构是现代计算机的基础，由运算器、控制器、存储器、输入/输出设备五部分组成
 - “存储程序”概念是冯诺依曼架构的核心创新，程序和数据都存储在内存中
 - 冯诺依曼瓶颈是指CPU和内存之间数据传输速度不匹配的问题
 - 哈佛架构将指令存储器和数据存储器分开，可以缓解冯诺依曼瓶颈
 - 计算机的基本工作流程是“取指-解码-执行-访存-写回”的循环
-

2.2 CPU的基本结构

2.2.1 CPU是什么

CPU（Central Processing Unit），即中央处理器，是计算机的大脑。它负责执行指令、处理数据，是计算机性能的核心决定因素。现代CPU通常是一个**集成电路（Integrated Circuit, IC）**芯片，封装在金属或陶瓷外壳中，通过引脚与外部电路连接。CPU的性能通常用以下几个指标来衡量：

- 时钟频率（Clock Frequency）：CPU的工作节拍，单位是GHz（吉赫兹）。1 GHz表示每秒可以执行10亿个时钟周期。频率越高，理论上CPU执行速度越快。
- 每周期指令数（Instructions Per Cycle, IPC）：每个时钟周期能完成的指令数。CPU架构的改进可以提高IPC。

- 核心数（Number of Cores）：CPU中独立的处理单元数量。多核CPU可以并行执行多个任务。

2.2.2 算术逻辑单元（ALU）

ALU（Arithmetic Logic Unit）是CPU中负责执行计算的部件。它的名字已经说明了它的两大类功能：算术运算和逻辑运算

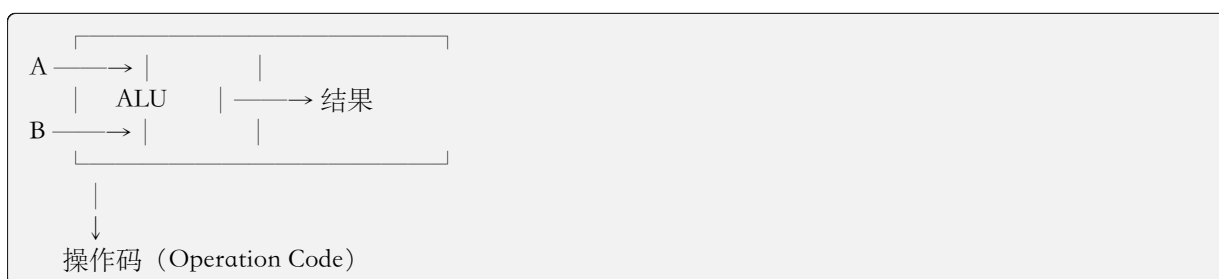
算术运算包括：

- 基本运算：加法、减法、乘法、除法
- 在现代CPU中，乘法和除法通常由专用的乘法器和除法器实现，比用加法器反复相加要快得多

逻辑运算包括：

- 按位与（AND）、或（OR）、异或（XOR）、取反（NOT）
- 比较运算：如大于、小于、等于
- 移位运算：左移、右移、算术移位、逻辑移位

ALU的基本结构：



ALU接受两个输入A和B，以及一个操作码（Opcode），输出运算结果。操作码指示ALU执行哪种操作。

标志位（Flag）：

ALU还会产生一些额外的信息，称为标志位或状态标志：

- 零标志（Zero Flag, ZF）：运算结果是否为0
- 符号标志（Sign Flag, SF）：运算结果是否为负数
- 进位标志（Carry Flag, CF）：无符号运算是否产生进位或借位
- 溢出标志（Overflow Flag, OF）：有符号运算是否发生溢出

这些标志位会被送到控制单元，用于决定后续的指令执行流程。

2.2.3 控制单元（CU）

****控制单元（Control Unit, CU）****是CPU的“指挥官”，负责协调和控制CPU内部各部件的工作，以及CPU与外部的交互。

控制单元的主要功能包括：

指令获取（Instruction Fetch）：

- 程序计数器（Program Counter, PC）保存下一条要执行的指令地址
- 控制单元将PC的值送到地址总线，从内存读取指令
- 读取的指令存入指令寄存器（Instruction Register, IR）

指令解码（Instruction Decode）：

- 控制单元分析指令的操作码（Opcode），确定这是什么指令
- 分析操作数（Operand），确定需要访问哪些寄存器或内存位置
- 向相关部件发出控制信号，准备执行

指令执行（Instruction Execution）：

- 根据解码结果，向ALU发出运算控制信号
- 控制数据在寄存器组、ALU、内存之间流动
- 记录执行结果，更新程序计数器

现代CPU的控制单元实现方式：

早期CPU使用硬连线控制（Hardwired Control）：控制单元是一组固定的逻辑电路，每个指令有对应的硬件线路。优点是

现代CPU更多使用微程序控制（Microprogrammed Control）：将每条指令分解为更基本的微操作（Micro-operation），存储在控制存储器（Control Store）中。微程序可以看作是指令的解释器。优点是设计灵活，易于修改；缺

RISC架构（如ARM、MIPS）倾向于使用硬连线控制，因为指令集简单规整；CISC架构（如x86）则常用微程序控制，因为

2.2.4 寄存器

****寄存器（Register）****是CPU内部的高速存储单元，用于临时保存操作数和运算结果。寄存器是CPU和内存之间数据传输的通道。CPU要处理的数据，先从内存加载到寄存器，运算完成后可能再写回内存。

寄存器的速度非常快，因为它们就在CPU内部，与CPU以相同的时钟频率工作。一个时钟周期内可以完成多次寄存器读写。寄存器的分类：

1. 通用寄存器（General Purpose Register, GPR）用于存储操作数、运算结果、地址等，编程人员可以自由使用。

以x86-64架构为例，有16个64位通用寄存器：

- `rax, rbx, rcx, rdx`：常用于特定用途（`rax`用于返回值，`rcx`用于计数等）
- `rsi, rdi`：用于字符串操作（源索引、目的索引）
- `rbp, rsp`：栈帧指针（基址指针、栈指针）
- `r8-r15`：通用用途

2. 程序计数器（Program Counter, PC）也称为指令指针（Instruction Pointer, IP），保存下一条要执行的指令地址。执行完一条指令后，PC自动更新为下一条指令的地址（除非是跳转指令）。

3. 指令寄存器（Instruction Register, IR）存储当前正在执行的指令。指令从内存取出后，首先存入IR，然后由控制单元解

4. 状态寄存器（Status Register）也称为标志寄存器（Flag Register）或程序状态字（Program Status Word, PSW）。存储ALU产生各种状态标志，如零标志、进位标志、溢出标志等，以及中断使能等控制位。

5. 栈指针（Stack Pointer, SP）指向当前栈的顶部。栈是一种重要的数据结构，用于函数调用、局部变量存储等。

6. 基址指针（Base Pointer, BP）也称为帧指针（Frame Pointer），用于访问函数的参数和局部变量。

2.2.5 指令的基本格式

指令是CPU执行任务的基本单位。每条指令都告诉CPU要做什么操作，以及操作的数据在哪里。

典型的指令格式包括：

操作码	操作数1	操作数2
Opcode	Operand1	Operand2

****操作码（Opcode）****指明要执行的操作类型，如加法、减法、跳转等。

****操作数（Operand）****指明操作的数据在哪里。操作数可以是：

- 立即数（Immediate）：指令中直接给出的常量值
- 寄存器：操作数在某个寄存器中
- 内存地址：操作数在内存中某个位置

指令长度：

- 在RISC架构（如ARM）中，所有指令长度相同（如32位）
- 在CISC架构（如x86）中，指令长度可变（1到15字节不等）

x86指令示例：

- `mov eax, 5`: 将立即数5传送到寄存器eax
- `add ebx, ecx`: 将ebx和ecx相加，结果存入ebx
- `cmp eax, 10`: 比较eax和10，设置标志位
- `jne loop`: 如果不相等（Zero Flag为0），跳转到loop标签

2.2.6 指令执行详解

让我们通过一个具体的例子，详细了解指令执行的完整过程。

假设我们要执行：`add eax, ebx`（将eax和ebx的值相加，结果存入eax）

这条指令的功能是： $eax = eax + ebx$

第一步：取指（Fetch）

1. 控制单元将程序计数器（PC）的值送到地址总线
2. 内存根据地址返回指令数据到数据总线
3. 指令数据存入指令寄存器（IR）
4. PC更新，指向下一条指令的地址

第二步：解码（Decode）

1. 控制单元分析IR中的操作码，确定这是一条加法指令
2. 分析操作数：源操作数是ebx和eax（隐含），目的操作数是eax
3. 控制单元发出控制信号，准备从寄存器读取ebx和eax的值

第三步：读取操作数（Read Operands）

1. 从寄存器文件（Register File）读取ebx的值
2. 从寄存器文件读取eax的值
3. 将这两个值送到ALU的输入端

第四步：执行（Execute）

1. 控制单元向ALU发出“加法”控制信号
2. ALU执行加法运算
3. ALU产生结果： $eax + ebx$
4. ALU同时产生各种标志位（零标志、进位标志等）

第五步：写回（Write Back）

1. 控制单元将ALU的结果写回eax寄存器
2. 更新状态寄存器中的标志位

第六步：更新PC

1. 如果不是跳转指令，PC已经指向下一条指令
2. 如果是跳转指令，且跳转条件满足，PC被设置为目标地址

这个过程看起来很复杂，但现代CPU可以在一个时钟周期内完成多个步骤，或者通过流水线技术并行处理不同指令的不同

本节小结

- CPU是计算机的大脑，由ALU、控制单元和寄存器组成
- ALU负责算术运算和逻辑运算，是CPU的“计算器”
- 控制单元负责指令的获取、解码和执行控制
- 寄存器是CPU内部的高速存储单元，用于临时保存数据
- 指令执行的基本流程是：取指→解码→读操作数→执行→写回
- 通用寄存器可用于存储任意数据，专用寄存器有特定用途

2.3 内存层次结构

2.3.1 为什么要分层

让我们先思考一个问题：既然CPU需要从内存读写数据，为什么不把内存做得又快又大，满足所有需求呢？答案在于一个基本的物理和经济规律：速度、容量和成本是一个不可能三角（Impossible Triangle）。

- 速度更快的存储技术，通常制造成本更高，消耗更多能量，体积也更大
- 容量更大的存储技术，通常速度更慢

举例来说：

- CPU内部的寄存器，访问延迟只有不到1纳秒（ns），但容量只有几十到几百个字节
- SRAM（静态随机存取存储器，用作CPU缓存），延迟约2-5纳秒，每MB成本很高
- DRAM（动态随机存取存储器，用作内存），延迟约50-100纳秒，成本适中
- SSD（固态硬盘），延迟约100微秒，容量可以很大，成本较低
- 机械硬盘，延迟约10毫秒，容量最大，成本最低

如果我们只用最快的存储技术，1TB容量的“寄存器”将贵得离谱。如果我们只用最便宜的存储技术，计算机将慢得无法使用。因此，现代计算机采用内存层次结构（Memory Hierarchy），将不同类型的存储设备组合起来，兼顾速度、容量和成本。

2.3.2 内存层次结构图

内存层次结构从上到下，大致分为以下几个层次：



(continued from previous page)



每一层都服务于其上一层：当上一层需要数据时，下一层负责提供；当上一层有数据要保存时，可能需要写到下一层。

2.3.3 寄存器 (Registers)

寄存器是内存层次结构中最顶层的存储设备，完全位于CPU内部。

寄存器的特点：

- 极快的访问速度：访问延迟小于1纳秒，通常一个时钟周期就可以完成读写
- 极小的容量：桌面处理器通常有十几个到几十个通用寄存器，每个64位（8字节），总共也就几百字节
- 直接参与运算：ALU的输入直接来自寄存器，运算结果也直接写回寄存器

寄存器的类型：

- 通用寄存器：存储任意数据
- 专用寄存器：如程序计数器（PC）、栈指针（SP）、基址指针（BP）
- 状态寄存器：存储各种标志位

寄存器是CPU与内存之间数据传输的最终点。CPU要处理的数据，必须加载到寄存器中；计算结果，也最终保存在寄存器

2.3.4 高速缓存 (Cache)

高速缓存是介于CPU寄存器和主存之间的存储层次，用于减少CPU访问主存的次数，从而缓解冯诺依曼瓶颈。

为什么需要Cache：

程序运行具有局部性 (Locality)：

- 时间局部性 (Temporal Locality)：如果某个数据被访问过，那么近期它很可能再次被访问。例如，循环变量在循环中反复使用。
- 空间局部性 (Spatial Locality)：如果某个数据被访问过，那么与它相邻的数据很可能也会被访问。例如，遍历数组时，相邻元素会被连续访问。

由于局部性的存在，我们可以把最近访问的数据（以及相邻的数据）复制到Cache中。如果CPU要访问的数据已经在Cache中（Cache Hit），就可以快速获取；如果不在（缓存未命中，Cache Miss），则需要从主存读取，同时把该数据及其相邻数据块一起加载到Cache中。

Cache的结构：

Cache的容量比主存小得多，但速度比主存快很多。以L1 Cache为例：

- 容量：通常32KB到128KB
- 延迟：约1-5纳秒
- 访问周期：约4-10个时钟周期

Cache的基本单元是缓存行 (Cache Line)，也称为块 (Block)。每条缓存行包含：

- 数据块 (Data Block)：存储从主存复制过来的数据，通常是几十字节（如64字节）
- 标签 (Tag)：主存地址的一部分，用于标识这个缓存行对应哪个主存地址

- 有效位 (Valid Bit) : 标识这个缓存行是否包含有效数据

缓存映射方式:

由于Cache容量远小于主存, 需要一种机制来确定主存中的某个地址的数据现在位于Cache的哪个位置。常见的映射方式有:

1. 直接映射 (Direct-Mapped) : 每个主存地址只能映射到Cache中的一个特定位置
 - 优点: 查找简单, 速度快
 - 缺点: 容易发生冲突
2. 组相联 (Set-Associative) : 每个主存地址可以映射到Cache中的一组位置, 查找时在这一组中遍历
 - 例如, 4路组相联: 每个地址映射到4个可能的Cache位置
 - 优点: 减少冲突, 提高命中率
 - 缺点: 查找稍微复杂
3. 全相联 (Fully-Associative) : 每个主存地址可以放在Cache中的任意位置
 - 优点: 最灵活, 冲突最少
 - 缺点: 需要并行比较所有Cache行的标签, 成本高, 只适合小容量Cache (如TLB)

缓存的读写策略:

读操作:

1. CPU发出要读取的内存地址
2. Cache控制器检查该地址是否在Cache中 (通过标签比较)
3. 如果命中, 直接从Cache返回数据给CPU
4. 如果未命中, 从主存读取数据, 同时:
 - 将数据返回给CPU
 - 将数据存入某个Cache行 (可能需要替换已有的数据)

写操作:

1. 写直达 (Write-Through) : 同时写Cache和主存
 - 优点: 主存数据始终有效, 一致性好
 - 缺点: 每次写操作都要访问主存, 速度慢
2. 写回 (Write-Back) : 只写Cache, 只有当该Cache行被替换时, 才写回主存
 - 优点: 减少对主存的写操作, 提高性能
 - 缺点: 需要额外的“脏位” (Dirty Bit) 来标识哪些行被修改过

2.3.5 多级缓存

现代CPU通常有多个级别的缓存:

L1 Cache (一级缓存) :

- 最接近CPU核心, 速度最快
- 容量最小, 通常32KB到128KB
- 通常分为指令缓存 (I-Cache) 和数据缓存 (D-Cache)
- 延迟: 约1-2纳秒

L2 Cache (二级缓存) :

- 比L1慢一些, 容量更大
- 可以是每个核心私有, 也可以是共享的

- 容量通常256KB到几MB
- 延迟：约3-10纳秒

L3 Cache（三级缓存）：

- 比L2更慢一些，容量更大
- 通常是所有核心共享的
- 容量可以达几十MB
- 延迟：约10-20纳秒

缓存层级的工作方式：

- 当CPU要访问内存时，首先检查L1 Cache
- 如果L1未命中，检查L2 Cache
- 如果L2未命中，检查L3 Cache
- 如果L3也未命中，则访问主存

这种逐级查找的方式，对于具有良好局部性的程序，可以大大减少对主存的访问次数。

2.3.6 主存 (Main Memory)

主存通常指DRAM（Dynamic Random Access Memory，动态随机存取存储器），是我们通常所说的“内存”。

DRAM的工作原理：

DRAM使用电容来存储每一位数据。电容充电时表示1，放电时表示0。但电容会随着时间慢慢放电（漏电），所以需要**

DRAM的基本单元（一个bit）由一个电容和一个晶体管组成，结构简单，可以做得非常密集。

DRAM的访问：

CPU不能直接访问DRAM。需要通过**内存控制器（Memory Controller）**来访问。

内存控制器是CPU和DRAM之间的桥梁，负责：

- 接收CPU的内存访问请求
- 转换地址格式
- 刷新DRAM
- 处理数据传输

在早期，内存控制器集成在北桥芯片中。现在，为了降低延迟，内存控制器通常集成在CPU内部。

DRAM的时序：

DRAM的访问涉及多个步骤，需要精确的时序控制。主要参数包括：

- CAS延迟（CL）：列地址选通延迟，即从发出读命令到数据可用的时间
- tRCD：行地址到列地址延迟
- tRP：行预充电时间

DDR SDRAM：

DDR（Double Data Rate）是一种内存技术，在时钟的上升沿和下降沿都传输数据，所以数据传输率是时钟频率的两倍。

DDR技术经历了多代发展：

- DDR1：2000年，传输频率200-400MHz
- DDR2：2003年，传输频率400-800MHz
- DDR3：2007年，传输频率800-2133MHz
- DDR4：2014年，传输频率2133-3200MHz

- DDR5：2020年，传输频率4800-6400MHz

2.3.7 虚拟内存

虚拟内存（Virtual Memory）是操作系统提供的一种内存管理技术，它让程序认为它有一大片连续的地址空间（虚拟地址空间）。
为什么需要虚拟内存：

1. 程序可以使用连续的地址：没有虚拟内存时，程序需要自己管理物理内存的分配和地址，容易出现碎片化和地址冲突。
2. 程序间相互隔离：有了虚拟内存，不同程序的虚拟地址可以映射到不同的物理地址，实现程序间的内存隔离，提高安全性。
3. 物理内存可以小于虚拟地址空间：通过**swapping（换页）**技术，不常用的内存页可以暂时写入磁盘，等需要时再换回内存。
4. 内存利用率更高：操作系统可以根据需要动态分配物理内存，减少浪费。

虚拟地址到物理地址的转换：

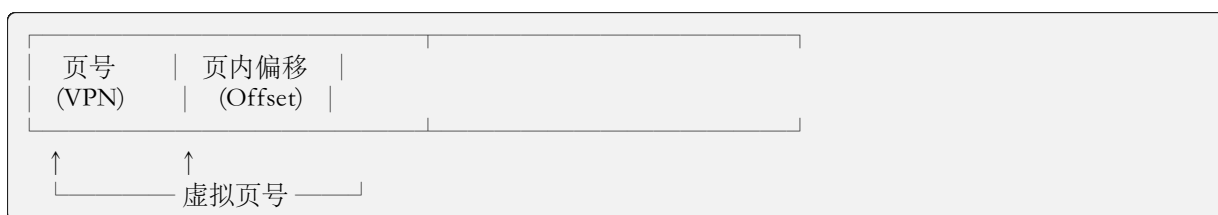
虚拟地址到物理地址的转换由**内存管理单元（Memory Management Unit, MMU）**完成。

MMU通过**页表（Page Table）**来存储虚拟地址到物理地址的映射关系。每个程序有自己的页表。

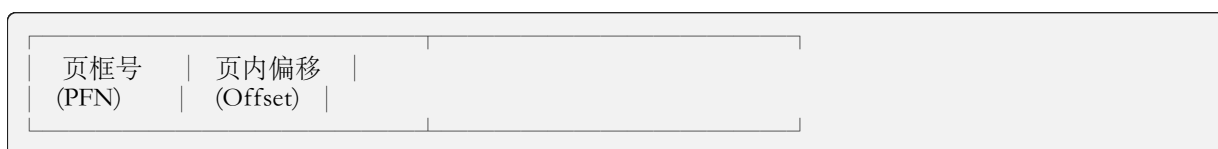
分页（Paging）是常见的虚拟内存管理方式：

- 虚拟地址空间被划分为固定大小的页（Page）
- 物理内存被划分为相同大小的页框（Page Frame）
- 页表记录每个虚拟页对应的物理页框

虚拟地址结构：



物理地址结构：



转换时，MMU根据虚拟地址的页号查页表，得到对应的物理页框号，再加上页内偏移，就得到了物理地址。

TLB（Translation Lookaside Buffer）：

每次地址转换都要查页表，而页表可能很大，导致多次内存访问。为了加速地址转换，CPU引入了TLB，这是一个专门缓存最近使用过的虚拟地址到物理地址映射的表。

TLB的工作方式：

- CPU给出虚拟地址
- MMU首先查TLB，看是否有该虚拟地址的映射
- 如果命中（TLB Hit），直接获得物理地址
- 如果未命中（TLB Miss），查页表，获得映射，同时更新TLB

2.3.8 局部性原理详解

局部性原理（Principle of Locality）是缓存技术和虚拟内存技术的理论基础。它表明程序在执行过程中，对内存的访问具有局部性。

时间局部性（Temporal Locality）：

同一个内存位置，在短期内可能被多次访问。

典型例子：

- 循环中的循环变量，每次迭代都要检查和更新
- 函数的返回地址，调用后很快就会用到
- 常用的配置数据

```
// 时间局部性示例
int sum = 0;
for (int i = 0; i < N; i++) {
    sum += arr[i]; // i在每次迭代都被访问
}
```

空间局部性（Spatial Locality）：

当某个内存位置被访问时，相邻的内存位置很可能也会被访问。

典型例子：

- 数组元素依次访问
- 顺序执行的代码（指令在内存中是连续存放的）
- 结构体的成员（通常连续存放在内存中）

```
// 空间局部性示例
for (int i = 0; i < N; i++) {
    sum += arr[i]; // arr[i+1] 在 arr[i] 之后被访问
}
```

评估局部性：

了解局部性原理对于编写高效代码很重要：

- 数组应该尽量顺序访问，避免跳跃式访问
- 嵌套循环应该尽量让内层循环访问的数据连续
- 频繁访问的数据应该尽量保持在缓存中

本节小结

- 内存层次结构是为了解决速度、容量和成本之间的矛盾
- 寄存器是速度最快、容量最小的存储，位于CPU内部
- Cache利用局部性原理，减少对主存的访问，是CPU和主存之间的缓冲区
- 多级缓存（L1/L2/L3）提供不同速度和容量的缓存层次
- 主存（DRAM）容量较大但速度较慢，需要内存控制器访问
- 虚拟内存让程序使用连续的虚拟地址空间，由MMU通过页表转换
- 局部性原理包括时间局部性和空间局部性，是缓存技术的理论基础

2.4 CPU与内存交互

2.4.1 总线概述

****总线（Bus）****是计算机中各部件传输数据的共享通道。就像城市里的主干道，总线是连接CPU、内存、I/O设备的“高速公路”。

总线的特点：

- 共享性：多个设备可以连接到同一条总线，但同一时刻只能有一个设备使用

- 方向性：数据总线是双向的（可以发送也可以接收），地址总线通常是单向的（由CPU发出）
- 速度匹配：总线速度必须满足连接设备中速度最慢的那个

总线的分类：

1. 系统总线（System Bus）：连接CPU和内存，包括：
 - 数据总线（Data Bus）：传输数据，宽度（如64位）影响一次传输的数据量
 - 地址总线（Address Bus）：传输内存地址，宽度（如32位、64位）决定可寻址空间大小
 - 控制总线（Control Bus）：传输控制信号，如读/写信号、时钟信号、中断信号
2. I/O总线：连接CPU和外部设备，如PCIe、USB、SATA等
3. 外部总线：连接不同设备，如以太网、串口等

2.4.2 数据传输过程

当CPU需要从内存读取数据时，数据传输过程如下：

读操作：

1. CPU将内存地址放到地址总线上
2. CPU发出读控制信号（RD）
3. 内存控制器检测到地址和控制信号，从指定地址读取数据
4. 内存将数据放到数据总线上
5. CPU从数据总线读取数据并存入寄存器

写操作：

1. CPU将内存地址放到地址总线上
2. CPU将数据放到数据总线上
3. CPU发出写控制信号（WR）
4. 内存控制器检测到地址、控制信号和数据，将数据写入指定地址

总线的带宽：

总线带宽是指单位时间内可以传输的数据量，计算公式是：

$$\text{带宽} = \text{总线频率} \times \text{总线宽度}$$

例如，一个64位（8字节）宽度的总线，工作频率为100MHz，则理论带宽是：

$$100\text{MHz} \times 8\text{字节} = 800\text{MB/s}$$

2.4.3 直接内存访问（DMA）

****DMA（Direct Memory Access，直接内存访问）****是一种允许外部设备直接与内存交换数据的技术，无需CPU介入。

为什么需要DMA：

在没有DMA的时代，外设与内存交换数据需要CPU“居中协调”：

- 外设数据先读到CPU寄存器
- 再从CPU寄存器写到内存

这称为程序控制I/O（Programmed I/O）。问题是，如果传输大量数据（如从硬盘读取文件），CPU就会浪费大量时间在

DMA的工作原理：

1. CPU配置DMA控制器：
 - 设置源地址（数据从哪里来）

- 设置目标地址（数据到哪里去）
 - 设置传输的数据量
 - 启动DMA传输
2. DMA控制器接管总线，开始传输数据
 - 数据从外设直接传输到内存（或反之）
 - CPU此时可以执行其他任务
 3. 传输完成后，DMA控制器向CPU发出中断信号
 - CPU知道数据传输已完成
 - CPU可以处理这些数据

DMA的优势：

- CPU可以在数据传输期间执行其他计算任务
- 提高系统整体效率
- 减少CPU负载

DMA的应用场景：

- 磁盘I/O：硬盘读写
- 网络数据传输：网卡收发的数据包
- 音频/视频播放：音频/视频数据流
- 显卡显存更新：GPU与显存之间的数据交换

2.4.4 缓存一致性

在具有缓存的系统中，一个重要的问题是缓存一致性（Cache Coherence）。

问题场景：

- CPU和DMA控制器都访问同一块内存
- CPU可能已经把某个内存位置的数据加载到缓存
- 如果DMA直接修改了主存中的数据，而CPU还在使用旧缓存中的数据，就会产生不一致

缓存一致性的解决方案：

1. 缓存直写（Write-Through）：每次写操作同时更新缓存和主存
 - 主存数据始终是最新的
 - 但会降低写性能
2. 缓存失效（Cache Invalidation）：当DMA要访问某块内存时，通知CPU使相关缓存行失效
 - CPU下次访问时必须从主存重新读取
 - 需要硬件支持（如嗅探协议）
3. 缓存同调（Cache Coherent）协议：如MESI协议，确保缓存和主存的数据一致
 - MESI：Modified（修改）、Exclusive（独占）、Shared（共享）、Invalid（无效）
 - 当某个CPU修改数据时，其他CPU的相关缓存行会被无效或更新

2.4.5 内存保护

操作系统需要保护内存，防止恶意或错误的程序访问不该访问的内存区域。

内存保护机制：

1. 特权级别（Privilege Level）：
 - CPU支持多个特权级别（如Ring 0到Ring 3）
 - 操作系统运行在最高特权级（Ring 0）
 - 用户程序运行在较低特权级（Ring 3）
 - 特权指令只能在高特权级执行
2. 段保护（Segment Protection）：
 - 每个内存段有访问权限（如只读、读写、执行）
 - 尝试违反权限的操作会触发异常
3. 页保护（Page Protection）：
 - 每个内存页有访问权限
 - 页表项中包含读/写/执行权限位
 - 尝试违反权限的操作会触发页错误（Page Fault）

2.4.6 中断机制

****中断（Interrupt）****是计算机中一种重要的异步通信机制，允许外部设备通知CPU需要处理的事件。

中断的类型：

1. 硬件中断（Hardware Interrupt）：
 - 由外部设备发出，如键盘按键、鼠标移动、定时器到期、网络数据包到达
 - 通过中断控制器（如Intel的8259或现代的APIC）路由到CPU
2. 软件中断（Software Interrupt）：
 - 由程序执行特定指令触发
 - 如x86的int 0x80（系统调用）
3. 异常（Exception）：
 - 由CPU在执行指令时检测到错误情况产生
 - 如除零、非法指令、页错误

中断的处理过程：

1. 外设发出中断请求（IRQ）
2. 中断控制器收集并优先级排序所有IRQ
3. 中断控制器向CPU发送中断信号
4. CPU完成当前指令（取决于中断类型）
5. CPU保存当前执行上下文（寄存器、PC等）到栈
6. CPU根据中断号查找中断向量表，获得中断处理程序地址
7. CPU跳转到中断处理程序执行
8. 中断处理完成后，恢复保存的上下文，继续执行被中断的代码

2.4.7 时钟和同步

****时钟（Clock）****是计算机系统的时间基准，用于同步各部件的工作。

时钟的作用：

- 为CPU提供基本的时序参考
- 决定CPU的执行频率
- 同步总线和数据传输时序
- 为定时器提供基准

同步与异步：

- 同步系统：所有部件使用统一的时钟信号协同工作
- 异步系统：部件之间通过握手信号协调，不需要统一时钟

现代计算机通常是同步的（有全局时钟），但某些总线和数据传输可能是异步的（通过握手协议协调）。

本节小结

- 总线是连接各部件的数据通道，分为系统总线、I/O总线等
- 数据传输涉及地址、数据和控制信号的交互
- DMA允许外设直接与内存交换数据，减轻CPU负担
- 缓存一致性确保多级缓存和主存的数据一致
- 内存保护通过特权级别、段保护和页保护实现
- 中断是外设通知CPU的异步机制，用于处理紧急事件
- 时钟为计算机系统提供统一的时间基准

本章小结

本章深入介绍了计算机的核心组成——CPU和内存的工作原理。

冯诺依曼架构是现代计算机的基础，由运算器、控制器、存储器、输入设备和输出设备五部分组成。”存储程序”概念允许

CPU的结构包括：

- ALU负责算术和逻辑运算
- 控制单元负责指令的取指、解码和执行控制
- 寄存器用于快速存储数据，是CPU内部最快速的存储

指令执行流程是”取指-解码-读取操作数-执行-写回”的循环，CPU通过程序计数器来跟踪下一条要执行的指令。

内存层次结构解决了速度、容量和成本的矛盾：

- 寄存器速度最快但容量最小
- 多级缓存利用局部性原理加速数据访问
- 主存（DRAM）提供大容量存储
- 虚拟内存通过页表和MMU实现地址转换和内存保护

CPU与内存交互通过总线进行，DMA允许外设直接访问内存而不需要CPU介入。缓存一致性、内存保护、中断机制等都是理解CPU和内存的工作原理，对于理解计算机系统的行为和编写高效程序都至关重要。

思考与练习

1. 冯诺依曼架构
 - 画出冯诺依曼架构的框图，标出五大组成部分及其关系。
 - 解释什么是“冯诺依曼瓶颈”，并列举两种缓解该瓶颈的技术。
2. CPU结构
 - 描述CPU中ALU和控制单元的功能，并说明它们如何协同工作。
 - 为什么说寄存器是内存层次结构中最快的一层？
3. 指令执行
 - 详细描述执行`mov eax, [0x1000]`这条指令（从内存读取数据到寄存器）的完整过程。
 - 程序计数器（PC）的作用是什么？什么情况下PC的值会改变？
4. 内存层次
 - 解释时间局部性和空间局部性的含义，并各举一个程序中的例子。
 - 为什么L1 Cache通常分为指令缓存和数据缓存两个部分？
 - 如果L1 Cache未命中，CPU接下来会检查哪里？
5. 虚拟内存
 - 虚拟内存解决了哪些问题？请列出至少三个。
 - 假设一个系统使用32位虚拟地址，页大小为4KB。请计算虚拟地址空间的大小，以及虚拟页号占多少位。
6. 总线与DMA
 - 解释总线带宽的概念。如果总线宽度为64位，频率为100MHz，理论带宽是多少？
 - DMA相对于程序控制I/O有什么优势？请解释DMA的工作流程。
7. 缓存一致性
 - 考虑这个场景：CPU A和CPU B共享一块内存，CPU A修改了该内存中的数据，但CPU B不知道修改发生，仍然使用旧数据。这种问题应该怎么解决？
8. 中断机制
 - 什么是中断？中断在计算机系统有什么作用？
 - 描述CPU处理一个硬件中断的完整过程。

第三章 程序运行原理

学习目标

- 理解从源代码到可执行程序的全过程：编译、链接、加载
 - 掌握进程和线程的概念，理解它们的区别和联系
 - 理解并发与并行的区别
 - 掌握操作系统在程序运行中的角色
 - 了解系统调用的概念和常用系统调用
-

3.1 从源代码到可执行程序

3.1.1 程序的生命周期

当我们编写一个程序时，从键盘敲下第一行代码，到程序最终在计算机上运行，经历了多个阶段。这个过程可以概括为：源代码 → 预处理器 → 编译器 → 汇编器 → 目标文件 → 链接器 → 可执行文件 → 加载器 → 进程

这个看似简单的流程，实际上涉及了多个复杂的转换步骤。理解这些转换过程，对于理解计算机系统的工作方式至关重要。

3.1.2 编译过程详解

编译（Compilation）是将高级语言（如C、C++、Rust）转换为机器能理解的形式的第一步。但“编译”这个术语实际

1. 预处理（Preprocessing）

预处理是编译的第一个阶段，负责处理源代码中的预处理指令。

预处理指令以#开头，常见的包括：

- #include：包含头文件内容
- #define：定义宏
- #if、#ifdef、#endif：条件编译

例如，对于以下代码：

```
#define PI 3.14159
#include <stdio.h>

int main() {
    printf("Hello, PI is %f\n", PI);
    return 0;
}
```

预处理后，#define PI 3.14159会被替换为实际的数值，#include <stdio.h>会被stdio.h文件的内容替换。

2. 编译（Compiling）

编译阶段将预处理后的源代码转换为汇编语言。这一阶段是整个编译过程中最复杂的部分。

编译阶段主要包括：

- 词法分析（Lexical Analysis）：将源代码分割成一个个的标记（Token），如关键字、标识符、运算符等
- 语法分析（Syntax Analysis）：将标记组织成语法树，检查语法是否正确
- 语义分析（Semantic Analysis）：检查类型是否匹配、变量是否声明等语义问题
- 优化（Optimization）：对代码进行各种优化，如常量折叠、死代码消除、循环优化等
- 代码生成（Code Generation）：生成目标平台的汇编代码

3. 汇编（Assembly）

汇编阶段将汇编语言转换为机器码，生成目标文件（Object File）。

目标文件是二进制格式，包含：

- 机器指令
- 程序数据
- 符号表（Symbol Table）：记录程序中的变量和函数名及其地址
- 调试信息
- 重定位信息

不同平台的目标文件格式不同：

- Linux/Unix: ELF (Executable and Linkable Format)
- Windows: PE (Portable Executable)
- macOS: Mach-O

3.1.3 链接过程

****链接 (Linking) ****是将多个目标文件合并成一个可执行文件的过程。

为什么需要链接？

现代程序通常由多个源文件组成，每个源文件独立编译成目标文件。链接器负责：

- 合并所有目标文件
- 解析符号引用（一个文件中引用的函数/变量可能在另一个文件中定义）
- 确定每个符号的最终地址
- 决定内存布局

链接的两种类型：

1. 静态链接 (Static Linking)

静态链接在编译时完成，所有代码被合并到一个可执行文件中。

优点：

- 可执行文件是独立的，不依赖外部库
- 加载速度快（无需在运行时加载库）

缺点：

- 可执行文件体积较大
- 如果多个程序使用同一个库，每个程序都有一份库的副本
- 更新库需要重新编译所有程序

2. 动态链接 (Dynamic Linking)

动态链接在程序加载或运行时完成，库代码不直接写入可执行文件，而是记录对库的引用。

优点：

- 可执行文件体积较小
- 多个程序共享同一个库的代码，节省内存
- 更新库不需要重新编译使用该库的程序

缺点：

- 加载速度稍慢（需要在运行时解析符号）
- 如果库文件缺失或版本不匹配，程序无法运行

动态链接库：

- Linux: .so文件 (Shared Object)
- Windows: .dll文件 (Dynamic Link Library)
- macOS: .dylib文件

符号解析 (Symbol Resolution)：

链接器需要解析各种符号引用：

- 外部符号 (External Symbol)：在一个目标文件中引用，但在另一个目标文件中定义
- 强符号 (Strong Symbol)：已初始化的全局变量和函数

- 弱符号（Weak Symbol）：未初始化的全局变量

链接规则：

- 每个符号只能有一个定义
- 如果有多个强符号，会产生链接错误（重复定义）
- 弱符号可以被强符号覆盖

3.1.4 可执行文件格式

****ELF（Executable and Linkable Format）****是Linux和许多Unix系统使用的可执行文件格式。理解ELF格式有助于理解程序

ELF文件结构：

ELF Header	← 文件基本信息：入口点、段表位置等
Program Header Table	← 加载信息（运行时需要）
.text	← 代码段：机器指令
.rodata	← 只读数据段：常量字符串等
.data	← 已初始化数据段：全局变量等
.bss	← 未初始化数据段
...其他段...	
Section Header Table	← 段表（链接时需要）

ELF Header包含：

- Magic number：标识ELF文件
- 程序入口点（Entry Point）
- Program Header Table的位置和大小
- Section Header Table的位置和大小
- 等等

Program Header Table告诉加载器如何加载文件：

- 每个表项描述一个段（Segment）
- 包括段的类型、文件中的偏移、虚拟地址、权限（读/写/执行）等

常见的段（Section）：

- .text：存放可执行指令
- .data：存放已初始化的全局和静态变量
- .bss：存放未初始化的全局和静态变量（不占文件空间，只预留位置）
- .rodata：存放只读数据，如字符串常量
- .symtab：符号表
- .strtab：字符串表
- .debug：调试信息

3.1.5 加载可执行文件

当我们运行一个程序时，**加载器（Loader）**负责将可执行文件加载到内存中，并开始执行。

Linux下的加载过程：

- 1. shell调用execve系统调用
- 2. 内核检查可执行文件格式（如ELF）
- 3. 读取ELF Header，获取入口点地址
- 4. 根据Program Header Table，将文件的各个段映射到虚拟内存
- 5. 创建栈、设置参数和环境变量
- 6. 设置寄存器，将控制权交给入口点

内存布局：

程序加载后，在虚拟内存中的典型布局：



本节小结

- 程序从源代码到可执行文件需要经过预处理、编译、汇编、链接四个阶段
- 预处理处理宏包含和条件编译，编译进行词法分析、语法分析、语义分析、优化和代码生成
- 链接将多个目标文件合并成可执行文件，解析符号引用
- 静态链接在编译时完成，动态链接在运行时完成
- ELF是Linux常用的可执行文件格式，包含ELF Header、Program Header Table和各种段
- 加载器负责将可执行文件加载到内存，并开始执行程序

3.2 进程与线程

3.2.1 进程的概念

进程（Process）是操作系统中一个非常重要的概念。它是程序执行的实例，是操作系统进行资源分配和调度的基本单位。要理解进程，我们需要从两个角度来看：

- 从用户角度看：进程是运行中的程序，是一个活动的实体
- 从操作系统角度看：进程是一个资源容器的数据结构，包含程序运行所需的所有资源

进程的组成：

每个进程都有自己独立的虚拟地址空间、寄存器状态、程序计数器、栈、堆等资源。

进程的数据结构（进程控制块/PCB）

- PID（进程标识符）
- 状态（运行、就绪、阻塞等）
- 程序计数器
- 寄存器状态
- 内存管理信息（页表/段表）
- 打开的文件列表
- 信号处理方式
- 资源使用信息
- ...其他信息

进程的状态：

进程在生命周期内会处于不同的状态：

- 创建（New）：进程正在被创建
- 就绪（Ready）：进程已准备好运行，等待CPU分配
- 运行（Running）：进程正在CPU上执行
- 阻塞（Blocked）：进程因等待某个事件（如I/O完成）而暂停
- 终止（Terminated）：进程执行完毕

状态转换：

- 新进程创建 → 就绪
- 调度程序选择该进程 → 运行
- 时间片用完 → 就绪
- 进程请求I/O → 阻塞
- I/O完成 → 就绪
- 进程结束 → 终止

3.2.2 进程的创建

在Linux系统中，进程的创建通过fork系统调用完成。

fork的作用：

fork创建一个新的进程，称为子进程。调用fork的进程是父进程。

fork的特殊之处在于：它被调用一次，但返回两次——在父进程中返回子进程的PID，在子进程中返回0。

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("我是子进程，PID=%d\n", getpid());
    } else {
        printf("我是父进程，子进程PID=%d\n", pid);
    }
}
```

(continues on next page)

(continued from previous page)

```
}  
  
return 0;  
}
```

执行结果：

```
我是父进程，子进程PID=12345  
我是子进程，PID=12346
```

fork之后发生了什么：

fork之后，父进程和子进程拥有：

- 相同的代码段
- 相同的全局变量和堆（复制）
- 独立的栈空间（复制）
- 独立的寄存器状态和程序计数器

子进程是父进程的副本，但它们是独立的进程，有各自的虚拟地址空间（尽管内容相同）。对子进程的修改不会影响父进程。

3.2.3 进程的终止

进程可以通过多种方式终止：

正常终止：

- main函数返回
- 调用exit()函数
- 调用_exit()系统调用

异常终止：

- 收到信号（如SIGSEGV、SIGINT）
- 调用abort()函数
- 未处理的异常

进程退出的资源回收：

当进程终止时，它并不会立即消失，而是变成一个僵尸进程（Zombie Process）。僵尸进程保留进程的退出状态等信息，等待父进程调用wait系统调用来收集这些信息。

如果父进程没有调用wait，子进程就会一直保持僵尸状态。但init进程（PID=1）会定期回收孤儿进程的僵尸。

孤儿进程（Orphan Process）：

如果父进程先于子进程终止，子进程就会变成孤儿进程，被init进程（PID=1）收养。init会负责回收这些子进程。

3.2.4 线程的概念

****线程（Thread）****是进程内的执行单元，是CPU调度的基本单位。

在引入线程之前，进程是调度的基本单位。但进程有一个问题：同一进程内的多个任务无法共享地址空间，如果要共享资源，线程的引入解决了这个问题：同一进程内的多个线程共享：

- 地址空间
- 全局变量
- 堆
- 打开的文件

- 信号处理函数

但每个线程有自己独立的：

- 程序计数器
- 寄存器状态
- 栈
- errno（错误码）

3.2.5 进程与线程的区别

特性	进程	线程
创建速度	较慢（需要复制整个地址空间）	较快（可以共享进程资源）
通信速度	较慢（需要IPC机制）	较快（直接共享内存）
隔离性	完全隔离，一个进程崩溃不影响其他进程	共享地址空间，一个线程崩溃可能导致整个进程崩溃
资源占用	较多（每个进程有独立地址空间）	较少（共享大部分资源）
同步复杂度	较低（独立地址空间天然隔离）	较高（需要显式同步，避免竞态条件）

3.2.6 线程的创建与同步

创建线程（以POSIX线程为例）：

```
#include <pthread.h>
#include <stdio.h>

void* thread_function(void* arg) {
    int* num = (int*)arg;
    printf("线程运行，参数=%d\n", *num);
    return NULL;
}

int main() {
    pthread_t thread;
    int arg = 42;

    // 创建线程
    pthread_create(&thread, NULL, thread_function, &arg);

    // 等待线程结束
    pthread_join(thread, NULL);

    return 0;
}
```

线程同步：

由于多个线程共享地址空间，如果不加以控制，并发访问共享数据会导致竞态条件（Race Condition）。

常见的线程同步机制：

1. 互斥锁（Mutex）

互斥锁用于保护共享资源，一次只允许一个线程访问。

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&mutex);
```

(continues on next page)

(continued from previous page)

```
// 访问共享资源
pthread_mutex_unlock(&mutex);
```

2. 信号量 (Semaphore)

信号量是一个计数器，用于控制对共享资源的访问数量。

```
sem_t sem;
sem_init(&sem, 0, 1); // 初始值为1

sem_wait(&sem); // 减1，如果为0则阻塞
// 访问共享资源
sem_post(&sem); // 加1
```

3. 条件变量 (Condition Variable)

条件变量用于线程间的等待和通知机制。

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

pthread_mutex_lock(&mutex);
while (some_condition_is_false) {
    pthread_cond_wait(&cond, &mutex); // 解锁mutex并等待
}
// 条件满足，处理
pthread_mutex_unlock(&mutex);
```

3.2.7 线程的状态

线程与进程类似，也有多种状态：

- 创建：线程正在被创建
- 就绪：线程已准备好运行，等待CPU
- 运行：线程正在CPU上执行
- 阻塞：线程因等待某个条件而暂停
- 终止：线程执行完毕

线程的创建和终止比进程快，因为线程不需要创建新的地址空间。

本节小结

- 进程是程序执行的实例，是操作系统资源分配的基本单位
- 进程有独立的地址空间，通过fork创建，通过exit/_exit终止
- 线程是进程内的执行单元，是CPU调度的基本单位
- 同一进程的多个线程共享地址空间，但有独立的栈和寄存器
- 线程同步机制包括互斥锁、信号量和条件变量，用于防止竞态条件
- 进程和线程各有优缺点，选择取决于具体应用场景

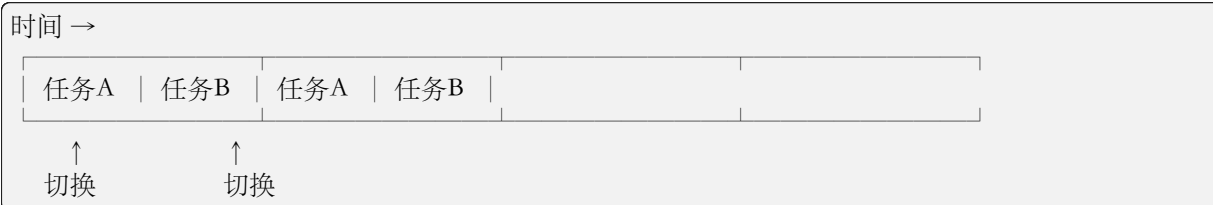
3.3 并发与并行

3.3.1 并发与并行的概念

****并发（Concurrency）和并行（Parallelism）****是两个容易混淆但含义不同的概念。

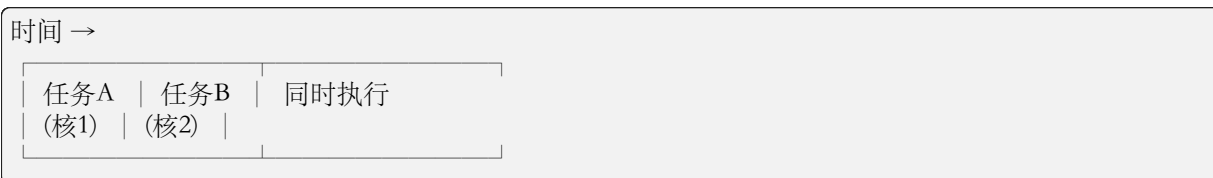
并发（Concurrency）：

并发是指多个任务在同一时间段内交替执行。在单核CPU上，由于只有一个核心，操作系统通过快速切换任务，让人感觉



并行（Parallelism）：

并行是指多个任务在同一时刻同时执行。这需要多核CPU或多台计算机，每个处理单元同时执行不同的任务。



简单类比：

- 并发就像一个厨师同时管理多个锅，通过切换注意力和时间来“同时”处理多个烹饪任务
- 并行就像多个厨师各自负责一个锅，真正同时地处理多个任务

3.3.2 并发的应用场景

并发不是为了加快执行速度，而是为了：

- 提高资源利用率：当一个任务等待I/O时，CPU可以切换到另一个任务
- 提高系统响应性：如GUI程序中，后台任务不会阻塞用户界面
- 简化问题模型：某些问题天然适合分解为多个独立或半独立子任务

I/O密集型任务的并发：

对于涉及大量I/O操作的任务（如Web服务器），并发可以显著提高吞吐量。

单线程顺序处理：

请求1 → [等待I/O] → 处理 → 响应1
 请求2 → [等待I/O] → 处理 → 响应2
 ...
 总时间 = $N \times (\text{I/O时间} + \text{处理时间})$

多线程/异步并发处理：

请求1 → [等待I/O]
 请求2 → [等待I/O] → 处理 → 响应2
 请求3 → [等待I/O] → 处理 → 响应3
 ...
 总时间 $\approx N \times \text{I/O时间} + \text{处理时间}$ （重叠I/O等待）

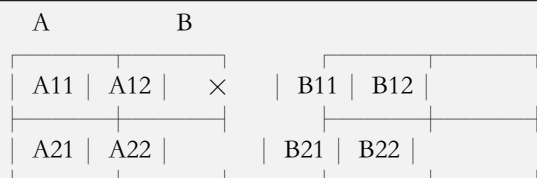
3.3.3 并行的应用场景

并行主要用于计算密集型任务，通过多核同时计算来加速执行。

并行计算的例子：

1. 矩阵乘法

对于大型矩阵乘法，可以将矩阵分块，分配给不同CPU核心计算：



A11和B11的乘法可以分配给核心1，A12和B21的乘法可以分配给核心2，以此类推。

2. 图像处理

图像的每个像素处理通常相互独立，可以将图像分成多个区域，并行处理。

3. 科学计算

气候模拟、流体力学模拟、分子动力学等科学计算，需要大量浮点运算，并行化可以显著加速。

3.3.4 并发与并行的实现方式

多进程并发：

进程A $\longleftrightarrow$ 进程B $\longleftrightarrow$ 进程C
(通过IPC通信，如管道、消息队列、共享内存)

优点：隔离性好，一个进程崩溃不影响其他进程 缺点：通信开销大（需要复制数据）

多线程并发：

线程A $\longleftrightarrow$ 线程B $\longleftrightarrow$ 线程C
(直接共享内存地址空间)

优点：通信效率高（直接访问共享内存） 缺点：需要显式同步，一个线程崩溃可能导致整个进程崩溃

异步编程：

异步编程是一种单线程并发模型，通过事件循环和回调函数实现。

```
# Python异步IO示例
import asyncio

async def fetch(url):
    # 发起网络请求，但不阻塞
    response = await request.get(url)
    return response

async def main():
    # 并发发起多个请求
    results = await asyncio.gather(
        fetch("http://example.com/1"),
        fetch("http://example.com/2"),
        fetch("http://example.com/3"),
    )

asyncio.run(main())
```

3.3.5 并发的问题与挑战

并发编程比顺序编程复杂得多，因为并发会引入新的问题：

1. 竞态条件（Race Condition）

当多个线程并发访问和修改共享数据时，最终结果取决于线程的执行顺序，就发生了竞态条件。

示例：

```
// 共享变量
int counter = 0;

// 线程A执行的函数
void* thread_a(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++; // 读取、+1、写回
    }
    return NULL;
}

// 线程B执行的函数
void* thread_b(void* arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++; // 读取、+1、写回
    }
    return NULL;
}
```

期望结果：counter = 2000000 但由于竞态条件，实际结果往往小于2000000。

2. 死锁（Deadlock）

死锁是指两个或多个线程互相等待对方释放资源，导致都无法继续执行。

示例：

线程A：获取锁1，等待锁2
线程B：获取锁2，等待锁1

结果：两个线程都在等待对方，永远无法继续

死锁产生的四个必要条件（咖啡馆定律）：

- 互斥：资源每次只能被一个线程使用
- 持有并等待：线程持有资源的同时请求其他资源
- 不可抢占：资源不能被强制从线程中夺走
- 循环等待：存在一个线程循环等待链

避免死锁的方法：

- 破坏其中一个条件
- 固定顺序获取锁
- 使用try_lock而非lock，设置超时

3. 活锁（Livelock）

活锁是指线程一直在运行，但无法取得进展。与死锁不同，活锁中的线程不是在等待，而是不断重复相同的操作。

例如：两辆车在狭窄的桥上相遇，双方都礼让对方，但由于礼让动作相同，双方都无法通过。

4. 饥饿（Starvation）

饥饿是指某些线程由于各种原因无法获得所需的资源，一直无法执行。

例如：如果系统总是调度高优先级任务，低优先级任务可能永远得不到CPU时间。

3.3.6 Amdahl定律

Amdahl定律是并行计算中的重要定律，用于计算并行化能达到的最大加速比。

假设程序中有P比例的部分可以并行化，(1-P)比例的部分必须串行执行。

- 单核执行时间：T
- 并行执行时间： $T_{parallel} = T \times (1 - P) + \frac{T \times P}{N}$

其中N是核心数量。

加速比 (Speedup)：

$$S = \frac{T}{T_{parallel}} = \frac{1}{(1 - P) + \frac{P}{N}}$$

举例：

如果90%的代码可以并行化 ($N \rightarrow \infty$)： $S = \frac{1}{0.1+0} = 10$

即使无限多的核心，加速比最多也只有10倍。

启示：

Amdahl定律告诉我们，并行化的收益是有限的。要提高加速比：

- 尽量增加可并行化的比例P
- 减少串行部分
- 但当P接近100%时，加速比主要受核心数量N限制

本节小结

- 并发是多个任务在同一时间段内交替执行（单核），并行是多个任务在同一时刻同时执行（多核）
 - 并发提高资源利用率和系统响应性，并行提高计算速度
 - 并发实现方式包括多进程、多线程和异步编程
 - 并发编程面临竞态条件、死锁、活锁、饥饿等挑战
 - Amdahl定律指出并行化的加速比受串行部分限制
-

3.4 操作系统与系统调用

3.4.1 操作系统的作用

****操作系统 (Operating System, OS) ****是计算机系统中最基础的系统软件，它是用户和硬件之间的桥梁。

操作系统的核心功能：

1. 资源管理
 - CPU调度：为多个进程分配CPU时间
 - 内存管理：分配和回收内存，实现虚拟内存
 - 设备管理：管理各种I/O设备
 - 文件系统管理：组织和管理磁盘上的数据
2. 提供服务
 - 进程管理：创建、终止、调度进程

- 进程间通信：提供进程间通信机制
- 保护和安全：隔离进程，保护资源

3. 抽象接口

- 将硬件抽象为更易用的接口
- 提供统一的系统调用接口

3.4.2 内核与用户态

现代操作系统通常采用**微内核（Microkernel）或宏内核（Monolithic Kernel）**设计。

宏内核（如Linux）：

将大多数操作系统服务（进程管理、内存管理、文件系统、设备驱动等）都运行在内核态（Kernel Mode）。

优点：

- 性能好（内核内部函数调用速度快）
- 各组件配合紧密

缺点：

- 内核臃肿（包含大量功能）
- 一个组件的问题可能影响整个系统

微内核（如MINIX）：

只将最核心的功能（调度、进程间通信）放在内核态，其他服务（文件系统、设备驱动）运行在用户态，作为独立进程。

优点：

- 系统更稳定（一个服务崩溃不会影响其他服务）
- 内核小巧

缺点：

- 组件间通信（IPC）开销大
- 性能稍差

用户态与内核态：

CPU有特权级别（Privilege Level）或Ring（如Ring 0到Ring 3）：

- 内核态（Ring 0）：可以执行特权指令，访问所有硬件
- 用户态（Ring 3）：只能访问有限的资源，不能直接执行特权指令

用户程序运行在用户态，当需要操作系统服务时，通过系统调用进入内核态。

3.4.3 系统调用概述

系统调用（System Call）是用户程序请求操作系统服务的接口。

当用户程序需要：

- 读取/写入文件
- 创建新进程
- 分配内存
- 进行网络通信
- 其他需要特权的操作

必须通过系统调用请求内核完成。

系统调用与普通函数调用的区别：

普通函数调用：

- 用户代码 → 用户库函数
- 在同一地址空间执行
- 只是一次普通的函数跳转

系统调用：

- 用户代码 → 用户库函数 → 触发软中断/syscall指令
- 切换到内核态
- 内核代码执行请求的操作
- 返回用户态
- 返回用户代码

3.4.4 常见系统调用

进程控制：

- `fork()`：创建新进程
- `execve()`：执行新程序
- `exit()`：终止进程
- `wait()`：等待子进程终止
- `getpid()`：获取进程ID
- `getppid()`：获取父进程ID

文件操作：

- `open()`：打开文件
- `read()`：读取文件
- `write()`：写入文件
- `close()`：关闭文件
- `lseek()`：移动文件指针
- `unlink()`：删除文件

内存管理：

- `brk()`：改变数据段大小
- `mmap()`：映射内存
- `munmap()`：解除内存映射

进程通信：

- `pipe()`：创建管道
- `socket()`：创建套接字
- `connect()`：连接套接字
- `send()/recv()`：发送/接收数据

时间相关：

- `time()`：获取当前时间

- `alarm()`: 设置闹钟
- `sleep()`: 进程睡眠

3.4.5 系统调用的实现

Linux系统调用过程:

1. 用户程序调用C库函数 (如`read`)
2. C库准备系统调用参数 (如文件描述符、缓冲区地址、字节数)
3. C库执行`syscall`指令 (或`int 0x80`软中断)
4. CPU切换到内核态, 跳转到内核设定的系统调用入口
5. 内核根据系统调用号, 调用相应的内核函数
6. 内核执行操作, 将结果返回
7. CPU切换回用户态, 返回到C库
8. C库返回到用户程序

系统调用号:

每个系统调用都有一个唯一的编号 (系统调用号), 如:

- Linux x86-64:
 - `read` = 0
 - `write` = 1
 - `open` = 2
 - `close` = 3
 - `fork` = 57
 - `execve` = 59
 - `exit` = 60
 - `getpid` = 39

系统调用表:

内核维护一个系统调用表 (`sys_call_table`), 通过系统调用号索引, 存储对应内核函数的地址。

3.4.6 C库与系统调用

大多数情况下, 程序员不直接使用系统调用, 而是通过**C标准库 (`libc`) **来间接使用。

例如, `printf`函数内部会调用`write`系统调用来输出数据:

用户代码 → `printf()` → `write()` → `syscall` → 内核

使用C库的好处:

- 接口更友好 (比系统调用更易用)
- 可移植性 (不同操作系统的系统调用不同, 但C库提供统一接口)
- 功能更丰富 (如`printf`的格式化功能是C库实现的, 不是系统调用)

3.4.7 操作系统启动过程

理解操作系统启动过程有助于理解系统各组件如何配合工作。

BIOS/UEFI启动:

1. 计算机上电, CPU开始执行BIOS/UEFI代码
2. BIOS/UEFI进行POST (加电自检)
3. BIOS/UEFI根据启动顺序, 读取启动设备 (通常是硬盘) 的第一个扇区 (MBR或EFI分区)
4. 加载引导装载程序 (Bootloader, 如GRUB)

引导装载程序:

1. 从磁盘读取操作系统内核
2. 将控制权交给操作系统内核

Linux内核启动:

1. 内核解压 (如果是压缩内核)
2. 初始化内核各个子系统 (内存管理、进程调度等)
3. 挂载根文件系统 (root filesystem)
4. 启动第一个用户空间进程 (PID 1, 通常是systemd/init)
5. 系统进入多用户模式

init进程:

init进程是系统中的第一个用户空间进程, 负责:

- 启动系统服务 (如网络服务、SSH服务)
- 管理用户登录
- 回收孤儿进程
- 处理系统关机

3.4.8 应用程序的执行

当我们在shell中运行一个程序时:

1. shell解析命令, 如果是内置命令则直接执行, 否则认为是可执行文件
2. shell调用fork(), 创建一个子进程
3. 子进程调用execve(), 将可执行文件加载到内存
4. 新程序开始执行
5. 父进程调用wait(), 等待子进程结束
6. 子进程结束后, 父进程回收资源, 继续等待下一个命令

```
// shell执行命令的简化逻辑
if (!builtin(command)) {
    pid = fork();
    if (pid == 0) {
        // 子进程
        execve(command, args, envp);
        // 如果execve成功, 不会到达这里
        perror("execve");
        exit(1);
    } else {
        // 父进程
```

(continues on next page)

(continued from previous page)

```
wait(NULL);  
}  
}
```

本节小结

- 操作系统管理计算机资源，提供用户程序与硬件之间的接口
- 内核运行在特权模式（内核态），用户程序运行在用户态
- 系统调用是用户程序请求内核服务的接口，通过软中断或syscall指令触发
- 常见系统调用包括进程控制（fork/exec/exit）、文件操作（open/read/write）和内存管理（mmap）
- 程序员通常通过C库间接使用系统调用，C库提供更友好的接口
- 程序执行时，shell通过fork+execve创建新进程

本章小结

本章介绍了程序从源代码到运行的全过程，以及操作系统在程序运行中的角色。

从源代码到可执行文件：

- 编译过程包括预处理、编译、汇编、链接四个阶段
- 预处理处理宏和头文件，编译进行词法分析、语法分析和代码生成
- 链接合并目标文件并解析符号引用，分为静态链接和动态链接
- ELF是Linux常用的可执行文件格式

进程与线程：

- 进程是程序执行的实例，是资源分配的基本单位
- 线程是进程内的执行单元，是CPU调度的基本单位
- 同一进程的多个线程共享地址空间，但有独立的栈和寄存器
- 线程同步机制包括互斥锁、信号量和条件变量

并发与并行：

- 并发是多个任务在同一时间段交替执行（单核），并行是多个任务同时执行（多核）
- 并发提高资源利用率，并行提高计算速度
- 并发编程面临竞态条件、死锁、活锁、饥饿等挑战
- Amdahl定律指出并行化的加速比受串行部分限制

操作系统与系统调用：

- 操作系统管理计算机资源，提供用户程序与硬件之间的接口
- 系统调用是用户程序请求内核服务的接口
- 常用系统调用包括进程控制（fork/exec）和文件操作（open/read/write）
- 程序执行时，shell通过fork+execve创建新进程

理解程序运行原理，有助于编写正确的程序和诊断问题。

思考与练习

1. 编译与链接

- 描述从源代码到可执行文件的完整过程，各个阶段分别完成什么任务？
- 静态链接和动态链接各有什么优缺点？
- 什么是符号解析？为什么链接时需要解析符号？

2. 进程与线程

- `fork()`系统调用有什么特殊之处？为什么它返回两次？
- 进程和线程有什么区别？什么情况下应该使用多线程而不是多进程？
- 什么是竞态条件？请举例说明。

3. 并发与并行

- 并发和并行的区别是什么？请各举一个例子。
- 什么是死锁？死锁产生的四个必要条件是什么？
- 如果程序中有50%的代码可以并行化，使用4核处理器，最大加速比是多少？

4. 系统调用

- 为什么用户程序不能直接访问硬件，而必须通过系统调用？
- 描述一次`read`系统调用的完整过程。
- `open()`和`fopen()`有什么区别？各自属于哪一层？

5. 综合应用

- 假设你编写了一个Web服务器程序，分析它可能使用哪些系统调用？
- 解释为什么一个线程的崩溃可能导致整个进程崩溃，而进程的隔离性可以防止这种情况？
- 在多线程程序中，为什么需要同步机制？请分析如果不同步会发生什么问题？

本章小结

本章为计算机基础预备知识章节，共包含三个部分：

第一部分：进制与编码

- 介绍了二进制、八进制、十六进制的基本概念和相互转换方法
- 详细讲解了原码、反码、补码的原理，以及计算机如何用补码表示有符号整数
- 解释了IEEE 754浮点数标准，包括规格化数、非规格化数和特殊值
- 介绍了ASCII和UTF-8字符编码的原理

第二部分：CPU与内存

- 阐述了冯诺依曼架构的基本原理和五大组成部分
- 深入分析了CPU的结构：ALU、控制单元、寄存器和指令执行流程
- 讲解了内存层次结构：寄存器、高速缓存、主存、虚拟内存的工作原理
- 介绍了局部性原理（时间局部性和空间局部性）是缓存技术的基础
- 说明了总线、DMA、缓存一致性和中断机制等CPU与内存交互的关键概念

第三部分：程序运行原理

- 描述了从源代码到可执行程序的完整过程：预处理、编译、汇编、链接
- 解释了ELF可执行文件格式的结构和程序的内存布局
- 深入讲解了进程与线程的概念、区别和同步机制

- 分析了并发与并行的区别，以及并发编程面临的问题（竞态条件、死锁等）
- 介绍了操作系统在内核态与用户态的角色，以及系统调用的工作机制

思考与练习

1. 进制转换与编码

- 将二进制数 1101011_2 转换为十进制、八进制和十六进制。
- 为什么计算机使用补码而不是原码来表示有符号整数？
- 解释浮点数精度问题：为什么 $0.1 + 0.2 \neq 0.3$ 在大多数编程语言中成立？

2. CPU与内存

- 描述冯诺依曼架构的基本工作流程，解释什么是“冯诺依曼瓶颈”。
- 假设L1 Cache未命中，CPU接下来会检查哪里？请描述缓存逐级查找的过程。
- 什么是虚拟内存？它解决了哪些问题？

3. 进程与线程

- `fork()`系统调用有什么特殊之处？为什么它返回两次？
- 多线程程序中，为什么需要互斥锁来保护共享数据？请分析如果不加保护会发生什么问题。
- 线程与进程的根本区别是什么？

4. 并发与并行

- 并发（Concurrency）和并行（Parallelism）的区别是什么？请各举一个实际应用场景。
- 什么是死锁？死锁产生的四个必要条件是什么？如何避免死锁？
- 如果程序中有90%的代码可以并行化，使用8核处理器，理论最大加速比是多少？（根据Amdahl定律计算）

5. 系统调用与程序运行

- 为什么用户程序不能直接访问硬件？系统调用是如何实现用户态和内核态切换的？
- 描述在Linux终端执行 `./a.out` 时，从shell创建进程到程序运行的完整过程。
- 静态链接和动态链接各有什么优缺点？

1.4.2 Chap02 Python编程

本章学习目标

1. 掌握Python基础语法
2. 理解NumPy数组操作
3. 学会使用PyTorch进行深度学习

前置知识

- Chap01 计算机基础

内容导航

Python基础语法

学习目标

- 理解Python中的变量与基本数据类型
- 掌握条件判断和循环控制流的写法
- 能够定义和调用函数

- 了解面向对象编程的核心概念：类与对象
- 掌握继承与多态的实现方式
- 理解列表推导式、生成器、迭代器和装饰器等高级特性

1.1 变量与数据类型

1.1.1 变量概述

变量（Variable）是编程中最基础的概念之一，用于存储数据。在Python中，变量不需要预先声明类型，解释器会根据赋值（Assignment）来推断变量的类型（Dynamic Typing）。

```
# 变量赋值示例
name = "Alice" # 字符串类型
age = 25       # 整数类型
height = 1.68  # 浮点数类型
is_student = True # 布尔类型
```

上述代码演示了Python中最常用的几种基本数据类型。变量名就像一个个容器，每个容器存放着不同类型的数据。

name、age、height、is_student

1.1.2 基本数据类型

Python内置了丰富的数据类型，主要分为以下几类：

数值类型

整数（int）：表示没有小数部分的数值，范围理论上无限制。

```
a = 100
b = -50
c = 0

# 不同进制的表示
binary_num = 0b1010 # 二进制：10
octal_num = 0o755    # 八进制：493
hex_num = 0xFF       # 十六进制：255

# 大整数运算
large_num = 10 ** 100 # 10的100次方
print(large_num)      # 输出：100000000...
```

浮点数（float）：表示带小数部分的数值，采用IEEE 754双精度标准。

```
pi = 3.14159
gravity = 9.8
negative = -0.5

# 科学计数法
speed_of_light = 3e8 # 3 × 10^8
```

复数（complex）：Python原生支持复数运算。

```
z = 3 + 4j
print(z.real) # 输出：3.0
print(z.imag) # 输出：4.0
```

字符串 (str)

字符串是字符的序列，用单引号或双引号包围。

```
# 字符串定义
single_quote = 'Hello'
double_quote = "World"
multi_line = """这是
一个多行
字符串"""

# 字符串拼接
greeting = "Hello" + ", " + "World"
print(greeting) # 输出: Hello, World

# 字符串重复
echo = "Ha" * 3
print(echo) # 输出: HaHaHa

# 字符串长度
text = "Python"
print(len(text)) # 输出: 6

# 字符串索引与切片
first_char = text[0] # 'P'
substring = text[0:3] # 'Pyt'
reversed_text = text[::-1] # 'nohtyP'
```

转义字符：用反斜杠 \ 表示特殊含义的字符。

```
new_line = "第一行\n第二行" # 换行
tab_indent = "列1\t列2" # 制表符
quote_in_string = "她说\"你好\"" # 引号
backslash_path = "C\\Users\\name" # 反斜杠
```

字符串方法：Python为字符串提供了丰富的内置方法。

```
s = " Hello, Python! "
```

大小写转换

```
print(s.upper()) # ' HELLO, PYTHON! '
print(s.lower()) # ' hello, python! '
print(s.capitalize()) # ' hello, python! '
```

去除空白

```
print(s.strip()) # 'Hello, Python!'
print(s.lstrip()) # 'Hello, Python! '
print(s.rstrip()) # ' Hello, Python!'
```

查找与替换

```
print(s.find("Python")) # 9 (首次出现的索引)
print(s.replace("Python", "World")) # ' Hello, World! '
```

分割与连接

```
words = "apple,banana,orange".split(",") # ['apple', 'banana', 'orange']
joined = "-".join(words) # 'apple-banana-orange'
```

格式化

(continues on next page)

(continued from previous page)

```
name = "Alice"
age = 25
formatted = f"姓名: {name}, 年龄: {age}" # f-string格式化
print(formatted) # 输出: 姓名: Alice, 年龄: 25
```

布尔类型 (bool)

布尔类型只有两个值: True (真) 和 False (假)。

```
is_valid = True
is_empty = False

# 布尔运算
print(True and False) # False
print(True or False)  # True
print(not True)       # False
```

在Python中, 以下值在布尔上下文中被视为假 (False) :

- None
- False
- 数值零: 0、0.0、0j
- 空序列: "", [], ()
- 空字典: {}
- 自定义对象, 其 `__bool__()` 返回 False 或 `__len__()` 返回 0

None类型

None 表示空值或 absence of value, 类似于其他语言中的 null。

```
result = None
if result is None:
    print("没有结果")
```

注意: 比较 None 时应使用 `is None` 或 `is not None`, 而不是 `== None` 或 `!= None`, 因为 `is` 比较的是对象身份, 而 `==` 比较的是值相等性。

1.1.3 类型转换

Python提供了显式转换数据类型的能力。

```
# 字符串转整数
num_str = "42"
num = int(num_str) # 42
num_float = float(num_str) # 42.0

# 整数转字符串
age = 25
age_str = str(age) # "25"

# 字符串转浮点数
pi_str = "3.14"
pi = float(pi_str) # 3.14
```

(continues on next page)

(continued from previous page)

```
# 浮点数转整数（截断小数部分）
x = 3.7
print(int(x))      # 3（不是四舍五入！）

# 四舍五入
print(round(3.7))   # 4
print(round(3.14159, 2)) # 3.14（保留两位小数）

# 转换为布尔
print(bool(0))      # False
print(bool(1))      # True
print(bool(""))     # False
print(bool("hello")) # True
```

1.1.4 常量

Python中没有真正的常量机制，但通常使用全大写变量名表示“不应被修改”的量。

```
# 约定：全大写表示常量
MAX_RETRY_COUNT = 3
PI = 3.14159
GRAVITY = 9.8
```

1.2 控制流

控制流（Control Flow）决定了程序执行的顺序。Python使用缩进（通常为4个空格）来表示代码块的层级关系。

1.2.1 条件判断（if语句）

```
# 基础if语句
age = 18

if age >= 18:
    print("已成年")
```

```
# if-else语句
temperature = 30

if temperature > 35:
    print("高温预警")
else:
    print("温度正常")
```

```
# if-elif-else语句（多条件）
score = 85

if score >= 90:
    grade = "A"
    print("优秀")
elif score >= 80:
    grade = "B"
    print("良好")
elif score >= 70:
    grade = "C"
    print("中等")
```

(continues on next page)

(continued from previous page)

```
elif score >= 60:
    grade = "D"
    print("及格")
else:
    grade = "F"
    print("不及格")

print(f"等级: {grade}")
```

三元表达式：Python支持单行条件表达式。

```
age = 20
status = "成年" if age >= 18 else "未成年"
print(status) # 输出: 成年
```

1.2.2 循环 (for循环)

for 循环用于遍历可迭代对象 (Iterable)，如列表、字符串、字典等。

```
# 遍历列表
fruits = ["苹果", "香蕉", "橙子"]

for fruit in fruits:
    print(fruit)

# 遍历字符串
for char in "Python":
    print(char)

# 使用range()函数
for i in range(5):
    print(i)      # 输出: 0, 1, 2, 3, 4

for i in range(1, 6):
    print(i)      # 输出: 1, 2, 3, 4, 5

for i in range(0, 10, 2):
    print(i)      # 输出: 0, 2, 4, 6, 8 (偶数)

# 遍历字典
person = {"name": "Alice", "age": 25, "city": "北京"}

for key in person:
    print(f"{key}: {person[key]}")

for key, value in person.items():
    print(f"{key}: {value}")

for value in person.values():
    print(value)
```

enumerate()函数：同时获取索引和值。

```
fruits = ["苹果", "香蕉", "橙子"]

for index, fruit in enumerate(fruits):
```

(continues on next page)

(continued from previous page)

```
print(f"{index}: {fruit}")
# 输出:
# 0: 苹果
# 1: 香蕉
# 2: 橙子
```

1.2.3 循环 (while循环)

while 循环在条件为真时重复执行代码块。

```
# 基础while循环
count = 0
while count < 5:
    print(count)
    count += 1

# while循环与break
while True:
    user_input = input("输入 'quit' 退出: ")
    if user_input == "quit":
        break
    print(f"你输入了: {user_input}")

# while循环与continue
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue # 跳过偶数
    print(i) # 输出: 1, 3, 5, 7, 9
```

1.2.4 循环的else子句

Python特有的语法：循环正常结束时会执行 else 子句（如果循环被 break 终止则不执行）。

```
# 搜索示例
numbers = [1, 3, 5, 7, 9]
target = 6

for n in numbers:
    if n == target:
        print(f"找到{target}")
        break
else:
    print(f"未找到{target}")
```

1.2.5 嵌套循环

循环内部可以包含另一个循环。

```
# 打印九九乘法表
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{j} × {i} = {i*j}", end="\t")
    print() # 换行
```

1.3 函数定义

函数（Function）是一段可重复使用的代码，用于完成特定任务。

1.3.1 函数基础

```
# 定义函数
def greet():
    """问候函数"""
    print("你好！")

# 调用函数
greet() # 输出：你好！

# 带参数的函数
def greet_person(name):
    """向指定人打招呼"""
    print(f"你好，{name}！")

greet_person("Alice") # 输出：你好， Alice！

# 带返回值的函数
def add(a, b):
    """返回两数之和"""
    return a + b

result = add(3, 5)
print(result) # 8

# 多返回值函数
def divide(a, b):
    """返回商和余数"""
    quotient = a // b
    remainder = a % b
    return quotient, remainder

q, r = divide(10, 3)
print(f"商：{q}，余数：{r}") # 商： 3，余数： 1
```

1.3.2 参数默认值

```
def power(base, exponent=2):
    """计算幂次，默认平方"""
    return base ** exponent

print(power(3)) # 9（使用默认值）
print(power(3, 3)) # 27（使用指定值）
```

注意：带默认值的参数必须放在无默认值参数的后面。

```
# 错误示例
# def func(a=1, b): # SyntaxError
#     pass

# 正确示例
def func(a, b=2):
    pass
```

1.3.3 关键字参数

使用参数名指定参数值，可以不按顺序传递。

```
def describe_pet(animal, name):
    print(f"我有一只{animal}，名叫{name}")

describe_pet(animal="猫", name="小白") # 我有一只猫，名叫小白
describe_pet(name="小黑", animal="狗") # 我有一只狗，名叫小黑
```

1.3.4 可变参数

使用 *args 和 **kwargs 处理不确定数量的参数。

```
# *args: 接收任意数量的位置参数（元组）
def sum_all(*numbers):
    """计算所有参数的和"""
    total = 0
    for n in numbers:
        total += n
    return total

print(sum_all(1, 2, 3))    # 6
print(sum_all(1, 2, 3, 4, 5)) # 15

# **kwargs: 接收任意数量的关键字参数（字典）
def print_info(**info):
    """打印所有关键字参数"""
    for key, value in info.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25, city="北京")
# 输出:
# name: Alice
# age: 25
# city: 北京
```

1.3.5 参数类型注解

Python 3.5+ 支持类型注解（Type Hints），使代码更易读。

```
def greet(name: str) -> str:
    """返回问候语"""
    return f"你好, {name}! "

def add(a: int, b: int) -> int:
    return a + b

# 列表类型注解
from typing import List

def process_numbers(numbers: List[int]) -> List[int]:
    return [n * 2 for n in numbers]
```

1.3.6 变量作用域

Python中的作用域遵循 LEGB 规则：Local（局部）→ Enclosing（闭包）→ Global（全局）→ Built-in（内置）。

```
# 全局变量
count = 10

def modify_global():
    global count # 声明使用全局变量
    count = 20

def read_only():
    print(count) # 读取全局变量

modify_global()
read_only() # 输出：20
```

闭包：内层函数可以访问外层函数的变量。

```
def outer(x):
    def inner(y):
        return x + y
    return inner

add_5 = outer(5)
print(add_5(3)) # 8
print(add_5(10)) # 15
```

1.3.7 匿名函数 (lambda)

lambda 用于创建简短的匿名函数。

```
# 基础lambda
square = lambda x: x ** 2
print(square(5)) # 25

# 多参数lambda
add = lambda a, b: a + b
print(add(3, 4)) # 7

# 与内置函数配合使用
numbers = [1, 2, 3, 4, 5]

# 按绝对值排序
sorted_nums = sorted(numbers, key=lambda x: -x)
print(sorted_nums) # [5, 4, 3, 2, 1]

# map: 应用函数到每个元素
doubled = list(map(lambda x: x * 2, numbers))
print(doubled) # [2, 4, 6, 8, 10]

# filter: 过滤元素
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4]
```

1.4 面向对象基础

面向对象编程（Object-Oriented Programming, OOP）是一种组织代码的方式，通过“类”定义对象的结构和行为。

1.4.1 类与对象

类（Class）：对象的蓝图或模板，定义了对象的属性和方法。

对象（Object）：类的实例，具有具体的状态和行为。

```
# 定义类
class Dog:
    """狗类"""

    # 类属性（所有实例共享）
    species = "犬科"

    # 初始化方法（构造函数）
    def __init__(self, name, age):
        # 实例属性
        self.name = name
        self.age = age

    # 实例方法
    def bark(self):
        """狗叫"""
        return f"{self.name}汪汪叫！"

    def get_info(self):
        """获取信息"""
        return f"{self.name}, {self.age}岁"

# 创建对象（实例化）
dog1 = Dog("小白", 3)
dog2 = Dog("小黑", 5)

# 访问属性
print(dog1.name)  # 小白
print(dog2.age)  # 5

# 调用方法
print(dog1.bark())  # 小白汪汪叫！

# 类属性
print(Dog.species)  # 犬科
```

1.4.2 面向对象的三大特性

封装（Encapsulation）

封装将数据和操作封装在类内部，隐藏内部实现细节。

```
class BankAccount:
    """银行账户类"""

    def __init__(self, owner, balance=0):
        self.owner = owner
        # 私有属性（以双下划线开头）
```

(continues on next page)

(continued from previous page)

```

    self.__balance = balance

# 公开方法访问私有属性
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
        return True
    return False

def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount
        return True
    return False

def get_balance(self):
    return self.__balance

# 使用
account = BankAccount("Alice", 1000)
print(account.get_balance()) # 1000
account.deposit(500)
print(account.get_balance()) # 1500
account.withdraw(200)
print(account.get_balance()) # 1300

# 无法直接访问私有属性
# print(account.__balance) # AttributeError

```

继承 (Inheritance)

继承允许创建基于现有类的新类，子类自动获得父类的属性和方法。

```

# 父类
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("子类必须实现speak方法")

    def info(self):
        return f"名字: {self.name}"

# 子类
class Cat(Animal):
    def speak(self):
        return f"{self.name} 喵喵叫"

class Dog(Animal):
    def speak(self):
        return f"{self.name} 汪汪叫"

class Bird(Animal):
    def speak(self):
        return f"{self.name} 叽叽叫"

```

(continues on next page)

(continued from previous page)

```
# 创建子类实例
cat = Cat("小白")
dog = Dog("小黑")
bird = Bird("小小")

print(cat.speak()) # 小白喵喵叫
print(dog.speak()) # 小黑汪汪叫
print(bird.speak()) # 小小叽叽叫
```

多态 (Polymorphism)

多态允许不同类的对象对同一消息（方法调用）做出不同响应。

```
# 统一接口处理不同对象
def make_speak(animal):
    print(animal.speak())

animals = [Cat("白猫"), Dog("黑狗"), Bird("小鸟")]

for animal in animals:
    make_speak(animal)
# 输出:
# 白猫喵喵叫
# 黑狗汪汪叫
# 小鸟叽叽叫
```

1.4.3 方法的类型

```
class MyClass:
    class_attr = "类属性" # 类属性

    def __init__(self, value):
        self.instance_attr = value # 实例属性

    # 实例方法：第一个参数是self
    def instance_method(self):
        return f"实例方法，访问：{self.instance_attr}"

    # 类方法：第一个参数是cls
    @classmethod
    def class_method(cls):
        return f"类方法，访问：{cls.class_attr}"

    # 静态方法：没有self或cls参数
    @staticmethod
    def static_method():
        return "静态方法，不访问任何实例或类属性"

# 调用
obj = MyClass("value")
print(obj.instance_method()) # 实例方法，访问：value
print(MyClass.class_method()) # 类方法，访问：类属性
print(MyClass.static_method()) # 静态方法，不访问任何实例或类属性
```

1.4.4 属性装饰器 (@property)

使用 @property 将方法转换为只读属性。

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        """圆的半径"""
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("半径不能为负数")
        self._radius = value

    @property
    def area(self):
        """圆的面积（只读）"""
        return 3.14159 * self._radius ** 2

# 使用
circle = Circle(5)
print(circle.radius) # 5（像访问属性一样）
print(circle.area) # 78.53975

circle.radius = 10 # 修改半径
print(circle.area) # 314.159

# circle.radius = -5 # ValueError: 半径不能为负数
```

1.5 Python高级特性

1.5.1 列表推导式

列表推导式（List Comprehension）是一种简洁的创建列表的方式。

```
# 基础语法
# [表达式 for 变量 in 可迭代对象]

# 传统方式
squares = []
for i in range(1, 6):
    squares.append(i ** 2)
print(squares) # [1, 4, 9, 16, 25]

# 列表推导式
squares = [i ** 2 for i in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]

# 带条件筛选
evens = [i for i in range(1, 11) if i % 2 == 0]
print(evens) # [2, 4, 6, 8, 10]

# 嵌套列表推导式
```

(continues on next page)

(continued from previous page)

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# 带if-else的列表推导式
labels = ["正数" if x > 0 else "非正数" for x in [-3, -1, 0, 2, 5]]
print(labels) # ['非正数', '非正数', '非正数', '正数', '正数']
```

其他推导式：

```
# 字典推导式
squares_dict = {i: i ** 2 for i in range(1, 6)}
print(squares_dict) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# 集合推导式
unique_squares = {x ** 2 for x in [-2, -1, 0, 1, 2]}
print(unique_squares) # {0, 1, 4}

# 生成器表达式（惰性求值）
gen = (x ** 2 for x in range(1, 1000001))
print(next(gen)) # 1
print(next(gen)) # 4
# 注意：生成器只能遍历一次
```

1.5.2 生成器与迭代器

迭代器（Iterator）：实现了 `__iter__()` 和 `__next__()` 方法的对象。

```
# 使用iter()和next()
my_list = [1, 2, 3]
iterator = iter(my_list)

print(next(iterator)) # 1
print(next(iterator)) # 2
print(next(iterator)) # 3
# print(next(iterator)) # StopIteration
```

生成器（Generator）：使用 `yield` 关键字的函数，每次调用返回一个值并暂停执行状态。

```
# 生成器函数
def count_up_to(n):
    """计数生成器"""
    count = 1
    while count <= n:
        yield count
        count += 1

# 使用生成器
counter = count_up_to(5)
print(next(counter)) # 1
print(next(counter)) # 2
print(next(counter)) # 3

# 遍历生成器
for num in count_up_to(3):
    print(num)
```

(continues on next page)

(continued from previous page)

```
# 输出: 1, 2, 3

# 生成器表达式 (惰性求值, 节省内存)
def fibonacci():
    """斐波那契数列生成器"""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib), end=" ")
# 输出: 0 1 1 2 3 5 8 13 21 34
```

生成器 vs 列表:

```
import sys

# 列表: 一次性占用所有内存
my_list = [i for i in range(10000)]
print(sys.getsizeof(my_list), "bytes") # 约80000 bytes

# 生成器: 惰性求值, 占用少量内存
my_gen = (i for i in range(10000))
print(sys.getsizeof(my_gen), "bytes") # 约200 bytes
```

1.5.3 装饰器

装饰器 (Decorator) 是一种修改函数或类行为的高级语法, 可以在不修改原函数的情况下添加额外功能。

```
# 简单装饰器
def my_decorator(func):
    """装饰器函数"""
    def wrapper(*args, **kwargs):
        print("函数执行前")
        result = func(*args, **kwargs)
        print("函数执行后")
        return result
    return wrapper

@my_decorator
def say_hello():
    print("你好!")

say_hello()
# 输出:
# 函数执行前
# 你好!
# 函数执行后
```

带参数的装饰器:

```
import time

def timer(seconds):
```

(continues on next page)

(continued from previous page)

```

"""计时器装饰器工厂"""
def decorator(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f"函数执行耗时: {elapsed:.2f}秒")
        return result
    return wrapper
return decorator

@timer(2)
def slow_function():
    time.sleep(1)
    print("函数执行完成")

slow_function()

```

类装饰器：

```

class CallCounter:
    """统计函数调用次数"""
    def __init__(self, func):
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"函数被调用了{self.count}次")
        return self.func(*args, **kwargs)

@CallCounter
def greet():
    print("你好！")

greet() # 函数被调用了1次，你好！
greet() # 函数被调用了2次，你好！

```

functools.wraps 保留原函数信息：

```

import functools

def log_calls(func):
    @functools.wraps(func) # 保留原函数元信息
    def wrapper(*args, **kwargs):
        print(f"调用 {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_calls
def add(a, b):
    """返回两数之和"""
    return a + b

print(add.__name__) # add (而不是wrapper)
print(add.__doc__) # 返回两数之和

```

1.5.4 上下文管理器

上下文管理器（Context Manager）用于管理资源，确保资源正确获取和释放。

```
# 使用with语句
with open("example.txt", "w") as f:
    f.write("Hello, World!")
# 文件自动关闭，无需f.close()

# try-finally等价
f = open("example.txt", "w")
try:
    f.write("Hello, World!")
finally:
    f.close()
```

自定义上下文管理器：

```
# 方法1：使用类
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False # 不屏蔽异常

with FileManager("test.txt", "w") as f:
    f.write("Hello!")

# 方法2：使用contextlib
from contextlib import contextmanager

@contextmanager
def file_manager(filename, mode):
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()
```

本章小结

本章介绍了Python编程的基础知识，主要包括：

1. 变量与数据类型：Python的动态类型系统、基本数据类型（整数、浮点数、字符串、布尔值、None）、类型转换规则。
2. 控制流：条件判断（if-elif-else）、循环（for、while）、break/continue语句、Python特有的循环else子句。
3. 函数定义：函数的基本结构、参数类型（默认值、关键字参数、可变参数 *args 和 **kwargs）、类型注解、变量作用域、lambda表达式。
4. 面向对象编程：类和对象的概念、封装（私有属性）、继承（子类化）、多态（统一接口）、实例方法/类方法/静态方法。
5. Python高级特性：列表推导式、生成器与迭代器、装饰器模式、上下文管理器。

这些知识构成了Python编程的核心基础，掌握它们为后续学习NumPy和PyTorch等数据科学工具打下坚实基础。

思考与练习

1. 编写一个函数，接受一个整数列表，返回其中所有奇数的和。例如，输入 [1, 2, 3, 4, 5]，输出 9 (1+3+5)。
2. 创建一个 Rectangle 类，包含 width 和 height 属性，以及计算面积和周长的方法。添加属性验证，确保宽高为正数。
3. 使用列表推导式将 [[1, 2, 3], [4, 5, 6], [7, 8, 9]] 展平为一维列表 [1, 2, 3, 4, 5, 6, 7, 8, 9]。
4. 编写一个装饰器 retry，在函数执行失败时自动重试最多3次，每次重试间隔1秒。适用于网络请求等可能临时失败的
5. 实现一个生成器函数 primes(limit)，返回小于 limit 的所有质数。要求使用埃拉托斯特尼筛法 (Sieve of Eratosthenes) 算法实现。

NumPy基础

学习目标

- 理解NumPy的核心概念：ndarray数组
- 掌握NumPy数组的创建、索引和切片操作
- 熟悉NumPy的形状操作 (reshape、transpose等)
- 掌握NumPy的逐元素运算和矩阵运算
- 理解NumPy的广播机制及其应用场景
- 了解常用NumPy函数和性能注意事项

2.1 NumPy概述

NumPy (Numerical Python) 是Python生态中用于数值计算的基础库，提供了高性能的多维数组对象 ndarray 以及丰富的数学函数库。几乎所有深度学习框架 (PyTorch、TensorFlow等) 的张量操作都借鉴了NumPy的设计思

2.1.1 安装与导入

```
# 安装NumPy
# pip install numpy

# 导入NumPy (约定使用np作为别名)
import numpy as np
```

2.1.2 NumPy的核心：ndarray

ndarray (N-dimensional array) 是NumPy的核心数据结构，是一个多维同质数组 (即所有元素类型相同)。

```
# 一维数组
a = np.array([1, 2, 3, 4, 5])
print(a)      # [1 2 3 4 5]
print(type(a)) # <class 'numpy.ndarray'>

# 二维数组 (矩阵)
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b)
# [[1 2 3]
#  [4 5 6]]
```

(continues on next page)

(continued from previous page)

```
# 三维数组
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
print(c)
# [[[1 2]
#    [3 4]]
#   [[5 6]
#    [7 8]]]
```

NumPy数组 vs Python列表：

特性	Python列表	NumPy数组
元素类型	任意类型	相同类型
内存占用	较大	紧凑
计算速度	慢（逐元素操作）	快（向量化）
功能	基础列表操作	丰富数学函数

```
# Python列表：每个元素都是独立对象
py_list = [1, 2, 3, 4, 5]
print(py_list * 2) # [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]（重复连接）

# NumPy数组：逐元素乘法
np_array = np.array([1, 2, 3, 4, 5])
print(np_array * 2) # [2, 4, 6, 8, 10]（数学运算）
```

2.2 创建数组

NumPy提供了多种创建数组的方式。

2.2.1 从Python列表转换

```
# 从列表创建
list1 = [1, 2, 3, 4, 5]
arr1 = np.array(list1)

# 指定数据类型
arr2 = np.array([1, 2, 3], dtype=np.float32)
print(arr2.dtype) # float32

# 嵌套列表创建多维数组
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
print(matrix.shape) # (2, 3)
```

2.2.2 预定义数组

```
# 全0数组
zeros_1d = np.zeros(5)      # 一维
zeros_2d = np.zeros((3, 4)) # 二维（3行4列）
zeros_3d = np.zeros((2, 3, 4)) # 三维
print(zeros_2d)
# [[0. 0. 0. 0.]
#   [0. 0. 0. 0.]
```

(continues on next page)

(continued from previous page)

```

# [0. 0. 0. 0.]

# 全1数组
ones = np.ones((2, 3))
print(ones)
# [[1. 1. 1.]
#  [1. 1. 1.]]

# 填充指定值
full = np.full((2, 3), 7)    # 全部填充为7
print(full)
# [[7 7 7]
#  [7 7 7]]

# 单位矩阵（方阵）
identity = np.eye(4)        # 4×4单位矩阵
print(identity)
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]
#  [0. 0. 0. 1.]]

# 对角矩阵
diag = np.diag([1, 2, 3, 4])
print(diag)
# [[1 0 0 0]
#  [0 2 0 0]
#  [0 0 3 0]
#  [0 0 0 4]]

```

2.2.3 范围数组

```

# arange: 类似Python的range
a = np.arange(10)    # 0到9，步长1
print(a)             # [0 1 2 3 4 5 6 7 8 9]

b = np.arange(1, 10) # 1到9
print(b)             # [1 2 3 4 5 6 7 8 9]

c = np.arange(0, 10, 2) # 0到9，步长2
print(c)             # [0 2 4 6 8]

# linspace: 等差数列（包含端点）
d = np.linspace(0, 1, 5) # 0到1之间均匀取5个点
print(d)                # [0. 0.25 0.5 0.75 1. ]

# logspace: 对数等比数列
e = np.logspace(0, 2, 5) # 10^0到10^2之间取5个点
print(e)                # [ 1.      3.16227766 10.

```

2.2.4 随机数组

```

# 设置随机种子（结果可复现）
np.random.seed(42)

```

(continues on next page)

(continued from previous page)

```
# rand: 均匀分布[0, 1)
uniform = np.random.rand(3, 4)
print(uniform)
# [[0.37454012 0.95071431 0.73199394 0.59865848]
#  [0.15601864 0.15599452 0.05808361 0.86617615]
#  [0.60111501 0.70807258 0.02058449 0.96990985]]

# randn: 标准正态分布 (均值0, 方差1)
normal = np.random.randn(3, 4)
print(normal)

# randint: 整数随机数组
integers = np.random.randint(0, 10, (3, 4)) # 0到9, 随机整数
print(integers)

# choice: 随机选择
choices = np.random.choice([1, 2, 3, 4, 5], size=10)
print(choices)

# shuffle: 打乱数组
arr = np.arange(10)
np.random.shuffle(arr)
print(arr)
```

2.3 数组属性

NumPy数组具有多个重要属性，用于描述数组的结构和特征。

```
arr = np.array([[1, 2, 3],
               [4, 5, 6]])

# shape: 形状 (元组)
print(arr.shape) # (2, 3) - 2行3列

# ndim: 维度数
print(arr.ndim) # 2

# size: 元素总数
print(arr.size) # 6

# dtype: 数据类型
print(arr.dtype) # int64

# itemsize: 每个元素的字节大小
print(arr.itemsize) # 8 (int64为8字节)

# nbytes: 总字节数
print(arr.nbytes) # 48 (6 × 8)

# T: 转置
print(arr.T)
# [[1 4]
#  [2 5]
#  [3 6]]
```

常用数据类型：

dtype	说明	字节
int32	32位整数	4
int64	64位整数	8
float32	32位浮点	4
float64	64位浮点	8
bool	布尔	1
complex64	复数（双32位）	8
complex128	复数（双64位）	16

```
# 指定数据类型创建数组
arr = np.array([1, 2, 3], dtype=np.float32)
print(arr.dtype) # float32

# 类型转换
arr_float = arr.astype(np.float64)
print(arr_float.dtype) # float64

arr_int = arr.astype(np.int32)
print(arr_int.dtype) # int32
```

2.4 索引与切片

NumPy数组支持丰富的数据访问方式，包括基本索引、切片、布尔索引和花式索引。

2.4.1 基本索引

```
# 一维数组索引
a = np.array([1, 2, 3, 4, 5])
print(a[0]) # 1（第一个元素）
print(a[-1]) # 5（最后一个元素）

# 二维数组索引
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(b[0]) # [1 2 3]（第一行）
print(b[1, 2]) # 6（第二行第三列）
print(b[-1, -1]) # 9（最后一行最后一列）
```

2.4.2 切片操作

切片语法：start:stop:step，其中 start 默认为0，stop 默认为该维度大小，step 默认为1。

```
# 一维数组切片
a = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

print(a[2:7]) # [2 3 4 5 6]（索引2到6）
print(a[:5]) # [0 1 2 3 4]（从头到索引4）
print(a[5:]) # [5 6 7 8 9]（索引5到末尾）
print(a[::2]) # [0 2 4 6 8]（步长2）
print(a[::-1]) # [9 8 7 6 5 4 3 2 1 0]（倒序）

# 二维数组切片
```

(continues on next page)

(continued from previous page)

```

b = np.array([[0, 1, 2, 3],
              [4, 5, 6, 7],
              [8, 9, 10, 11]])

print(b[0, :]) # [0 1 2 3] (第一行)
print(b[:, 1]) # [1 5 9] (第二列)
print(b[2:, :2]) # [[0 1] [4 5]] (左上2×2子矩阵)
print(b[1:, 2:]) # [[6 7] [10 11]] (右下2×2子矩阵)
print(b[:, 2, ::2]) # [[0 2] [8 10]] (跳行跳列)

```

切片是视图，不是副本：

```

# 重要：切片返回的是视图，修改视图会影响原数组
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]
print(b) # [2 3 4]
b[0] = 99 # 修改视图
print(a) # [1 99 3 4 5] (原数组被修改！)

# 如果需要副本，使用copy()
c = a[1:4].copy()
c[0] = 100
print(a) # [1 99 3 4 5] (原数组不变)

```

2.4.3 布尔索引

使用布尔数组进行索引，常用于条件筛选。

```

# 一维布尔索引
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 找出所有偶数
mask = (a % 2 == 0)
print(mask) # [False True False True False True False True False True]
print(a[mask]) # [2 4 6 8 10]

# 直接使用条件表达式
print(a[a > 5]) # [6 7 8 9 10]
print(a[(a > 2) & (a < 8)]) # [3 4 5 6 7] (&表示AND, |表示OR)

# 二维布尔索引
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# 找出大于5的元素
mask = b > 5
print(mask)
# [[False False False]
#  [False False True]
#  [ True  True  True]]
print(b[mask]) # [6 7 8 9] (返回一维数组)

```

2.4.4 花式索引

使用整数数组进行索引，可以不按顺序甚至使用重复索引。

```
# 一维花式索引
a = np.array([10, 20, 30, 40, 50])

# 按特定顺序获取元素
indices = [0, 2, 4]
print(a[indices]) # [10 30 50]

# 按逆序获取元素
print(a[::-1]) # [50 40 30 20 10]

# 二维花式索引
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

# 按行索引数组（获取第0行和第2行）
print(b[[0, 2]]) # [[1 2 3] [7 8 9]]

# 使用整数数组指定行和列
rows = np.array([0, 1, 2])
cols = np.array([2, 1, 0])
print(b[rows, cols]) # [3 5 7]（对应位置的元素）

# 使用np.ix_创建网格索引
print(b[np.ix_([0, 2], [0, 2])])
# [[1 3]
# [7 9]]
```

2.5 形状操作

NumPy提供了丰富的形状操作函数，用于改变数组的维度、大小和布局。

2.5.1 reshape - 改变形状

```
a = np.arange(12)
print(a) # [0 1 2 3 4 5 6 7 8 9 10 11]

# 重塑为3行4列
b = a.reshape(3, 4)
print(b)
# [[0 1 2 3]
# [4 5 6 7]
# [8 9 10 11]]

# -1自动计算维度
c = a.reshape(3, -1) # 自动计算列数
print(c.shape) # (3, 4)

d = a.reshape(-1, 6) # 自动计算行数
print(d.shape) # (2, 6)

# 展平为一维
flat = b.reshape(-1) # 或 b.flatten()
```

(continues on next page)

(continued from previous page)

```
print(flat) # [0 1 2 3 4 5 6 7 8 9 10 11]
```

reshape返回视图:

```
# reshape通常返回视图
a = np.arange(12)
b = a.reshape(3, 4)
print(b.base is a) # True (共享数据)

# 验证: 修改b会影响a
b[0, 0] = 99
print(a[0]) # 99
```

2.5.2 transpose - 转置

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape) # (2, 3)

# 转置
b = a.T
print(b.shape) # (3, 2)
print(b)
# [[1 4]
#  [2 5]
#  [3 6]]

# 高维数组转置: 指定轴的顺序
c = np.arange(24).reshape(2, 3, 4)
print(c.shape) # (2, 3, 4)

# axes参数指定新轴的顺序
# 原始轴顺序 (0, 1, 2) -> 新顺序 (1, 0, 2)
d = c.transpose(1, 0, 2)
print(d.shape) # (3, 2, 4)

# np.moveaxis 移动轴
e = np.moveaxis(c, 2, 0) # 将第2轴移到第0位
print(e.shape) # (4, 2, 3)
```

2.5.3 squeeze与expand_dims

```
# squeeze: 移除大小为1的维度
a = np.arange(12).reshape(1, 3, 4, 1)
print(a.shape) # (1, 3, 4, 1)

b = np.squeeze(a)
print(b.shape) # (3, 4)

# 指定移除特定维度
c = np.squeeze(a, axis=0) # 移除第0维
print(c.shape) # (3, 4, 1)

# expand_dims: 添加大小为1的维度
d = np.array([1, 2, 3])
```

(continues on next page)

(continued from previous page)

```
print(d.shape) # (3,)

e = np.expand_dims(d, axis=0) # 添加第0维
print(e.shape) # (1, 3)

f = np.expand_dims(d, axis=1) # 添加第1维
print(f.shape) # (3, 1)
```

2.5.4 连接与分割

```
# 连接
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# concatenate: 沿现有轴连接
c = np.concatenate([a, b])
print(c) # [1 2 3 4 5 6]

# 沿不同轴连接
d = np.array([[1, 2], [3, 4]])
e = np.array([[5, 6], [7, 8]])

f = np.concatenate([d, e], axis=0) # 垂直连接
print(f)
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

g = np.concatenate([d, e], axis=1) # 水平连接
print(g)
# [[1 2 5 6]
#  [3 4 7 8]]

# vstack: 垂直堆叠
h = np.vstack([d, e]) # 等价于concatenate([d, e], axis=0)

# hstack: 水平堆叠
i = np.hstack([d, e]) # 等价于concatenate([d, e], axis=1)

# 分割
arr = np.arange(12).reshape(3, 4)
print(arr)
# [[0 1 2 3]
#  [4 5 6 7]
#  [8 9 10 11]]

# 水平分割为4个数组
split_h = np.hsplit(arr, 4)

# 垂直分割为3个数组
split_v = np.vsplit(arr, 3)

# 指定分割位置
a, b, c = np.split(arr, [1, 3], axis=1) # 按列索引分割
```

(continues on next page)

(continued from previous page)

```
print(a) # 第0列
print(b) # 第1-2列
print(c) # 第3列
```

2.6 数组运算

NumPy的核心优势之一是其向量化运算能力，无需显式循环即可对整个数组进行数学运算。

2.6.1 逐元素运算

```
a = np.array([1, 2, 3, 4, 5])

# 基本算术运算（逐元素应用）
print(a + 10) # [11 12 13 14 15]
print(a - 5)  # [-4 -3 -2 -1  0]
print(a * 2)  # [ 2  4  6  8 10]
print(a / 2)  # [0.5 1.  1.5 2.  2.5]
print(a ** 2) # [ 1  4  9 16 25]
print(a % 2)  # [1 0 1 0 1]（取模）

# 两个数组逐元素运算
b = np.array([10, 20, 30, 40, 50])
print(a + b)  # [11 22 33 44 55]
print(a * b)  # [10 40 90 160 250]

# 比较运算
print(a > 3)  # [False False False  True  True]
print(a == 3) # [False False  True False False]
print(a != 3) # [ True  True False  True  True]
```

通用函数（ufunc）：

NumPy的通用函数是一类对数组进行逐元素运算的函数，性能远优于Python循环。

```
a = np.array([1, 2, 3, 4, 5])

# 三角函数
print(np.sin(a)) # [0.84147098 0.90929743 0.14112001 -0.7568025 -0.95892427]
print(np.cos(a)) # [ 0.54030231 -0.41614684 -0.9899925 -0.65364362  0.28366219]
print(np.tan(a)) # [ 1.55740772 -2.18503986 -0.14254654  1.15782128 -3.38051501]

# 指数与对数
print(np.exp(a)) # [ 2.71828183  7.3890561  20.08553692  54.59815003 148.4131591]
print(np.log(a)) # [0.         0.69314718 1.09861229 1.38629436 1.60943791]
print(np.log2(a)) # [0.         1.         1.5849625  2.         2.32192809]
print(np.log10(a)) # [0.         0.30103   0.47712125 0.60205999 0.69897   ]

# 幂函数
print(np.sqrt(a)) # [1.         1.41421356 1.73205081 2.         2.23606798]

# 绝对值
c = np.array([-1, -2, -3, 4, 5])
print(np.abs(c)) # [1 2 3 4 5]

# 四舍五入
d = np.array([1.234, 2.567, 3.891])
```

(continues on next page)

(continued from previous page)

```
print(np.round(d, 1)) # [1.2 2.6 3.9]
print(np.floor(d))   # [1. 2. 3.] (向下取整)
print(np.ceil(d))    # [2. 3. 4.] (向上取整)
```

2.6.2 矩阵运算

NumPy使用 @ 运算符或 np.dot() 函数进行矩阵乘法。

```
# 二维矩阵乘法
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

# 矩阵乘法（维度必须兼容）
C = A @ B
print(C)
# [[1*5+2*7 1*6+2*8] [[19 22]
# [3*5+4*7 3*6+4*8]] = [43 50]]

# 或使用np.dot
D = np.dot(A, B)
print(np.array_equal(C, D)) # True

# 向量点积
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
dot = np.dot(v1, v2)
print(dot) # 1*4 + 2*5 + 3*6 = 32

# 向量范数
norm = np.linalg.norm(v1) # 默认L2范数
print(norm) # sqrt(1+4+9) = 3.7416...

# 矩阵的转置
print(A.T)
# [[1 3]
# [2 4]]
```

2.6.3 广播机制

广播（Broadcasting）是NumPy处理不同形状数组运算的机制，使运算更加便捷。

广播规则：

1. 让所有输入数组向形状相同的方向扩展
2. 每个维度的大小要么相同，要么其中一个为1
3. 从右向左比较维度

```
# 基本广播示例
a = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)

b = np.array([10, 20, 30]) # shape (3,)

# b被广播为 [[10, 20, 30], [10, 20, 30]]
```

(continues on next page)

(continued from previous page)

```

result = a + b
print(result)
# [[11 22 33]
#  [14 25 36]]

# 行向量广播
c = np.array([[1], [2]]) # shape (2, 1)
# c被广播为 [[1, 1, 1], [2, 2, 2]]
result = a + c
print(result)
# [[ 2  3  4]
#  [ 6  7  8]]

# 标量与数组运算（自动广播）
d = 100
result = a + d # d被广播为与a相同形状的数组
print(result)
# [[101 102 103]
#  [104 105 106]]

```

更复杂的广播示例：

```

# 图像数据处理示例
# 假设有batch_size=3, height=4, width=5, channels=3 的图像
images = np.random.randn(3, 4, 5, 3) # (3, 4, 5, 3)

# 减去均值（沿最后一维计算）
mean = np.mean(images, axis=-1, keepdims=True) # (3, 4, 5, 1)
normalized = images - mean
print(normalized.shape) # (3, 4, 5, 3)

# 加权求和示例
weights = np.array([0.2, 0.5, 0.3]) # (3,)
# weights被广播为 (3, 4, 5, 3) 进行逐元素乘法
weighted = images * weights.reshape(1, 1, 1, 3)
print(weighted.shape) # (3, 4, 5, 3)

```

2.6.4 归约操作

归约操作将数组缩减为更小的维度或单一值。

```

a = np.array([[1, 2, 3],
              [4, 5, 6]])

# 求和
print(np.sum(a))      # 21（所有元素之和）
print(np.sum(a, axis=0)) # [5 7 9]（按列求和）
print(np.sum(a, axis=1)) # [ 6 15]（按行求和）

# 均值
print(np.mean(a))      # 3.5
print(np.mean(a, axis=0)) # [2.5 3.5 4.5]

# 标准差
print(np.std(a))       # 约1.707

```

(continues on next page)

(continued from previous page)

```
# 最大最小值
print(np.min(a))      # 1
print(np.max(a))      # 6
print(np.argmin(a))   # 0 (最小值的索引)
print(np.argmax(a))   # 5 (最大值的索引)

# 累积操作
print(np.cumsum(a))   # [ 1  3  6 10 15 21]
print(np.cumprod(a))  # [ 1  2  6 24 120 720]
```

2.6.5 比较与掩码操作

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 找出满足条件的元素索引
print(np.where(a > 5)) # (array([5, 6, 7, 8, 9]),)

# 条件替换
b = np.where(a > 5, a, 0) # 大于5保留，否则为0
print(b)                 # [0 0 0 0 0 6 7 8 9 10]

# 判断是否全部/任意满足条件
c = np.array([[1, 2, 3],
               [4, 5, 6]])
print(np.all(c > 0)) # True (所有元素都大于0)
print(np.any(c > 5)) # True (存在大于5的元素)

# 排序
d = np.array([3, 1, 4, 1, 5, 9, 2, 6])
sorted_idx = np.argsort(d) # 返回排序后的索引
print(d[sorted_idx]) # [1 1 2 3 4 5 6 9]
```

2.7 实用技巧

2.7.1 常用函数速查

```
# 数组复制
a = np.array([1, 2, 3])
b = a.copy() # 完全独立的副本

# 排序
a = np.array([3, 1, 2])
sorted_a = np.sort(a) # 返回副本
a.sort() # 原地排序

# 唯一值
a = np.array([1, 2, 2, 3, 3, 3])
unique = np.unique(a) # [1 2 3]

# 集合运算
a = np.array([1, 2, 3, 4])
b = np.array([3, 4, 5, 6])
print(np.intersect1d(a, b)) # [3 4] (交集)
print(np.union1d(a, b))    # [1 2 3 4 5 6] (并集)
print(np.setdiff1d(a, b))  # [1 2] (差集)
```

(continues on next page)

(continued from previous page)

```
# 重排数组
a = np.arange(10)
np.random.shuffle(a) # 原地打乱
print(a)
```

2.7.2 文件I/O

```
# 保存数组到文件
a = np.arange(12).reshape(3, 4)

# 保存为文本格式
np.savetxt("array.txt", a)

# 保存为二进制格式 (.npy)
np.save("array.npy", a)

# 保存多个数组
np.savez("arrays.npz", a=a, data=a * 2)

# 加载数组
loaded = np.load("array.npy")
print(np.array_equal(a, loaded)) # True

# 加载多个数组
multi = np.load("arrays.npz")
print(multi["a"]) # 原始数组
print(multi["data"]) # data数组
```

2.7.3 性能注意事项

```
# 避免循环，使用向量化
# 慢：
result = []
for i in range(1000):
    result.append(i ** 2)
result = np.array(result)

# 快：
result = np.arange(1000) ** 2

# 使用原地操作节省内存
a = np.arange(1000000)
# 慢：创建新数组
b = a + 1
# 快：原地操作
np.add(a, 1, out=a)

# 使用np.einsum进行高效矩阵运算
A = np.random.randn(100, 200)
B = np.random.randn(200, 50)

# 等价于 C = A @ B，但更灵活
C = np.einsum('ij,jk->ik', A, B)
```

(continues on next page)

(continued from previous page)

```
# 使用np.take代替花式索引（某些情况下更快）
indices = np.array([0, 5, 10])
a = np.arange(10000)
# 慢：a[indices]
# 快：np.take(a, indices)
```

2.7.4 常见错误处理

```
# 形状不匹配
a = np.array([1, 2, 3])
b = np.array([4, 5])
try:
    c = a + b # ValueError
except ValueError as e:
    print(f"形状不匹配: {e}")

# 正确做法：使用广播或重塑
b_reshaped = b.reshape(1, 2) # 或 b.reshape(-1, 1)
c = a.reshape(3, 1) + b
print(c)
# [[5 6]
# [6 7]
# [7 8]]

# 类型错误
a = np.array([1, 2, 3], dtype=np.int32)
b = np.array([1.5, 2.5, 3.5], dtype=np.float64)
c = a + b # int32转换为float64后运算
print(c) # [2.5 4.5 6.5]
```

本章小结

本章介绍了NumPy的基础知识，主要包括：

1. NumPy数组创建：从Python列表转换、预定义数组（zeros、ones、eye等）、范围数组（arange、linspace）、随机数。
2. 数组属性：shape、ndim、size、dtype、itemsize等核心属性，以及数据类型转换。
3. 索引与切片：基本索引、多维数组索引、切片（返回视图）、布尔索引（条件筛选）、花式索引（整数数组索引）。
4. 形状操作：reshape改变形状、transpose转置、squeeze移除单维度、expand_dims添加单维度、连接与分割操作。
5. 数组运算：逐元素运算（算术、比较、通用函数）、矩阵乘法、广播机制（核心概念）、归约操作（sum、mean、std等）。
6. 实用技巧：常用函数速查、文件I/O（保存和加载）、性能优化建议、常见错误处理。

NumPy的高性能数组操作是后续学习PyTorch的基础，很多概念和API设计在PyTorch中都有相似体现。

思考与练习

1. 创建一个 5×5 的矩阵，元素为 1 到 25 的整数，然后：
 - 提取主对角线元素
 - 提取上三角部分
 - 计算矩阵的转置
2. 给定两个数组 `a = np.array([1, 2, 3, 4, 5])` 和 `b = np.array([5, 4, 3, 2, 1])`，使用NumPy计算它们的相关系数（Pearson相关系数），不使用Python循环。
3. 有一个形状为 (100, 32, 32, 3) 的图像数据集（100张 32×32 的RGB图像），请：

- 计算每张图像的均值和标准差
 - 对所有图像按通道计算均值（形状为(3,)的数组）
 - 将像素值归一化到 [0, 1] 范围
4. 实现一个函数，输入一个二维NumPy数组，返回其主对角线元素之和（矩阵迹）。分别用 `np.trace` 和手动实现两种方式完成。
 5. 给定一个随机数组，使用NumPy找出其中第k大的元素，要求时间复杂度优于 $O(n \log n)$ 排序。

PyTorch基础

学习目标

- 理解PyTorch的核心数据结构：Tensor（张量）
- 掌握PyTorch Tensor的创建、操作和基本运算
- 理解PyTorch的自动求导机制（autograd）
- 学会使用PyTorch构建简单的神经网络
- 掌握PyTorch的数据加载和训练流程
- 了解模型的保存与加载方法

3.1 PyTorch概述

PyTorch是Facebook开发的开源深度学习框架，以其动态计算图（Dynamic Graph）和Python优先的设计理念，成为当前最流行的深度学习框架之一。

Computation

3.1.1 安装与导入

```
# 安装PyTorch（CPU版本）
# pip install torch

# 安装PyTorch（CUDA版本，根据CUDA版本选择）
# pip install torch --index-url https://download.pytorch.org/whl/cu118

# 导入PyTorch
import torch
```

3.1.2 Tensor概述

Tensor（张量）是PyTorch的核心数据结构，可以理解为多维数组，与NumPy的ndarray非常相似，但Tensor支持GPU加速。

```
# 标量（0维张量）
scalar = torch.tensor(3.14)
print(scalar)      # tensor(3.1400)
print(scalar.dim()) # 0

# 向量（1维张量）
vector = torch.tensor([1, 2, 3, 4, 5])
print(vector)      # tensor([1, 2, 3, 4, 5])
print(vector.shape) # torch.Size([5])

# 矩阵（2维张量）
matrix = torch.tensor([[1, 2, 3],
                        [4, 5, 6]])
print(matrix)
```

(continues on next page)

(continued from previous page)

```
# tensor([[1, 2, 3],
#         [4, 5, 6]])
print(matrix.shape) # torch.Size([2, 3])

# 3维张量
tensor_3d = torch.randn(2, 3, 4) # 随机初始化的3维张量
print(tensor_3d.shape) # torch.Size([2, 3, 4])
```

3.2 Tensor创建

PyTorch提供了多种创建Tensor的方式。

3.2.1 从数据创建

```
# 从Python列表创建
a = torch.tensor([1, 2, 3, 4, 5])
print(a) # tensor([1, 2, 3, 4, 5])

# 从NumPy数组创建
import numpy as np
np_array = np.array([[1, 2, 3], [4, 5, 6]])
torch_tensor = torch.from_numpy(np_array)
print(torch_tensor)
# tensor([[1, 2, 3],
#         [4, 5, 6]])

# Tensor转NumPy
back_to_np = torch_tensor.numpy()
print(back_to_np)
# [[1 2 3]
#  [4 5 6]]

# 注意：共享内存（修改一方会影响另一方）
torch_tensor[0, 0] = 99
print(np_array[0, 0]) # 99

# 不共享内存的转换
copy_tensor = torch_tensor.clone().numpy()
```

3.2.2 预定义Tensor

```
# 全0Tensor
zeros_1d = torch.zeros(5)
zeros_2d = torch.zeros(3, 4)
zeros_3d = torch.zeros(2, 3, 4)
print(zeros_2d)
# tensor([[[0., 0., 0., 0.],
#         [0., 0., 0., 0.],
#         [0., 0., 0., 0.]])

# 全1Tensor
ones = torch.ones(2, 3)
print(ones)
# tensor([[[1., 1., 1.],
#         [1., 1., 1.]])
```

(continues on next page)

(continued from previous page)

```
# 填充指定值
full = torch.full((2, 3), 7.0)
print(full)
# tensor([[7., 7., 7.],
#         [7., 7., 7.]])

# 单位矩阵
eye = torch.eye(4)
print(eye)
# tensor([[1., 0., 0., 0.],
#         [0., 1., 0., 0.],
#         [0., 0., 1., 0.],
#         [0., 0., 0., 1.]])
```

3.2.3 范围和序列

```
# arange: 类似Python的range
a = torch.arange(10)      # 0到9
print(a) # tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

b = torch.arange(1, 10)   # 1到9
print(b) # tensor([1, 2, 3, 4, 5, 6, 7, 8, 9])

c = torch.arange(0, 10, 2) # 0到9，步长2
print(c) # tensor([0, 2, 4, 6, 8])

# linspace: 等差数列
d = torch.linspace(0, 1, 5) # 0到1之间均匀取5个点
print(d) # tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

3.2.4 随机Tensor

```
# 设置随机种子（结果可复现）
torch.manual_seed(42)

# 均匀分布[0, 1)
uniform = torch.rand(3, 4)
print(uniform)
# tensor([[0.3745, 0.9507, 0.7320, 0.5987],
#         [0.1560, 0.1560, 0.0581, 0.8662],
#         [0.6011, 0.7081, 0.0206, 0.9699]])

# 标准正态分布（均值0，方差1）
normal = torch.randn(3, 4)
print(normal)

# 整数随机Tensor
integers = torch.randint(0, 10, (3, 4)) # 0到9
print(integers)

# 随机打乱
arr = torch.arange(10)
shuffled = arr[torch.randperm(len(arr))] # 随机排列索引
print(shuffled)
```

(continues on next page)

(continued from previous page)

```
# 生成随机初始化的Tensor，指定尺寸
linear = torch.randn(512, 1024) # 权重初始化常用
```

3.3 Tensor属性

```
x = torch.randn(3, 4, 5)

# shape: 形状
print(x.shape)    # torch.Size([3, 4, 5])
print(x.size())   # torch.Size([3, 4, 5])

# ndim/dim: 维度数
print(x.dim())    # 3

# dtype: 数据类型
print(x.dtype)    # torch.float32

# device: 所在设备
print(x.device)   # cpu

# numel: 元素总数
print(x.numel())  # 60

# layout: 内存布局
print(x.layout)   # torch.strided
```

Tensor数据类型:

dtype	说明
torch.float32 / torch.float	32位浮点
torch.float64 / torch.double	64位浮点
torch.float16 / torch.half	16位浮点
torch.int8	8位整数
torch.int16 / torch.short	16位整数
torch.int32 / torch.int	32位整数
torch.int64 / torch.long	64位整数
torch.bool	布尔

```
# 指定数据类型
a = torch.tensor([1, 2, 3], dtype=torch.float32)
print(a.dtype) # torch.float32

# 类型转换
b = a.to(torch.int32)
print(b.dtype) # torch.int32

c = a.int() # 简写形式
print(c.dtype) # torch.int32

# 与NumPy的类型对应
print(torch.float32 == np.float32) # True
print(torch.int64 == np.int64)     # True
```

3.4 Tensor操作

3.4.1 索引与切片

PyTorch的索引和切片语法与NumPy非常相似。

```
x = torch.arange(12).reshape(3, 4)
print(x)
# tensor([[ 0,  1,  2,  3],
#         [ 4,  5,  6,  7],
#         [ 8,  9, 10, 11]])

# 基本索引
print(x[0])    # tensor([0, 1, 2, 3])
print(x[1, 2]) # tensor(6)
print(x[-1, -1]) # tensor(11)

# 切片
print(x[:, 0]) # tensor([0, 4, 8]) (第一列)
print(x[1:, :2]) # tensor([[4, 5], [8, 9]]) (后两行的前两列)
print(x[:, :2]) # tensor([[0, 2], [8, 10]]) (跳行跳列)

# 布尔索引
mask = x > 5
print(mask)
# tensor([[False, False, False, False],
#         [False, False,  True,  True],
#         [ True,  True,  True,  True]])
print(x[mask]) # tensor([ 6,  7,  8,  9, 10, 11])

# 条件索引
print(x[x > 5]) # tensor([ 6,  7,  8,  9, 10, 11])
```

3.4.2 形状操作

```
# reshape: 改变形状
x = torch.arange(12)
y = x.reshape(3, 4)
print(y.shape) # torch.Size([3, 4])

# view: 类似reshape, 但不复制数据
z = x.view(3, 4)

# flatten: 展平为一维
flat = y.flatten()
print(flat.shape) # torch.Size([12])

# squeeze/unsqueeze: 移除/添加单维度
a = torch.randn(1, 3, 1, 4)
print(a.shape) # torch.Size([1, 3, 1, 4])

b = torch.squeeze(a) # 移除所有单维度
print(b.shape) # torch.Size([3, 4])

c = torch.squeeze(a, dim=0) # 移除指定维度的单维度
print(c.shape) # torch.Size([3, 1, 4])
```

(continues on next page)

(continued from previous page)

```

d = torch.unsqueeze(b, dim=0) # 添加单维度
print(d.shape) # torch.Size([1, 3, 4])

# transpose: 转置 (2维)
x = torch.randn(3, 4)
y = x.T # 或 x.transpose(0, 1)
print(y.shape) # torch.Size([4, 3])

# permute: 多维转置
x = torch.randn(2, 3, 4, 5)
y = x.permute(2, 0, 3, 1) # (2,3,4,5) -> (4,2,5,3)
print(y.shape) # torch.Size([4, 2, 5, 3])

# repeat: 重复Tensor
x = torch.tensor([1, 2, 3])
y = x.repeat(3) # 重复3次
print(y) # tensor([1, 2, 3, 1, 2, 3, 1, 2, 3])

z = x.repeat(2, 1) # 沿第0维重复2次, 第1维重复1次
print(z)
# tensor([[1, 2, 3],
#         [1, 2, 3]])

```

3.4.3 连接与分割

```

# cat: 沿现有轴连接
a = torch.randn(2, 3)
b = torch.randn(2, 3)

c = torch.cat([a, b], dim=0) # 沿第0维连接
print(c.shape) # torch.Size([4, 3])

d = torch.cat([a, b], dim=1) # 沿第1维连接
print(d.shape) # torch.Size([2, 6])

# stack: 沿新轴连接 (会创建新维度)
e = torch.stack([a, b], dim=0)
print(e.shape) # torch.Size([2, 2, 3])

# split: 分割
x = torch.randn(8, 4)
# 按大小分割
splits = torch.split(x, [2, 3, 3], dim=0) # 分割为2,3,3

# chunk: 均匀分割
chunks = torch.chunk(x, 4, dim=0) # 沿第0维均匀分割为4份

# split与chunk对比
print([s.shape for s in splits]) # [torch.Size([2, 4]), torch.Size([3, 4]), torch.Size([3, 4])]
print([c.shape for c in chunks]) # [torch.Size([2, 4]), torch.Size([2, 4]), torch.Size([2, 4]), torch.Size([2, 4])]

```

3.5 Tensor运算

3.5.1 逐元素运算

```
a = torch.tensor([1.0, 2.0, 3.0, 4.0])

# 基本算术运算
print(a + 1)  # tensor([2., 3., 4., 5.])
print(a - 1)  # tensor([0., 1., 2., 3.])
print(a * 2)  # tensor([2., 4., 6., 8.])
print(a / 2)  # tensor([0.5000, 1.0000, 1.5000, 2.0000])
print(a ** 2) # tensor([1., 4., 9., 16.])

# 逐元素比较
b = torch.tensor([4, 2, 2, 5])
print(a > b)  # tensor([False, False, True, False])
print(a == b) # tensor([False, True, False, False])

# 常用数学函数
x = torch.tensor([0, 30, 60, 90]) * np.pi / 180
print(torch.sin(x)) # 正弦
print(torch.cos(x)) # 余弦
print(torch.exp(x)) # 指数
print(torch.log(x)) # 自然对数
print(torch.sqrt(x)) # 平方根
```

3.5.2 矩阵运算

```
# 矩阵乘法
A = torch.randn(3, 4)
B = torch.randn(4, 5)
C = torch.mm(A, B) # 或 A @ B
print(C.shape) # torch.Size([3, 5])

# 批量矩阵乘法
A = torch.randn(10, 3, 4)
B = torch.randn(10, 4, 5)
C = torch.bmm(A, B) # 批量矩阵乘法
print(C.shape) # torch.Size([10, 3, 5])

# 更通用的批矩阵乘法
C = torch.matmul(A, B) # 支持广播

# 向量点积
v1 = torch.tensor([1.0, 2.0, 3.0])
v2 = torch.tensor([4.0, 5.0, 6.0])
dot = torch.dot(v1, v2)
print(dot) # tensor(32.)

# 求范数
x = torch.tensor([3.0, 4.0])
l2_norm = torch.norm(x) # 默认L2范数
print(l2_norm) # tensor(5.)

l1_norm = torch.norm(x, p=1) # L1范数
print(l1_norm) # tensor(7.)
```

(continues on next page)

(continued from previous page)

```
# 矩阵转置
A = torch.randn(3, 4)
print(A.T.shape) # torch.Size([4, 3])
```

3.5.3 归约操作

```
a = torch.tensor([[[1, 2, 3],
                   [4, 5, 6]])

# 求和
print(torch.sum(a))      # tensor(21)
print(torch.sum(a, dim=0)) # tensor([5, 7, 9]) (按列)
print(torch.sum(a, dim=1)) # tensor([ 6, 15]) (按行)

# 均值
print(torch.mean(a))      # tensor(3.5000)
print(torch.mean(a, dim=0)) # tensor([2.5000, 3.5000, 4.5000])

# 标准差
print(torch.std(a))      # tensor(1.7078)

# 最大最小值
print(torch.min(a))      # tensor(1)
print(torch.max(a))      # tensor(6)
print(torch.argmax(a))   # tensor(0) (索引)
print(torch.argmin(a))   # tensor(5)

# 沿指定维度求最值
print(torch.max(a, dim=0)) # 返回(values, indices)
values, indices = torch.max(a, dim=1)
print(values) # tensor([3, 6])
print(indices) # tensor([2, 2])

# 累积操作
print(torch.cumsum(a, dim=0))
# tensor([[[1, 2, 3],
#          [5, 7, 9]])
```

3.5.4 广播机制

PyTorch的广播机制与NumPy类似，但需要注意维度匹配。

```
# 基本广播
a = torch.randn(3, 4)
b = torch.randn(4)
c = a + b # b被广播到(3,4)

# 显式添加维度
a = torch.randn(3, 4)
b = torch.randn(3, 1) # (3,1)广播到(3,4)
c = a + b

# 批量处理示例
images = torch.randn(32, 3, 224, 224) # batch_size=32, 3通道, 224x224
mean = torch.mean(images, dim=[2, 3]) # (32, 3)
```

(continues on next page)

(continued from previous page)

```
# mean需要reshape为(32, 3, 1, 1)才能与images广播
mean = mean.reshape(32, 3, 1, 1)
normalized = images - mean
print(normalized.shape) # (32, 3, 224, 224)
```

3.6 自动求导 (autograd)

自动求导 (Automatic Differentiation) 是PyTorch的核心特性之一，能够自动计算梯度。

3.6.1 requires_grad

默认情况下，Tensor不会追踪计算历史。设置 `requires_grad=True` 开启梯度追踪。

```
# 创建需要求导的Tensor
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
print(x.requires_grad) # True

# 简写方式
x = torch.ones(3, requires_grad=True)

# 从不需要求导的Tensor创建时，可显式指定
y = torch.tensor([1.0, 2.0, 3.0])
z = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
w = y + z # w.requires_grad 为 True
```

3.6.2 计算梯度

```
# 示例：计算  $y = x^2$  在  $x = [1, 2, 3]$  处的梯度
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2

# y = [1, 4, 9]
# dy/dx = 2x = [2, 4, 6]

# backward()自动计算梯度
y.sum().backward() # 对标量调用，或对非标量y.sum().backward()

print(x.grad) # tensor([2., 4., 6.])
```

非标量输出的反向传播：

```
# 非标量Tensor需要指定gradient参数或先求和
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2 # y = [1, 4, 9]

# 方式1：求和后反向传播
y.sum().backward()

# 方式2：传入与y形状相同的gradient参数
# y.backward(torch.ones_like(y)) # 效果等同于y.sum().backward()

print(x.grad) # tensor([2., 4., 6.])
```

3.6.3 计算图

PyTorch使用动态计算图（Dynamic Computation Graph），每次运算都会构建新的计算图。

```
# 计算图示例
x = torch.tensor([1.0], requires_grad=True)
y = x + 2    # 创建加法节点
z = y ** 2   # 创建幂运算节点
w = z * 3    # 创建乘法节点

w.backward()
print(x.grad) # dy/dx = ?

# 手动验证: w = 3*(x+2)^2
# dw/dx = 6*(x+2) = 6*3 = 18
print(x.grad) # tensor([18.])
```

叶子节点与依赖节点:

```
x = torch.tensor([1.0], requires_grad=True)
y = x + 2
z = y ** 2

print(x.is_leaf) # True (用户创建的)
print(y.is_leaf) # False (由运算生成)
print(z.is_leaf) # False

print(x.grad_fn) # None (叶子节点没有grad_fn)
print(y.grad_fn) # <AddBackward0>
print(z.grad_fn) # <PowBackward0>
```

3.6.4 梯度累积与 detach

计算图会累积梯度，多次反向传播时梯度会累加。

```
# 梯度累积示例
x = torch.tensor([1.0], requires_grad=True)

# 第一次计算
y = x ** 2
y.backward()
print(x.grad) # tensor([2.])

# 第二次计算前需要清零
x.grad.zero_()

# 第二次计算
z = x ** 3
z.backward()
print(x.grad) # tensor([3.])

# 如果不清零，梯度会累加: 2 + 3 = 5
```

detach(): 分离计算图:

```
x = torch.tensor([1.0], requires_grad=True)
y = x ** 2
```

(continues on next page)

(continued from previous page)

```
# 分离y，使z脱离计算图
z = y.detach()
print(z.requires_grad) # False

# 或者创建不需要梯度的副本
y_no_grad = y.detach().requires_grad_(False)
```

3.6.5 with torch.no_grad()

在推理（inference）阶段，不需要梯度追踪，使用 `no_grad()` 可以节省内存和计算。

```
x = torch.tensor([1.0], requires_grad=True)
y = x ** 2

# 不追踪梯度
with torch.no_grad():
    z = y + 1
    print(z.requires_grad) # False

# 装饰器形式
@torch.no_grad()
def inference(model, x):
    return model(x)

# 或者全局设置
torch.set_grad_enabled(False)
# ... 推理代码 ...
torch.set_grad_enabled(True)
```

3.7 神经网络构建

PyTorch通过 `torch.nn` 模块提供了构建神经网络的工具。

3.7.1 常用层

```
import torch.nn as nn

# Linear（全连接层）
linear = nn.Linear(in_features=10, out_features=5, bias=True)
input_tensor = torch.randn(3, 10) # batch_size=3, 输入特征=10
output = linear(input_tensor)
print(output.shape) # torch.Size([3, 5])

# 权重和偏置
print(linear.weight.shape) # torch.Size([5, 10])
print(linear.bias.shape) # torch.Size([5])

# Conv2d（卷积层）
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
# 输入：(batch, channels, height, width)
input_tensor = torch.randn(1, 3, 32, 32)
output = conv(input_tensor)
print(output.shape) # torch.Size([1, 16, 32, 32])

# MaxPool2d（最大池化）
```

(continues on next page)

(continued from previous page)

```
pool = nn.MaxPool2d(kernel_size=2, stride=2)
output = pool(output)
print(output.shape) # torch.Size([1, 16, 16, 16])

# AvgPool2d (平均池化)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)
output = avg_pool(output)
print(output.shape) # torch.Size([1, 16, 8, 8])
```

3.7.2 激活函数

```
import torch.nn.functional as F

x = torch.randn(5)

# ReLU
relu = F.relu(x)
print(relu) # max(0, x)

# Sigmoid
sigmoid = torch.sigmoid(x)
print(sigmoid) # 1 / (1 + exp(-x))

# Tanh
tanh = torch.tanh(x)
print(tanh)

# Softmax (沿指定维度)
x = torch.randn(2, 5)
softmax = F.softmax(x, dim=1) # 沿第1维归一化
print(softmax) # 每行和为1
```

3.7.3 构建模型

PyTorch有两种构建模型的方式：Sequential和类继承。

方式1: Sequential

```
import torch.nn as nn

# 简单模型用Sequential
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)

# 测试模型
x = torch.randn(32, 784) # batch_size=32
output = model(x)
print(output.shape) # torch.Size([32, 10])
```

方式2: 类继承

```

import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = SimpleNet()
print(model)
# SimpleNet(
#   (fc1): Linear(in_features=784, out_features=256, bias=True)
#   (fc2): Linear(in_features=256, out_features=128, bias=True)
#   (fc3): Linear(in_features=128, out_features=10, bias=True)
#   (relu): ReLU()
# )

# 访问参数
for name, param in model.named_parameters():
    print(f"{name}: {param.shape}")

```

3.7.4 损失函数

```

import torch.nn as nn

# 交叉熵损失（用于分类）
criterion = nn.CrossEntropyLoss()

# 模拟预测和目标
outputs = torch.randn(4, 10) # 4个样本，10个类别
targets = torch.tensor([0, 5, 3, 7]) # 真实类别索引

loss = criterion(outputs, targets)
print(f"CrossEntropyLoss: {loss.item():.4f}")

# MSE损失（用于回归）
mse_loss = nn.MSELoss()
predictions = torch.tensor([[1.0], [2.0], [3.0]])
targets = torch.tensor([[1.5], [2.5], [2.5]])
loss = mse_loss(predictions, targets)
print(f"MSELoss: {loss.item():.4f}")

# BCE损失（二分类）
bce_loss = nn.BCELoss()
sigmoid = nn.Sigmoid()
pred = sigmoid(torch.randn(4, 1))
target = torch.tensor([[1.0], [0.0], [1.0], [0.0]])
loss = bce_loss(pred, target)

```

(continues on next page)

(continued from previous page)

```
print(f"BCELoss: {loss.item():.4f}")
```

3.7.5 优化器

```
import torch.optim as optim

# 创建模型
model = SimpleNet()

# SGD优化器
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# Adam优化器（更常用）
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 典型训练步骤
for data, target in dataloader:
    optimizer.zero_grad() # 清零梯度
    output = model(data) # 前向传播
    loss = criterion(output, target) # 计算损失
    loss.backward() # 反向传播
    optimizer.step() # 更新参数
```

3.8 数据加载

3.8.1 Dataset与DataLoader

```
from torch.utils.data import Dataset, DataLoader

# 自定义数据集
class CustomDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

# 创建数据集
X = torch.randn(1000, 784) # 1000个样本
y = torch.randint(0, 10, (1000,)) # 10个类别
dataset = CustomDataset(X, y)

# 创建数据加载器
dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=0)

# 迭代数据
for batch_data, batch_labels in dataloader:
    print(batch_data.shape, batch_labels.shape)
    # torch.Size([32, 784]) torch.Size([32])
    break
```

3.8.2 torchvision数据集

PyTorch提供了常用的计算机视觉数据集。

```
import torchvision
import torchvision.transforms as transforms

# 下载并加载MNIST数据集
transform = transforms.Compose([
    transforms.ToTensor(), # 转换为Tensor
    transforms.Normalize((0.5,), (0.5,)) # 归一化
])

train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# 创建数据加载器
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

3.9 训练流程

完整的PyTorch训练流程如下：

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# 1. 准备数据
X = torch.randn(1000, 20)
y = torch.randn(1000, 1)
dataset = TensorDataset(X, y)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

# 2. 定义模型
class RegressionNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(20, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )
```

(continues on next page)

(continued from previous page)

```

def forward(self, x):
    return self.layers(x)

model = RegressionNet()
print(model)

# 3. 定义损失函数和优化器
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 4. 训练循环
num_epochs = 10
for epoch in range(num_epochs):
    model.train() # 设置为训练模式
    running_loss = 0.0

    for batch_X, batch_y in dataloader:
        # 前向传播
        predictions = model(batch_X)
        loss = criterion(predictions, batch_y)

        # 反向传播
        optimizer.zero_grad()
        loss.backward()

        # 更新参数
        optimizer.step()

    running_loss += loss.item()

    # 打印epoch平均损失
    avg_loss = running_loss / len(dataloader)
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}")

# 5. 评估模型
model.eval() # 设置为评估模式
with torch.no_grad():
    test_X = torch.randn(100, 20)
    test_y = torch.randn(100, 1)
    predictions = model(test_X)
    test_loss = criterion(predictions, test_y)
    print(f"Test Loss: {test_loss.item():.4f}")

```

3.10 模型保存与加载

3.10.1 保存模型

```

# 保存整个模型（不推荐，包含整个模型结构）
torch.save(model, 'model.pth')

# 保存模型参数（推荐，轻量且灵活）
torch.save(model.state_dict(), 'model_weights.pth')

# 保存检查点（保存训练状态）
checkpoint = {
    'epoch': epoch,

```

(continues on next page)

(continued from previous page)

```
'model_state_dict': model.state_dict(),
'optimizer_state_dict': optimizer.state_dict(),
'loss': loss,
}
torch.save(checkpoint, 'checkpoint.pth')
```

3.10.2 加载模型

```
# 加载整个模型
loaded_model = torch.load('model.pth')

# 加载模型参数
model = RegressionNet() # 先创建模型结构
model.load_state_dict(torch.load('model_weights.pth'))

# 加载检查点
checkpoint = torch.load('checkpoint.pth')
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
epoch = checkpoint['epoch']
loss = checkpoint['loss']
```

3.10.3 GPU训练

```
# 检查GPU是否可用
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# 将模型和数据移动到GPU
model = model.to(device)

# 训练循环中
for batch_X, batch_y in dataloader:
    batch_X = batch_X.to(device)
    batch_y = batch_y.to(device)

    predictions = model(batch_X)
    loss = criterion(predictions, batch_y)
    # ...

# 从GPU移回CPU
model = model.cpu()
predictions = predictions.cpu()
```

本章小结

本章介绍了PyTorch的基础知识，主要包括：

1. Tensor创建：从数据创建、预定义Tensor、范围数组、随机Tensor，以及与NumPy的相互转换。
2. Tensor属性：shape、dtype、device、numel等属性，以及数据类型转换。
3. Tensor操作：索引与切片、形状操作（reshape、view、transpose、permute等）、连接与分割。
4. Tensor运算：逐元素运算、矩阵运算、归约操作、广播机制。
5. 自动求导（autograd）：requires_grad、backward()、计算图、梯度累积、no_grad上下文管理器。

6. 神经网络构建：常用层（Linear、Conv2d、Pool）、激活函数、模型定义方式（Sequential和类继承）、损失函数、
7. 数据加载：Dataset、DataLoader、torchvision数据集。
8. 训练流程：完整的前向传播、反向传播、参数更新循环。
9. 模型保存与加载：state_dict方式、完整模型、检查点保存、GPU/CPU迁移。

PyTorch的动态计算图和Python化API使其成为深度学习研究和实践的首选框架。本章内容为后续构建更复杂的神经网络模

思考与练习

1. 创建两个形状为 (3, 4) 的随机Tensor A和B，实现：
 - 矩阵乘法 $A @ B^T$
 - 沿第0维计算元素-wise乘积（使用广播）
 - 计算每行的L2范数
2. 实现一个简单的多层感知机（MLP），包含两个隐藏层（每层128个神经元），用于对MNIST手写数字进行分类（10类）。
3. 给定一个自定义数据集类 `ImageDataset`，包含图像路径和标签，编写数据增强的transforms pipeline，包括：随机水平翻转、随机旋转（-15到15度）、归一化到[-1,1]。
4. 使用PyTorch实现一个学习率调度器（Learning Rate Scheduler），在训练过程中根据epoch数动态调整学习率：
 - 前5个epoch使用固定学习率0.01
 - 之后每个epoch将学习率衰减为原来的一半
5. 实现一个自定义损失函数 Focal Loss，用于处理类别不平衡的分类问题。Focal Loss的公式为：
 - $FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$
 - 其中 p_t 是模型预测的正确类别的概率， α 和 γ 是超参数。

本章小结

本章系统介绍了Python编程的基础知识，为后续学习AI系统打下坚实基础。

Python基础语法部分涵盖了变量与数据类型（整数、浮点、字符串、布尔值）、控制流语句（if/for/while）以及函数的定义。面向对象编程部分讲解了类与对象的概念、封装（私有属性）、继承（子类化）和多态（统一接口），以及实例方法、类方法。Python高级特性部分介绍了列表推导式、生成器与迭代器（惰性求值节省内存）、装饰器（修改函数行为）以及上下文管理器。NumPy基础部分详细讲解了ndarray数组的创建（从列表、预定义、范围、随机）、属性（shape、dtype、ndim）、索引与切片。PyTorch基础部分介绍了Tensor的创建与操作、自动求导机制（autograd）、神经网络的构建方法（Sequential和类继承）、训练流程。通过本章学习，读者将掌握Python编程的核心技能，理解NumPy的高性能数组操作，并具备使用PyTorch构建简单深度学习模型的能力。

思考与练习

1. Python基础：编写一个函数，接受一个整数列表，返回其中所有奇数的和。例如，输入 [1, 2, 3, 4, 5]，输出 9 (1+3+5)。
2. 面向对象：创建一个 Rectangle 类，包含 width 和 height 属性，以及计算面积和周长的方法。添加属性验证，确保宽度和高度为正数。
3. NumPy：创建一个 5×5 的矩阵，元素为 1 到 25 的整数，然后提取主对角线元素、提取上三角部分、计算矩阵的转置。
4. PyTorch：实现一个简单的多层感知机（MLP），包含两个隐藏层（每层128个神经元），用于对MNIST手写数字进行分类。
5. 综合应用：使用PyTorch实现一个学习率调度器，在训练过程中根据epoch数动态调整学习率：前5个epoch使用固定学习率0.01，之后每个epoch将学习率衰减为原来的一半。

1.4.3 Chap03 数学基础

本章学习目标

1. 掌握线性代数基础知识
2. 理解概率与统计核心概念
3. 了解优化基础理论

前置知识

- Chap01 计算机基础
- Chap02 Python编程

内容导航

线性代数基础

学习目标

1. 理解向量与矩阵的概念，掌握其基本运算
2. 理解行列式与逆矩阵的意义，掌握其计算方法
3. 掌握特征值与特征向量的概念及其计算
4. 理解各种范数的定义与应用场景

引言

线性代数（Linear Algebra）是人工智能和机器学习最重要的数学基础之一。在神经网络的构建中，数据以向量和矩阵的形式表示。本章将系统地介绍线性代数的基础知识，从最基础的向量和矩阵概念开始，逐步深入到矩阵分解、特征值理论以及范数等。

1 向量与矩阵

1.1 向量的概念与表示

向量（Vector）是线性代数中最基本的研究对象。从几何角度来看，向量是有大小和方向的量，可以用空间中带有箭头的线段表示。向量的定义：一个 n 维向量是由 n 个实数（或其他数域中的数）组成的有序数组，通常记作：

$$\vec{a} = (a_1, a_2, \dots, a_n)$$

或者用列形式表示为：

$$\vec{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

其中 a_i 称为向量的第 i 个分量（Component）或元素（Element）。在本书中，我们默认使用列向量表示，但有时为了节省空间，也会使用行向量表示。向量的维度（Dimension）是指它包含的分量个数。一个 n 维向量被称为 n 维向量空间 $\mathbb{R}^n$ 中的元素。向量的转置（Transpose）是指将行向量变为列向量或将列向量变为行向量的操作，记作 $\vec{a}^T$ 或 $\vec{a}^\top$ 。

向量相等的定义：两个向量相等当且仅当它们的维度相同，且对应位置上的所有分量都相等。这意味着即使两个向量在几何上表示不同的方向，只要它们的分量相同，它们就是相等的。

1.2 向量运算

1.2.1 向量加法

向量加法 (Vector Addition) 是将两个维度相同的向量的对应分量相加，得到一个新的同维度向量。设 $\vec{a} = (a_1, a_2, \dots, a_n)$ 和 $\vec{b} = (b_1, b_2, \dots, b_n)$ ，则：

$$\vec{a} + \vec{b} = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$$

向量加法的几何意义是平移。如果我们将向量 $\vec{b}$ 的起点平移到向量 $\vec{a}$ 的终点，则 $\vec{a} + \vec{b}$ 就是从 $\vec{a}$ 的起点指向 $\vec{b}$ 的终点的向量。这在物理上对应力的合成，或者在空间中对应点的平移。

向量加法满足以下运算规律：

- 交换律 (Commutative Law) : $\vec{a} + \vec{b} = \vec{b} + \vec{a}$
- 结合律 (Associative Law) : $(\vec{a} + \vec{b}) + \vec{c} = \vec{a} + (\vec{b} + \vec{c})$
- 存在零向量 (Zero Vector) : $\vec{a} + \vec{0} = \vec{a}$ ，其中 $\vec{0} = (0, 0, \dots, 0)$
- 存在相反向量：对于任意向量 $\vec{a}$ ，存在向量 $-\vec{a}$ 使得 $\vec{a} + (-\vec{a}) = \vec{0}$

1.2.2 标量乘法

标量乘法 (Scalar Multiplication) 是将一个向量与一个标量 (实数) 相乘，结果是向量的每个分量都与该标量相乘。设标量 $k \in \mathbb{R}$ ，向量 $\vec{a} = (a_1, a_2, \dots, a_n)$ ，则：

$$k\vec{a} = (ka_1, ka_2, \dots, ka_n)$$

标量乘法的几何意义是缩放。当 $k > 1$ 时，向量被拉伸；当 $0 < k < 1$ 时，向量被压缩；当 $k < 0$ 时，向量的方向会反转；当 $k = 0$ 时，结果是零向量。

标量乘法满足以下运算规律：

- 结合律： $(kl)\vec{a} = k(l\vec{a})$
- 分配律 (关于标量) : $(k + l)\vec{a} = k\vec{a} + l\vec{a}$
- 分配律 (关于向量) : $k(\vec{a} + \vec{b}) = k\vec{a} + k\vec{b}$
- 单位标量： $1\vec{a} = \vec{a}$

1.2.3 向量内积

向量内积 (Inner Product)，也称为点积 (Dot Product) 或数量积，是两个维度相同的向量的一种重要运算。设 $\vec{a} = (a_1, a_2, \dots, a_n)$ 和 $\vec{b} = (b_1, b_2, \dots, b_n)$ ，则它们的内积定义为：

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

内积的结果是一个标量 (数量)，而不是向量。这是它与后面将介绍的向量外积 (结果仍是向量) 的关键区别。

从几何角度来看，向量内积与夹角密切相关。设 θ 为向量 $\vec{a}$ 和 $\vec{b}$ 之间的夹角 ($0 \leq \theta \leq \pi$)，则：

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

其中 $\|\vec{a}\|$ 和 $\|\vec{b}\|$ 分别是两个向量的模 (长度)，将在后面范数一节详细讨论。这个公式给出了内积的几何解释：当两个向量垂直 (90° 或 $\pi/2$ 弧度) 时， $\cos \theta = 0$ ，内积为零；当两个向量同向 ($\theta = 0$) 时，内积取得最大值 $\|\vec{a}\| \|\vec{b}\|$ ；当两个向量反向 ($\theta = \pi$) 时，内积取得最小值 $-\|\vec{a}\| \|\vec{b}\|$ 。

内积的性质：

- 交换律： $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$
- 分配律： $\vec{a} \cdot (\vec{b} + \vec{c}) = \vec{a} \cdot \vec{b} + \vec{a} \cdot \vec{c}$

- 标量结合: $(k\vec{a}) \cdot \vec{b} = k(\vec{a} \cdot \vec{b})$
- 正定性: $\vec{a} \cdot \vec{a} \geq 0$, 且 $\vec{a} \cdot \vec{a} = 0$ 当且仅当 $\vec{a} = \vec{0}$

在机器学习中, 内积有广泛的应用。例如, 在支持向量机中, 需要计算输入向量与权向量的内积来判断分类边界; 在神经网络中, 内积用于计算激活函数的输入。

1.2.4 向量外积

向量外积 (Outer Product), 也称为叉积 (Cross Product) 或向量积, 是另一种向量运算, 但仅在三维空间中定义。设 $\vec{a} = (a_1, a_2, a_3)$ 和 $\vec{b} = (b_1, b_2, b_3)$ 是三维向量, 则它们的叉积为:

$$\vec{a} \times \vec{b} = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix}$$

叉积的结果是一个向量, 其方向垂直于 $\vec{a}$ 和 $\vec{b}$ 所在的平面, 遵循右手定则: 右手四指从 $\vec{a}$ 指向 $\vec{b}$, 拇指所指的方向即为 $\vec{a} \times \vec{b}$ 的方向。

外积的模与两向量的夹角关系为:

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \|\vec{b}\| \sin \theta$$

这意味着当两向量平行 ($\theta = 0$ 或 π) 时, 外积为零; 当两向量垂直 ($\theta = \pi/2$) 时, 外积的模取得最大值。

外积在计算机图形学、机器人学和物理模拟中有重要应用, 例如计算力矩、判断旋转方向等。

1.3 矩阵的概念与表示

矩阵 (Matrix) 是由 mn 个数排成 m 行 n 列的有序数组, 是向量概念的自然推广。矩阵通常用大写字母表示, 例如:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

其中 a_{ij} 表示矩阵 A 第 i 行第 j 列的元素。矩阵 A 可以记作 $A \in \mathbb{R}^{m \times n}$, 表示它是一个 m 行 n 列的实数矩阵。

矩阵的行数 m 和列数 n 称为矩阵的维度 (Dimension) 或形状 (Shape)。一个 m 行 n 列的矩阵称为 $m \times n$ 矩阵。当 $m = n$ 时, 矩阵称为方阵 (Square Matrix)。方阵有特殊的性质, 在后面的章节中会重点讨论。

矩阵可以看作是由 n 个列向量拼接而成 (列表示) 或由 m 个行向量拼接而成 (行表示)。这种视角在理解矩阵与向量的乘法和转置时非常重要。

行向量与列向量: 只有一行的矩阵称为行向量 (Row Vector), 只有一列的矩阵称为列向量 (Column Vector)。在深度学习中, 我们通常将一个样本表示为列向量, 例如一个包含 784 个像素值的 MNIST 图像被表示为 $\mathbb{R}^{784}$ 中的列向量。

1.4 矩阵运算

1.4.1 矩阵加法

矩阵加法 (Matrix Addition) 要求两个矩阵具有相同的维度, 结果是对应位置上的元素相加。设 $A, B \in \mathbb{R}^{m \times n}$, 则:

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

矩阵加法满足交换律和结合律, 并且存在零矩阵 (所有元素均为 0) 作为加法单位元。

1.4.2 标量乘法

矩阵的标量乘法与向量的标量乘法类似，是将标量与矩阵的每个元素相乘。设标量 k 和矩阵 $A \in \mathbb{R}^{m \times n}$ ，则：

$$(kA)_{ij} = k \cdot a_{ij}$$

1.4.3 矩阵乘法

矩阵乘法 (Matrix Multiplication) 是线性代数中最重要的运算之一，也是深度学习计算的核心。设 $A \in \mathbb{R}^{m \times p}$ ， $B \in \mathbb{R}^{p \times n}$ ，则它们的乘积 $C = AB \in \mathbb{R}^{m \times n}$ 定义为：

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}$$

也就是说，乘积矩阵 C 的第 i 行第 j 列的元素，等于 A 的第 i 行与 B 的第 j 列的对应元素乘积之和。

理解矩阵乘法的几种视角：

1. 行视图：乘积矩阵的第 i 行是 A 的第 i 行左乘矩阵 B 的结果，即 A 的第 i 行与 B 的每个列向量的内积组成的行向量。
2. 列视图：乘积矩阵的第 j 列是矩阵 A 右乘 B 的第 j 列的结果，即 B 的第 j 列与 A 的每个行向量的内积组成的列向量。
3. 向量对视角：乘积矩阵可以看作 A 的行向量与 B 的列向量的所有可能配对内积的集合。

矩阵乘法的维度规则：只有当左侧矩阵的列数等于右侧矩阵的行数时，乘法才有定义。这是“内维度匹配，外维度决定结果”。
 $p) \cdot (p \times n) = (m \times n)$ 。

矩阵乘法的性质：

- 结合律： $(AB)C = A(BC)$
- 分配律： $A(B + C) = AB + AC$ ， $(A + B)C = AC + BC$
- 单位矩阵： $AI = IA = A$ ，其中 I 是单位矩阵
- 一般情况下不满足交换律： $AB \neq BA$ （甚至维度可能不匹配）

这个不满足交换律的性质在深度学习中非常重要。当我们写神经网络的前向传播时，数据的流动顺序是固定的，从输入到输出。

1.4.4 矩阵转置

矩阵转置 (Transpose) 是将矩阵的行和列互换。设 $A \in \mathbb{R}^{m \times n}$ ，则 $A^T \in \mathbb{R}^{n \times m}$ ，且 $(A^T)_{ij} = a_{ji}$ 。

转置运算的性质：

- $(A^T)^T = A$
- $(A + B)^T = A^T + B^T$
- $(AB)^T = B^T A^T$

1.4.5 矩阵求逆

矩阵的逆 (Inverse) 将在后面的专门章节详细讨论，这里先简单介绍概念。对于 n 阶方阵 A ，如果存在矩阵 B 使得 $AB = BA = I_n$ (n 阶单位矩阵)，则称 B 为 A 的逆矩阵，记作 A^{-1} 。不是所有矩阵都有逆矩阵，有逆矩阵的矩阵称为可逆矩阵或非奇异矩阵。

1.5 矩阵在人工智能中的应用

1.5.1 神经网络中的矩阵运算

在神经网络中，矩阵运算无处不在。考虑一个全连接层（也称为线性层或稠密层），假设输入是 m 维向量 $\vec{x} \in \mathbb{R}^m$ ，权重矩阵为 $W \in \mathbb{R}^{n \times m}$ ，偏置向量为 $\vec{b} \in \mathbb{R}^n$ ，则该层的输出为：

$$\vec{y} = W\vec{x} + \vec{b}$$

其中 $W\vec{x}$ 是矩阵与向量的乘法， $W\vec{x} + \vec{b}$ 是向量加法。如果有批量大小为 batch 的小批量数据，每批数据构成一个矩阵 $X \in \mathbb{R}^{\text{batch} \times m}$ ，则输出为 $Y = XW^T + \vec{b}$ ，其中 XW^T 是矩阵与矩阵的乘法。

1.5.2 卷积操作的矩阵表示

卷积神经网络中的卷积操作也可以用矩阵乘法来表示。通过 `im2col` (image to column) 操作，可以将输入特征图展开成矩阵，将卷积核展开成另一个矩阵，然后使用矩阵乘法来高效实现卷积。这种表示在 GPU 上利用高度优化的矩阵乘法库（如 cuBLAS）来加速计算。

1.5.3 注意力机制中的矩阵运算

Transformer 中的注意力机制可以简洁地表示为矩阵运算。设查询（Query）、键（Key）、值（Value）矩阵分别为 Q 、 K 、 V ，则注意力输出为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中 QK^T 是查询与键的矩阵乘法， $\sqrt{d_k}$ 是缩放因子， softmax 是按行应用的。这种表示使得我们可以利用高度优化的矩阵乘法库来加速计算。

2 行列式与逆矩阵

2.1 行列式的定义

行列式（Determinant）是方阵的一个重要标量函数，记作 $\det(A)$ 或 $|A|$ 。行列式最初来源于求解线性方程组的问题，但在数学和物理学中有着广泛的应用。对于 2×2 矩阵，行列式有简单的计算公式：

$$\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

对于 3×3 矩阵，行列式可以通过展开公式计算：

$$\det\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = a_{11} \det\begin{pmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{pmatrix} - a_{12} \det\begin{pmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{pmatrix} + a_{13} \det\begin{pmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{pmatrix}$$

对于 $n \times n$ 矩阵，行列式可以通过按任意行或列展开来递归计算，但这种方法计算复杂度较高。实际中通常使用 LU 分解等高效算法来计算行列式。

行列式的几何意义：在二维空间中，矩阵的行列式的绝对值等于由矩阵行（或列）向量张成的平行四边形的面积；在三维空间中，行列式的绝对值等于由矩阵行（或列）向量张成的平行六面体的体积。

2.2 行列式的性质

行列式具有以下重要性质：

1. 单位矩阵的行列式为 1: $\det(I) = 1$
2. 行列式与转置: $\det(A^T) = \det(A)$
3. 两行交换改变符号: 交换矩阵的两行会使得行列式变号
4. 行倍加不变: 将某行的倍数加到另一行上，行列式不变
5. 倍乘抽出: 将某行乘以标量 k ，行列式乘以 k （这意味着如果矩阵有两行成比例，行列式为零）
6. 乘积性质: $\det(AB) = \det(A)\det(B)$
7. 可逆性与行列式: 矩阵 A 可逆当且仅当 $\det(A) \neq 0$

性质 7 是判断矩阵是否可逆的重要依据。在数值计算中，如果行列式的值非常接近零，通常意味着矩阵接近奇异，在求解线性方程组时会遇到数值不稳定问题。

2.3 逆矩阵的定义与性质

逆矩阵 (Inverse Matrix) 的概念在前文已简要提及。对于 n 阶方阵 A ，如果存在矩阵 B 使得：

$$AB = BA = I_n$$

则称 A 是可逆的 (Invertible) 或非奇异的 (Non-singular)，并称 B 为 A 的逆矩阵，记作 A^{-1} 。

逆矩阵是唯一的。如果存在两个矩阵 B 和 C 都满足上述条件，则 $B = BI = B(AC) = (BA)C = IC = C$ ，因此逆矩阵是唯一的。

逆矩阵的性质：

1. $(A^{-1})^{-1} = A$
2. $(AB)^{-1} = B^{-1}A^{-1}$ (注意顺序反转)
3. $(A^T)^{-1} = (A^{-1})^T$
4. $(kA)^{-1} = \frac{1}{k}A^{-1}$ ($k \neq 0$)

2.4 逆矩阵的计算

2.4.1 伴随矩阵法

对于 2×2 矩阵，其逆矩阵可以直接通过公式计算：

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

更一般地，对于 $n \times n$ 矩阵，可以通过伴随矩阵 (Adjugate Matrix) 来求逆。矩阵 A 的伴随矩阵 $\text{adj}(A)$ 是由 A 的所有代数余子式的转置组成的矩阵。则：

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$$

这种方法在理论推导中很有价值，但由于计算复杂度为 $O(n!)$ ，在实际数值计算中并不使用。

2.4.2 高斯-约旦消元法

高斯-约旦消元法 (Gauss-Jordan Elimination) 是实际计算逆矩阵的常用方法。其基本思想是将增广矩阵 $[A|I]$ 通过初等行变换化为 $[I|A^{-1}]$ 的形式。

初等行变换包括：

1. 交换两行
2. 将某行乘以一个非零标量
3. 将某行的倍数加到另一行上

通过这些变换，我们可以将 A 化为单位矩阵，同时 I 就被化为了 A 的逆矩阵。

2.4.3 LU 分解法

LU 分解将矩阵 A 表示为下三角矩阵 L 和上三角矩阵 U 的乘积： $A = LU$ 。如果 A 可以进行 LU 分解，则求解 $Ax = b$ 可以分解为求解 $Ly = b$ 和 $Ux = y$ ，这两个问题都很容易通过前向和后向代换解决。

对于求逆问题， $Ax = I$ 可以通过 LU 分解高效求解。

2.5 逆矩阵在人工智能中的应用

2.5.1 线性系统的求解

在机器学习中，我们经常需要求解线性系统 $Ax = b$ 。例如，在线性回归的正规方程解中，我们需要计算 $(X^T X)^{-1}$ ；在神经网络的权重更新中，可能要求解某些线性系统。虽然现代深度学习主要使用梯度下降法来优化，但

2.5.2 协方差矩阵的逆

在统计学习和机器学习中，协方差矩阵的逆（精度矩阵）有着重要作用。例如，在高斯马尔可夫随机场中，精度矩阵的求逆是一个关键步骤。然而，当特征数量很大时，直接计算协方差矩阵的逆是困难的（复杂度为 $O(p^3)$ ，其中 p 是特征数）。因此发展出了许多近似方法和稀疏化技术。

2.5.3 矩阵求逆引理

在机器学习中，矩阵求逆引理（Woodbury Matrix Identity）是一个非常有用的工具：

$$(A + UCU^T)^{-1} = A^{-1} - A^{-1}U(C^{-1} + U^T A^{-1}U)^{-1}U^T A^{-1}$$

这个恒等式允许我们用较小的矩阵的逆来近似计算较大矩阵的逆，在处理高维数据时非常有用。例如，在神经网络的循环神经网络中，这个引理被广泛应用于加速计算。

3 特征值与特征向量

3.1 特征值与特征向量的定义

特征值（Eigenvalue）和特征向量（Eigenvector）是矩阵最重要的性质之一，在机器学习、信号处理、量子力学等领域都有广泛应用。设 A 是一个 $n \times n$ 方阵。如果存在非零向量 $\vec{v} \in \mathbb{R}^n$ 和标量 $\lambda \in \mathbb{R}$ 使得：

$$A\vec{v} = \lambda\vec{v}$$

则称 λ 是 A 的一个特征值， $\vec{v}$ 是属于特征值 λ 的一个特征向量。

从几何角度来看，特征向量的含义是：矩阵 A 作用于特征向量 $\vec{v}$ ，只是将 $\vec{v}$ 拉伸或压缩了 λ 倍，而没有改变它的方向（除非 $\lambda < 0$ ，此时方向反转）。这是一个非常强的约束条件，大多数向量在矩阵作用后都会改变方向。

3.2 特征值与特征向量的计算

将特征值方程 $A\vec{v} = \lambda\vec{v}$ 改写为：

$$(A - \lambda I)\vec{v} = \vec{0}$$

这是一个齐次线性方程组。要使这个方程组有非零解，系数矩阵必须是奇异的，即：

$$\det(A - \lambda I) = 0$$

这个方程称为特征方程（Characteristic Equation）或特征多项式（Characteristic Polynomial）。求解这个 n 次多项式方程，可以得到 n 个特征值（可能包含重根和复数根）。

对于每个特征值 λ_i ，解方程组 $(A - \lambda_i I)\vec{v} = \vec{0}$ 可以得到对应的特征向量。由于方程组是奇异的，解空间维度至少为 1（可能更高），因此每个特征值至少对应一个特征向量（可能对应无穷多个，因为特征向量的任何非零标量倍仍是特征向量）。

示例：考虑矩阵 $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ 。

特征方程为：

$$\det(A - \lambda I) = \det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = 0$$

解得 $\lambda_1 = 1$, $\lambda_2 = 3$ 。

对于 $\lambda_1 = 1$ ：解 $(A - I)\vec{v} = \vec{0}$ ，即 $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \vec{v} = \vec{0}$ ，得 $v_1 = -v_2$ ，所以特征向量可以取 $\vec{v}_1 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ 。

对于 $\lambda_2 = 3$ ：解 $(A - 3I)\vec{v} = \vec{0}$ ，即 $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \vec{v} = \vec{0}$ ，得 $v_1 = v_2$ ，所以特征向量可以取 $\vec{v}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ 。

3.3 特征值的性质

设 $A \in \mathbb{R}^{n \times n}$ ，其特征值为 $\lambda_1, \lambda_2, \dots, \lambda_n$ （可能有重复和复数），则：

1. 特征值之和（迹）： $\text{tr}(A) = \sum_{i=1}^n \lambda_i = a_{11} + a_{22} + \dots + a_{nn}$
2. 特征值之积（行列式）： $\det(A) = \prod_{i=1}^n \lambda_i$
3. 幂的特征值：如果 λ 是 A 的特征值，则 λ^k 是 A^k 的特征值
4. 可逆矩阵的特征值：如果 A 可逆，则 $1/\lambda_i$ 是 A^{-1} 的特征值
5. 相似矩阵有相同特征值：如果 $B = P^{-1}AP$ ，则 A 和 B 有相同的特征值

这些性质在理论分析和数值计算中都很实用。例如，如果我们只需要特征值的和（迹），可以直接计算矩阵对角线元素之和。

3.4 对称矩阵的特征值

对称矩阵（ $A = A^T$ ）的特征值有特别好的性质：

1. 实特征值：对称矩阵的特征值一定是实数
2. 正交特征向量：对称矩阵对应不同特征值的特征向量是正交的
3. 谱分解：对称矩阵可以分解为 $A = Q\Lambda Q^T$ ，其中 Q 是正交矩阵， Λ 是对角矩阵

对于实对称矩阵，一定可以进行特征分解（Eigendecomposition），将矩阵表示为特征向量和特征值的组合。这在主成分分析中非常有用。

3.5 特征值与特征向量的应用

3.5.1 主成分分析 (PCA)

主成分分析是一种经典的降维方法，其核心就是利用协方差矩阵的特征值和特征向量。给定数据矩阵 X ，我们首先计算协方差矩阵 $C = \frac{1}{n-1}XX^T$ 。然后计算 C 的特征值和特征向量。较大的特征值对应的特征方向是数据方差最大的方向。选取 k 个最大的特征值对应的特征向量，我们可以将数据投影到低维空间，同时尽可能保留原始数据的信息。

3.5.2 谱聚类

谱聚类（Spectral Clustering）是一类基于图论的聚类算法。其基本思想是：构建数据的相似性图，计算拉普拉斯矩阵的特征值和特征向量，然后使用 k -means 等算法进行聚类。谱聚类的理论基础与图论和矩阵谱理论密切相关。

3.5.3 搜索引擎中的 PageRank

PageRank 算法是 Google 创始人提出的用于网页排名的算法。其核心思想是将互联网抽象为一个有向图，网页之间的链接权重可以通过求解转移概率矩阵的主特征向量来得到。主特征向量（对应特征值为 1）给出了网页的长期访问概率分布，这个分布可以解释为网页的重要性排名。

3.5.4 矩阵幂迭代

在一些机器学习应用中，我们需要计算矩阵的幂，例如在马尔可夫链的平稳分布、图神经网络的消息传递等场景中。幂迭代（Power Iteration）是一种计算矩阵主特征向量的简单迭代算法：反复用矩阵乘以当前向量，并归一化，直到收敛。这个算法在 PageRank 和主题模型（如 LSA）中都有应用。

4 矩阵分解

4.1 矩阵分解的概念

矩阵分解（Matrix Decomposition）是将一个矩阵表示为若干个特殊矩阵乘积的形式。矩阵分解在线性代数理论、数值计算和机器学习等领域都有广泛应用。

不同的应用场景需要不同的矩阵分解形式。例如，当我们关心的是求解线性方程组时，LU 分解最为高效；当我们关心的是数据的降维表示时，SVD（奇异值分解）最为常用；当我们希望得到稀疏的或低秩的表示时，非负矩阵分解（NMF）可能更合适。

4.2 LU 分解

LU 分解 将矩阵 A 表示为下三角矩阵 L 和上三角矩阵 U 的乘积： $A = LU$ 。

- L 是下三角矩阵 (Lower Triangular Matrix)，对角线以上的元素均为零
- U 是上三角矩阵 (Upper Triangular Matrix)，对角线以下的元素均为零

LU 分解在求解线性方程组时特别有用。给定 $Ax = b$ ，我们首先求 $A = LU$ ，然后分别求解 $Ly = b$ (前向代换) 和 $Ux = y$ (后向代换)。

LU 分解的存在条件：并非所有矩阵都有 LU 分解。一个充分条件是矩阵 A 的所有顺序主子式都不为零。但在实际应用中，我们可以通过行交换 (部分主元 LU 分解) 来处理一般情况。

部分主元 LU 分解 ($PA = LU$)：对于可能病态的矩阵，我们通常先对行进行置换 P ，再进行 LU 分解。这种分解在数值计算中更加稳定，是线性代数软件包中求解线性方程组的标准方法。

4.3 QR 分解

QR 分解 将矩阵 A 表示为正交矩阵 Q 和上三角矩阵 R 的乘积： $A = QR$ 。

其中 Q 满足 $Q^T Q = I$ (正交矩阵)， R 是上三角矩阵。

QR 分解有多种计算方法，包括 Gram-Schmidt 正交化、Householder 变换和 Givens 旋转等。其中 Householder 变换是数值最稳定的方法。

QR 分解的应用：

1. 求解最小二乘问题：当方程组 $Ax = b$ 无解时，我们希望找到最小化 $\|Ax - b\|$ 的解 x 。这可以通过 QR 分解高效求解。
2. 特征值计算：QR 算法是计算矩阵特征值的主流算法之一。
3. 正交化：Gram-Schmidt 过程本质上就是 QR 分解。

4.4 奇异值分解 (SVD)

奇异值分解 (Singular Value Decomposition, 简称 SVD) 是最重要和最广泛应用的矩阵分解之一。对于任意 $m \times n$ 矩阵 A ，SVD 分解为：

$$A = U \Sigma V^T$$

其中：

- U 是 $m \times m$ 正交矩阵 (U 的列向量称为左奇异向量)
- Σ 是 $m \times n$ 对角矩阵，对角线上的元素称为奇异值 (non-negative, in descending order)
- V 是 $n \times n$ 正交矩阵 (V 的列向量称为右奇异向量)

SVD 的性质：

1. 存在性：任意矩阵都有 SVD 分解 (这一点比特征分解更普适)
2. 几何意义：SVD 将线性变换分解为旋转、缩放、再旋转的过程
3. 秩的解释：非零奇异值的个数等于矩阵的秩
4. 能量解释：奇异值的平方和等于矩阵的 Frobenius 范数的平方 (Parseval 定理)

截断 SVD：在许多应用中，我们只保留前 k 个最大的奇异值及其对应的奇异向量，得到矩阵的最佳低秩近似。这在降维、

4.5 SVD 在人工智能中的应用

4.5.1 降维与主成分分析

SVD 与主成分分析 (PCA) 有着深刻的联系。实际上, PCA 可以看作是对数据协方差矩阵进行 SVD 分解。设数据矩阵 X 的 SVD 为 $X = U\Sigma V^T$, 则协方差矩阵 $C = \frac{1}{n-1}XX^T = U\frac{\Sigma^2}{n-1}U^T$ 。因此, V 的列向量 (右奇异向量) 给出了数据方差最大的方向, 而奇异值的平方给出了各方向上方差的大小。

通过截断 SVD, 我们可以找到数据的低维表示, 同时最小化重构误差。这正是 PCA 的核心思想。

4.5.2 图像压缩

图像可以表示为像素值矩阵。对图像矩阵进行 SVD 分解, 保留前 k 个最大的奇异值, 我们可以得到原图像的最佳 k 秩近似。对于自然图像, 前几个奇异值通常捕获了大部分的能量, 因此使用很少的奇异值就可以获得较好的图像质量。10-20 个奇异值, 就可以较好地重构一张 512×512 的图像, 实现显著的压缩比。

4.5.3 推荐系统

在推荐系统中, 用户-物品交互矩阵通常非常稀疏 (大多数用户只与少数物品交互)。矩阵分解技术 (如 SVD 及其变体) 是协同过滤算法的核心。通过将用户-物品矩阵分解为用户特征矩阵和物品特征矩阵的乘积, 我们可以预测未交互的项。然而, 原始 SVD 无法处理稀疏矩阵 (因为未定义元素), 因此发展出了各种变体, 如 FunkSVD (通过梯度下降学习隐因子)、ALS (交替最小二乘法) 等。

4.5.4 伪逆

矩阵的伪逆 (Moore-Penrose Pseudoinverse) 可以通过 SVD 方便地定义。设 A 的 SVD 为 $A = U\Sigma V^T$, 则 A 的伪逆为:

$$A^+ = V\Sigma^+U^T$$

其中 Σ^+ 是将 Σ 的非零奇异值取倒数并转置后得到的矩阵。

伪逆在求解线性最小二乘问题中非常重要。当矩阵 A 列满秩时, 最小二乘问题 $Ax = b$ 的解可以表示为 $x = A^+b$ 。

4.6 其他矩阵分解

4.6.1 Cholesky 分解

Cholesky 分解是 LU 分解的特殊形式, 适用于对称正定矩阵。任何对称正定矩阵 A 都可以分解为 $A = LL^T$, 其中 L 是下三角矩阵。Cholesky 分解在求解正定系统的线性方程组时比一般 LU 分解快一倍, 并且不需要行置换。

4.6.2 特征分解

对于可以对角化的矩阵 A (有多少个线性无关的特征向量就能对角化), 可以分解为:

$$A = PDP^{-1}$$

其中 D 是对角矩阵, 对角线上的元素是特征值; P 的列向量是对应的特征向量。

特征分解在分析矩阵的幂、指数等问题时非常有用, 在微分方程和动力系统分析中有重要应用。

4.6.3 非负矩阵分解 (NMF)

非负矩阵分解 (Non-negative Matrix Factorization) 将非负矩阵分解为两个非负矩阵的乘积: $V \approx WH$ 。与 SVD 不同, NMF 强制分解后的矩阵元素都是非负的, 这使得分解结果具有更好的可解释性。在图像处理、文本

5 范数

5.1 范数的定义

范数（Norm）是向量和矩阵大小的一种度量。范数的概念来自数学分析中函数极限的思想，是绝对值概念在高维空间的直观地理解，范数是对向量“长度”的测量。就像在二维空间中我们用欧几里得距离（勾股定理）来测量向量的长度一样，向量范数的定义：如果函数 $\|\cdot\|: \mathbb{R}^n \rightarrow \mathbb{R}$ 满足以下三个条件，则称其为向量范数：

1. 正定性（Positive Definiteness）： $\|\vec{x}\| \geq 0$ ，且 $\|\vec{x}\| = 0$ 当且仅当 $\vec{x} = \vec{0}$
2. 齐次性（Homogeneity）： $\|k\vec{x}\| = |k|\|\vec{x}\|$ （ k 为标量）
3. 三角不等式（Triangle Inequality）： $\|\vec{x} + \vec{y}\| \leq \|\vec{x}\| + \|\vec{y}\|$

类似地，矩阵范数除了满足上述条件外，还需要满足次乘性（Submultiplicativity）： $\|AB\| \leq \|A\|\|B\|$ 。

5.2 常见向量范数

5.2.1 L0 范数

L0 范数定义为向量中非零元素的个数：

$$\|\vec{x}\|_0 = \sum_{i=1}^n 1(x_i \neq 0)$$

其中 $1(\cdot)$ 是指示函数。

L0 范数度量了向量的“稀疏程度”——非零元素越少，向量越稀疏。在压缩感知、稀疏编码等应用中，我们经常希望找到 L0 范数最小的解。然而，L0 范数的优化问题是 NP 难的，因此通常用 L1 范数来近似。

5.2.2 L1 范数

L1 范数（曼哈顿范数或曼哈顿距离）定义为向量元素绝对值之和：

$$\|\vec{x}\|_1 = \sum_{i=1}^n |x_i|$$

L1 范数的几何意义是向量各分量绝对值之和，对应于在网格线上移动从原点到点的距离（就像在曼哈顿街区行走）。在 L1 范数经常被用作正则化项来促进解的稀疏性。

5.2.3 L2 范数

L2 范数（欧几里得范数）是我们最熟悉的范数，定义为：

$$\|\vec{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\vec{x}^\top \vec{x}}$$

L2 范数的几何意义就是我们通常所说的“距离”或“长度”。在机器学习中，L2 范数（也称为欧几里得距离）是最常用的范数之一，用于度量向量之间的差异。

单位球：在不同的范数下，单位球（满足 $\|\vec{x}\| \leq 1$ 的点集）有不同的形状：

- L1 范数的单位球是一个旋转正方形（钻石形）
- L2 范数的单位球就是我们熟悉的单位圆（球）

这些不同的形状在优化和几何直观理解中很有用。

5.2.4 L_p 范数

更一般地，对于 $p \geq 1$ ，L_p 范数定义为：

$$\|\vec{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

- 当 $p = 1$ 时，得到 L1 范数
- 当 $p = 2$ 时，得到 L2 范数
- 当 $p \rightarrow \infty$ 时，得到 L ∞ 范数

5.2.5 L ∞ 范数

L ∞ 范数（无穷范数或切比雪夫范数）定义为向量元素绝对值的最大值：

$$\|\vec{x}\|_\infty = \max_i |x_i|$$

在某些应用中，我们关心的是向量中最大的元素，L ∞ 范数提供了这种度量。

5.3 矩阵范数

5.3.1 Frobenius 范数

Frobenius 范数是矩阵最常用的范数之一，定义为：

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(A^\top A)}$$

Frobenius 范数可以理解为将矩阵展成向量后的 L2 范数。它具有旋转不变性： $\|A\|_F = \|UAV^\top\|_F$ 对于任意正交矩阵 U、V 成立。

在机器学习中，Frobenius 范数经常被用作正则化项来控制模型复杂度，例如在矩阵分解中防止过拟合。

5.3.2 谱范数

矩阵的谱范数（Spectral Norm）是矩阵的最大奇异值：

$$\|A\|_2 = \sigma_{\max}(A) = \sqrt{\lambda_{\max}(A^\top A)}$$

谱范数是算子范数，对应于矩阵作为线性算子将向量映射到向量时的最大放大倍数。对于任意向量 $\vec{x}$ ，有 $\|A\vec{x}\| \leq \|A\|_2 \|\vec{x}\|$ 。

谱范数在分析迭代算法的收敛性、设计和分析神经网络的表达能力等方面有重要作用。

5.3.3 核范数

矩阵的核范数（Nuclear Norm）定义为奇异值的和：

$$\|A\|_* = \sum_{i=1}^r \sigma_i$$

其中 r 是矩阵的秩， σ_i 是奇异值。

核范数在低秩矩阵恢复问题中有重要应用，例如在推荐系统、图像恢复等任务中。核范数是秩函数的凸近似，因此可以用

5.4 范数在机器学习中的应用

5.4.1 损失函数中的范数

在机器学习中，范数经常被用作损失函数来度量预测值与真实值之间的差异：

- L2 损失（均方误差）： $\|\vec{y} - \hat{\vec{y}}\|_2^2$ ，对离群点敏感
- L1 损失（平均绝对误差）： $\|\vec{y} - \hat{\vec{y}}\|_1$ ，对离群点更鲁棒
- L^∞ 损失： $\|\vec{y} - \hat{\vec{y}}\|_\infty$ ，关注最大误差

不同的范数对误差的惩罚方式不同，L2 范数对大误差惩罚更重（平方增长），而 L1 范数对所有误差线性惩罚， L^∞ 范数只关心最大的误差。

5.4.2 正则化中的范数

范数在正则化中有广泛应用：

- L2 正则化（Ridge Regression）：在损失函数中加入 $\|W\|_F^2$ ，倾向于让权重均匀地小，防止过拟合
- L1 正则化（Lasso）：在损失函数中加入 $\|W\|_1$ ，倾向于产生稀疏的权重矩阵
- 核范数正则化：在矩阵分解问题中加入核范数约束，倾向于产生低秩解

不同的正则化策略会导致不同的解的性质，这在模型压缩、特征选择等场景中非常重要。

5.4.3 梯度下降中的范数

在优化算法的分析中，范数用于度量梯度的“大小”和参数的“尺度”。梯度下降的收敛性分析通常涉及梯度的范数约束。例如，梯度裁剪（Gradient Clipping）是一种防止梯度爆炸的技术，它将梯度的范数限制在一个预设的阈值内：

$$\vec{g} \leftarrow \vec{g} \cdot \min\left(1, \frac{c}{\|\vec{g}\|}\right)$$

其中 c 是阈值。

本章小结

本章系统介绍了线性代数的基础知识，这些知识是理解人工智能算法的数学基础。我们首先学习了向量与矩阵的基本概念，行列式与逆矩阵部分介绍了行列式的定义、性质和计算方法，以及逆矩阵的概念和计算方法。我们了解了矩阵可逆的条件，约旦消元法等实用的求逆算法。

特征值与特征向量是线性代数最核心的概念之一。我们看到特征值刻画了矩阵的“本质”性质，在主成分分析、谱聚类、PCA 等算法中都有核心应用。

矩阵分解部分介绍了 LU 分解、QR 分解和奇异值分解（SVD）。SVD 尤其重要，它对于任意矩阵都存在，在降维、压缩、推荐系统等场景中都有广泛应用。最后，我们介绍了各种范数的概念，包括 L_0 、 L_1 、 L_2 、 L^∞ 范数等，以及矩阵的 Frobenius 范数、谱范数和核范数。范数在机器学习中用于度量误差、正则化和分析算法收敛性。

这些数学工具将在后续的机器学习和深度学习算法中不断出现，读者应该熟练掌握这些基础概念，为深入学习更高级的内容打下坚实基础。

思考与练习

1. 向量运算验证：设 $\vec{a} = (1, 2, 3)$ ， $\vec{b} = (4, 5, 6)$ ，计算 $\vec{a} + \vec{b}$ 、 $3\vec{a}$ 、 $\vec{a} \cdot \vec{b}$ ，并解释每种运算的几何意义。
2. 矩阵乘法维度分析：如果 $A \in \mathbb{R}^{3 \times 4}$ ， $B \in \mathbb{R}^{4 \times 5}$ ， $C \in \mathbb{R}^{5 \times 2}$ ，那么 ABC 的维度是什么？计算 AB 和 $(AB)C$ 的复杂度分别是多少（用乘法次数衡量）？
3. 逆矩阵存在性判断：判断以下矩阵是否可逆，如果可逆，求其逆矩阵：

$$\bullet A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$$\bullet B = \begin{pmatrix} 2 & 4 \\ 3 & 6 \end{pmatrix}$$

- 特征值与特征向量计算：求矩阵 $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$ 的特征值和对应的特征向量，并验证 $A\vec{v} = \lambda\vec{v}$ 对每个特征对成立。
- SVD 的应用思考：考虑一个 100×100 的图像矩阵，如果对其进行 SVD 分解并保留前 10 个奇异值，重构后的矩阵秩是多少？与原矩阵相比可能有什么差异？
- 范数的比较：对于向量 $\vec{x} = (1, -2, 3, -4)$ ，计算其 L1 范数、L2 范数和 L^∞ 范数。如果使用 L1 范数作为正则化项，为什么会倾向于产生稀疏解（提示：考虑单位球的形状）？
- 矩阵分解的应用：在推荐系统中，假设用户-物品评分矩阵 R 是 1000×500 的矩阵（共 1000 个用户，500 个物品）。如果使用 SVD 将其分解为 $R \approx U\Sigma V^T$ ，其中 U 是 $1000 \times k$ ， Σ 是 $k \times k$ ， V 是 $500 \times k$ ，解释这种分解如何用于预测用户对未评分物品的偏好。 k 的选择对预测效果可能有什么影响？

概率与统计

学习目标

- 理解概率论的基本概念，包括概率定义、条件概率和贝叶斯定理
- 掌握随机变量的概念，理解离散和连续随机变量的区别
- 熟悉常见概率分布及其在机器学习中的应用
- 理解期望、方差等统计量的含义，掌握大数定律和中心极限定理

引言

概率论 (Probability Theory) 与统计学 (Statistics) 是人工智能和机器学习的另一核心数学基础。与线性代数不同，概率论在现实世界中，我们面临的问题往往充满不确定性：传感器读数有噪声、天气预测有随机性、用户的点击行为难以精确预测。在机器学习中，概率论的应用无处不在：

- 模型假设：许多机器学习模型隐含地或显式地使用概率分布来建模数据。例如，朴素贝叶斯分类器基于贝叶斯定理。
- 不确定性量化：深度学习中的_dropout_技术可以解释为贝叶斯近似，模型的预测置信度需要通过概率来解释。
- 优化目标：交叉熵 (Cross-Entropy) 等损失函数直接来源于概率论，最大似然估计 (Maximum Likelihood Estimation) 是模型参数学习的基本原则。
- 正则化：贝叶斯方法提供了一种自然的正则化机制，通过先验分布来约束模型参数。

本章将系统地介绍概率论和统计学的基础知识，从概率的定义开始，逐步深入到随机变量、常见分布，再到统计推断的概

1 概率基础

1.1 概率的定义

概率 (Probability) 是描述某事件发生可能性大小的数值。在给出概率的严格数学定义之前，我们先介绍一些基本概念。随机试验 (Random Experiment) 是指在相同条件下可以重复进行，但结果无法准确预测的试验。例如，抛掷一枚硬币、样本空间 (Sample Space) 是随机试验所有可能结果的集合，记作 Ω 。例如：

- 抛掷一枚硬币的样本空间： $\Omega = \{\text{正面}, \text{反面}\}$ 或 $\Omega = \{0, 1\}$
- 掷一颗六面骰子的样本空间： $\Omega = \{1, 2, 3, 4, 5, 6\}$
- 连续观察某股票价格，样本空间是所有可能的价格序列

事件 (Event) 是样本空间的子集。事件可以是单一结果，也可以是多个结果的集合。例如，“掷出偶数点数”是事件 $\{2, 4, 6\}$ 。

1.1.1 概率的公理化定义

现代概率论建立在严格的公理体系之上。1933 年，苏联数学家科尔莫戈罗夫（Andrey Kolmogorov）提出了概率论的公理化定义，至今仍是概率论的基础。

设 Ω 是某随机试验的样本空间， F 是由 Ω 的某些子集构成的集合（事件域）， P 是定义在 F 上的函数。如果 P 满足以下三个公理（Kolmogorov 公理），则称 P 为概率测度，简称概率：

公理 1（非负性）：对于任意事件 A ，有 $P(A) \geq 0$

公理 2（规范性）： $P(\Omega) = 1$ ，即样本空间的概率为 1

公理 3（可列可加性）：对于两两互不相容的事件序列 $A_1, A_2, \dots$ ，（即当 $i \neq j$ 时， $A_i \cap A_j = \emptyset$ ），有：

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

这三个公理看似简单，但它们构成了整个概率论的逻辑基础。从这三个公理出发，可以推导出概率论的许多重要性质，例如：

- $P(\emptyset) = 0$ （空事件的概率为零）
- $P(A) \leq 1$ （任何事件的概率不超过 1）
- $P(\bar{A}) = 1 - P(A)$ （补事件的概率）
- 如果 $A \subseteq B$ ，则 $P(A) \leq P(B)$ （包含关系下概率的单调性）
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ （容斥公式）

1.1.2 概率的频率解释与主观解释

关于“概率是什么”这个问题，有两种主要的哲学立场：

频率解释（Frequentist Interpretation）：概率是长期重复试验中事件发生的频率。例如，如果我们说抛掷硬币正面朝上的概率为 0.5，这意味着如果我们重复抛掷该硬币无限次，正面朝上的比例将趋近于 0.5。这种解释强调概率的客观性和可重复性。

主观解释（Bayesian Interpretation）或贝叶斯解释：概率是对事件发生相信程度的度量，代表主观的置信度。例如，“明天有 70% 的概率下雨”可以理解为对明天会下雨这件事有 70% 的相信程度。这种解释承认概率可以应用于单次事件，并且可以在机器学习中，这两种观点都有其应用场景。频率学派更关注客观的统计性质，常用方法如最大似然估计；贝叶斯学派将

1.2 条件概率

1.2.1 条件概率的定义

条件概率（Conditional Probability）是在已知某些信息（事件 B 已发生）的条件下，事件 A 发生的概率，记作 $P(A|B)$ ，读作“在 B 发生的条件下 A 发生的概率”。

条件概率的定义公式为：

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \text{ 前提是 } P(B) > 0$$

这个公式的直观理解是：当我们已经知道 B 发生时，样本空间缩小为 B ，原来的 $A \cap B$ 成为新的“成功”事件。因此，在 B 发生的条件下 A 发生的概率等于 A 和 B 同时发生的概率除以 B 发生的概率（这是新的“单位 1”）。

示例：考虑掷两颗骰子的试验。设事件 A = “两颗骰子点数之和为 8”，事件 B = “第一颗骰子点数为 4”。

- $P(A) = 5/36$ （36 种等可能结果中，有 (2,6), (3,5), (4,4), (5,3), (6,2) 五种情况）
- $P(B) = 1/6$ （第一颗骰子有 6 种等可能结果，每种出现概率相同）
- $A \cap B = \{(4,4)\}$ ，只有一种情况， $P(A \cap B) = 1/36$

因此， $P(A|B) = P(A \cap B)/P(B) = (1/36)/(1/6) = 1/6 = 6/36$ 。

这个结果符合直接计算：在第一颗骰子已知为 4 的情况下，要使两颗骰子之和为 8，第二颗骰子必须为 4，概率为 1/6。

1.2.2 乘法公式

从条件概率的定义可以直接得到乘法公式（Multiplication Rule）：

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A)$$

更一般地，对于 n 个事件 $A_1, A_2, \dots, A_n$ ，有：

$$P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1)P(A_2|A_1)P(A_3|A_1 \cap A_2) \cdots P(A_n|A_1 \cap \dots \cap A_{n-1})$$

乘法公式在计算多个事件同时发生的概率时非常有用，尤其是当这些事件有依赖关系时。

1.2.3 全概率公式

全概率公式（Law of Total Probability）是概率论中另一个重要工具，它将一个事件的概率分解为若干互不相容的情况的概率。设事件 $B_1, B_2, \dots, B_n$ 是样本空间 Ω 的一个划分（Partition），即满足：

- $B_i \cap B_j = \emptyset$ （两两互不相容）
- $\bigcup_{i=1}^n B_i = \Omega$ （覆盖整个样本空间）
- $P(B_i) > 0$ （每个部分概率为正）

则在一般情况下，对于任意事件 A ，有：

$$P(A) = \sum_{i=1}^n P(A|B_i)P(B_i)$$

全概率公式的直观理解是：事件 A 可能在各种不同的“情形” B_i 下发生，而情形 B_i 发生的概率是 $P(B_i)$ ，在情形 B_i 下 A 发生的条件概率是 $P(A|B_i)$ ，因此 A 的总概率是这些可能情形的加权平均。

1.3 贝叶斯定理

1.3.1 贝叶斯定理的定义

贝叶斯定理（Bayes' Theorem）是概率论中最重要和最有影响力的公式之一，它描述了如何根据新的证据来更新概率判断。贝叶斯定理的公式为：

$$P(B_i|A) = \frac{P(A|B_i)P(B_i)}{P(A)} = \frac{P(A|B_i)P(B_i)}{\sum_{j=1}^n P(A|B_j)P(B_j)}$$

其中：

- $P(B_i)$ 称为先验概率（Prior Probability），是在观察到证据 A 之前对 B_i 的概率判断
- $P(B_i|A)$ 称为后验概率（Posterior Probability），是在观察到证据 A 之后对 B_i 的更新概率
- $P(A|B_i)$ 称为似然（Likelihood），是在假设 B_i 成立的条件下观察到证据 A 的概率
- $P(A)$ 是证据的边缘概率，起归一化常数的作用

贝叶斯定理的本质是：后验概率 $\propto$ 似然 $\times$ 先验。它告诉我们如何将先验知识（先验概率）与新观察到的数据证据（似然）结合起来。

1.3.2 贝叶斯定理的直观解释

让我们通过一个经典例子来理解贝叶斯定理：疾病检测。

假设有一种疾病，发病率为 1%（即人群中随机一人患有该疾病的概率是 0.01）。有一种检测方法可以检测该疾病：

- 如果一个人确实患有该疾病，检测结果呈阳性的概率为 99%（灵敏度）
- 如果一个人没有患有该疾病，检测结果呈阴性的概率为 99%（特异度，即正确拒绝率）

现在的问题是：如果一个人检测结果呈阳性，他真正患有该疾病的概率是多少？

让我们定义事件：

- D : 患有该疾病
- $\bar{D}$: 不患有该疾病
- $+$: 检测结果呈阳性
- $-$: 检测结果呈阴性

已知:

- $P(D) = 0.01$ (发病率, 即先验概率)
- $P(+|D) = 0.99$ (灵敏度)
- $P(-|\bar{D}) = 0.99$ (特异度)
- $P(+|\bar{D}) = 0.01$ (假阳性率)

要求 $P(D|+)$, 即检测阳性时真正患病的概率。

由贝叶斯定理:

$$P(D|+) = \frac{P(+|D)P(D)}{P(+)} = \frac{P(+|D)P(D)}{P(+|D)P(D) + P(+|\bar{D})P(\bar{D})}$$

代入数值:

$$P(D|+) = \frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.01 \times 0.99} = \frac{0.0099}{0.0099 + 0.0099} = \frac{0.0099}{0.0198} = 0.5$$

令人惊讶的是, 即使检测方法的灵敏度和特异度都高达 99%, 但当检测结果呈阳性时, 真正患病的概率只有 50%! 这是因为疾病本身很罕见 (发病率只有 1%), 假阳性 (在不患病的人中约有 1% 检测呈阳性) 与真阳性 (在患者中约有 99% 检测呈阳性) 的数量相当。

这个例子说明了一个重要的教训: 在评估检测结果时, 必须考虑疾病的基率 (base rate)。仅仅知道检测的灵敏度和特异度是不够的, 我们还需要知道疾病的流行程度。这正是贝叶斯定理所教导我们的。

1.3.3 贝叶斯定理在机器学习中的应用

贝叶斯定理在机器学习中有着广泛而深刻的应用:

朴素贝叶斯分类器 (Naive Bayes Classifier): 这是贝叶斯定理最直接的应用之一。给定输入特征 x 和类别标签 y , 朴素贝叶斯分类器通过最大化后验概率来进行分类:

$$\hat{y} = \arg \max_y P(y|x) = \arg \max_y \frac{P(x|y)P(y)}{P(x)} \propto \arg \max_y P(x|y)P(y)$$

“朴素”之处在于假设给定类别标签后, 各特征之间相互条件独立, 即 $P(x|y) = \prod_i P(x_i|y)$ 。尽管这个假设在实际中往往不成立, 但朴素贝叶斯分类器在许多实际问题中表现良好, 并且具有计算效率高。

贝叶斯优化 (Bayesian Optimization): 在超参数调优中, 贝叶斯优化使用贝叶斯定理来构建目标函数的后验分布, 并根据

贝叶斯神经网络 (Bayesian Neural Networks): 传统的神经网络学习一个点估计 (point estimate), 而贝叶斯神经网络学习权重的分布。通过对权重分布进行贝叶斯推断, 我们可以得到预测的不确定性估计, 这又

概率图模型 (Probabilistic Graphical Models): 贝叶斯网络、马尔可夫随机场等概率图模型使用图结构来表示变量之间的

2 随机变量

2.1 随机变量的概念

随机变量 (Random Variable) 是从随机试验的样本空间到实数的函数。随机变量将随机试验的结果数量化, 使得我们可以用数值来描述随机事件。随机变量通常用大写字母表示, 如 X 、 Y 、 Z 。它的取值用小写字母表示, 如 x 、 y 、 z 。

示例:

- 抛掷一枚硬币, 定义随机变量 $X = 1$ 表示正面, $X = 0$ 表示反面
- 掷骰子, 随机变量 X 表示出现的点数

- 观察某网站一天内的访问量，随机变量 X 可以是任意非负整数

根据取值类型的不同，随机变量分为离散随机变量 (Discrete Random Variable) 和连续随机变量 (Continuous Random Variable) 两大类。

2.2 离散随机变量

离散随机变量 的取值是可数的 (有限或无限可数)。例如掷骰子的点数、网站的访问量等。

2.2.1 概率质量函数 (PMF)

对于离散随机变量 X ，概率质量函数 (Probability Mass Function, PMF) 定义为：

$$p_X(x) = P(X = x)$$

即随机变量 X 取值为 x 的概率。

PMF 必须满足以下条件：

1. $p_X(x) \geq 0$ (非负性)
2. $\sum_x p_X(x) = 1$ (归一性)

2.2.2 累积分布函数 (CDF)

累积分布函数 (Cumulative Distribution Function, CDF) 定义为：

$$F_X(x) = P(X \leq x)$$

对于离散随机变量，CDF 是阶梯函数，在每个可能的取值点跳跃，跳跃的高度等于该点的概率质量。

2.3 连续随机变量

连续随机变量 的取值是连续的，通常是某个区间内的所有实数。例如人的身高、测量误差、股票价格等。

2.3.1 概率密度函数 (PDF)

对于连续随机变量 X ，概率密度函数 (Probability Density Function, PDF) $f_X(x)$ 满足：

$$P(a \leq X \leq b) = \int_a^b f_X(x) dx$$

需要特别注意连续随机变量的概率计算：对于连续随机变量，单个点的概率为零，即 $P(X = x) = 0$ 。因此，在计算概率时，区间端点的开闭不影响结果： $P(a \leq X \leq b) = P(a < X < b) = P(a \leq X < b) = P(a < X \leq b)$ 。

PDF 必须满足以下条件：

1. $f_X(x) \geq 0$ (非负性)
2. $\int_{-\infty}^{+\infty} f_X(x) dx = 1$ (归一性)

2.3.2 累积分布函数 (CDF)

对于连续随机变量，CDF 定义与离散情况相同：

$$F_X(x) = P(X \leq x) = \int_{-\infty}^x f_X(t) dt$$

并且有 $f_X(x) = \frac{dF_X(x)}{dx}$ (在 F_X 可导的点)。

2.4 常见离散分布

2.4.1 伯努利分布

伯努利分布 (Bernoulli Distribution) 是最简单的离散分布, 描述单次只有两种结果 (成功/失败) 的随机试验。设随机变量 $X \sim \text{Bernoulli}(p)$, 其中 p 是成功概率 ($0 \leq p \leq 1$), 则:

- PMF: $P(X = 1) = p, P(X = 0) = 1 - p$
- 期望: $E[X] = p$
- 方差: $\text{Var}(X) = p(1 - p)$

伯努利分布在机器学习中用于建模二元分类问题。例如, 在逻辑回归中, 假设给定输入 x , 标签 y 服从伯努利分布。

2.4.2 二项分布

二项分布 (Binomial Distribution) 描述 n 次独立伯努利试验中成功的次数。

设随机变量 $X \sim \text{Binomial}(n, p)$, 则:

- PMF: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$, 其中 $k = 0, 1, 2, \dots, n$
- 期望: $E[X] = np$
- 方差: $\text{Var}(X) = np(1 - p)$

二项分布在统计分析、质量控制、遗传学等领域有广泛应用。在机器学习中, 它可用于建模多个独立二元决策的聚合结果。

2.4.3 泊松分布

泊松分布 (Poisson Distribution) 描述单位时间内随机事件发生的次数, 常用于建模稀有事件。

设随机变量 $X \sim \text{Poisson}(\lambda)$, 其中 $\lambda > 0$ 是单位时间内事件发生的平均次数 (也称为率参数), 则:

- PMF: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, 其中 $k = 0, 1, 2, \dots$
- 期望: $E[X] = \lambda$
- 方差: $\text{Var}(X) = \lambda$

泊松分布的特点是均值等于方差。这个分布在网络流量分析、生物学 (基因突变计数)、金融 (保险索赔次数) 等领域有广泛应用。

2.5 常见连续分布

2.5.1 均匀分布

均匀分布 (Uniform Distribution) 是最简单的连续分布, 在区间 $[a, b]$ 上概率密度均匀。

设随机变量 $X \sim U(a, b)$, 则:

- PDF: $f(x) = \frac{1}{b-a}$, 当 $a \leq x \leq b$; 否则 $f(x) = 0$
- CDF: $F(x) = 0$ (当 $x < a$), $F(x) = \frac{x-a}{b-a}$ (当 $a \leq x \leq b$), $F(x) = 1$ (当 $x > b$)
- 期望: $E[X] = \frac{a+b}{2}$
- 方差: $\text{Var}(X) = \frac{(b-a)^2}{12}$

均匀分布在随机数生成、采样方法 (如逆变换采样) 和初始化策略中扮演重要角色。

2.5.2 正态分布 (高斯分布)

正态分布 (Normal Distribution), 也称高斯分布 (Gaussian Distribution), 是概率论和统计学中最重要的分布。

设随机变量 $X \sim N(\mu, \sigma^2)$, 其中 μ 是均值, σ^2 是方差, 则:

- PDF: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- 期望: $E[X] = \mu$
- 方差: $\text{Var}(X) = \sigma^2$

标准正态分布是 $\mu = 0$, $\sigma^2 = 1$ 的正态分布, 记作 $Z \sim N(0, 1)$, 其 PDF 为:

$$\phi(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}$$

标准正态分布的 CDF 通常记作 $\Phi(z)$ 。

正态分布的重要性源于中心极限定理 (将在后面讨论), 该定理表明大量独立随机变量的和近似服从正态分布。正态分布

2.5.3 指数分布

指数分布 (Exponential Distribution) 描述独立随机事件发生的时间间隔。

设随机变量 $X \sim \text{Exp}(\lambda)$, 其中 $\lambda > 0$ 是率参数, 则:

- PDF: $f(x) = \lambda e^{-\lambda x}$, 当 $x \geq 0$; 否则 $f(x) = 0$
- CDF: $F(x) = 1 - e^{-\lambda x}$, 当 $x \geq 0$
- 期望: $E[X] = \frac{1}{\lambda}$
- 方差: $\text{Var}(X) = \frac{1}{\lambda^2}$

指数分布具有无记忆性 (Memoryless Property): 对于任意 $s, t \geq 0$, 有 $P(X > s + t | X > s) = P(X > t)$ 。这意味着如果已知事件已经发生时间 s , 再等待时间 t 的概率与从头等待时间 t 的概率相同。

指数分布在可靠性工程、排队论、生存分析等领域有广泛应用。在机器学习中, 指数分布可用于建模寿命数据、间隔时间

2.5.4 拉普拉斯分布

拉普拉斯分布 (Laplace Distribution) 与正态分布相似, 但尾部更重 (使用绝对值而非平方)。

设随机变量 $X \sim \text{Laplace}(\mu, b)$, 其中 μ 是位置参数, $b > 0$ 是尺度参数, 则:

- PDF: $f(x) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)$
- 期望: $E[X] = \mu$
- 方差: $\text{Var}(X) = 2b^2$

拉普拉斯分布在信号处理、图像处理 (用于去噪) 和机器学习 (作为拉普拉斯先验用于稀疏估计) 中有所应用。

2.6 分布的混合

现实世界中的数据往往来自多个分布的混合。例如, 一个包含多个类别数据的数据集可以建模为混合分布。

高斯混合模型 (Gaussian Mixture Model, GMM) 是最常用的混合模型, 假设数据来自 K 个高斯分布的混合:

$$P(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

其中 π_k 是混合系数 (满足 $\pi_k \geq 0$, $\sum \pi_k = 1$), $\mathcal{N}(x | \mu_k, \Sigma_k)$ 是第 k 个高斯分量的概率密度。

GMM 在聚类、密度估计、生成模型等任务中有广泛应用。GMM 的参数通常通过 EM 算法 (Expectation-Maximization) 来学习。

3 统计基础

3.1 期望

3.1.1 期望的定义

期望 (Expectation) 或数学期望是随机变量最基本的数字特征之一, 描述了随机变量的“平均水平”或“中心位置”。对于离散随机变量 X , 其 PMF 为 $p_X(x)$, 则期望定义为:

$$E[X] = \sum_x x \cdot p_X(x)$$

对于连续随机变量 X , 其 PDF 为 $f_X(x)$, 则期望定义为:

$$E[X] = \int_{-\infty}^{+\infty} x \cdot f_X(x) dx$$

期望存在的前提是上述求和或积分绝对收敛。

3.1.2 期望的性质

期望具有以下重要性质:

1. 线性性: $E[aX + bY] = aE[X] + bE[Y]$ (对任意随机变量 X 、 Y 和常数 a 、 b 成立, 不要求 X 、 Y 独立)
2. 常数的期望是常数本身: $E[c] = c$
3. 乘积的期望 (独立情况): 如果 X 和 Y 独立, 则 $E[XY] = E[X]E[Y]$
4. 单调性: 如果 $X \leq Y$ (逐点成立), 则 $E[X] \leq E[Y]$

3.1.3 条件期望

条件期望 (Conditional Expectation) 是在给定某条件下随机变量的期望。

- 离散情况: $E[X|Y=y] = \sum_x x \cdot P(X=x|Y=y)$
- 连续情况: $E[X|Y=y] = \int_{-\infty}^{+\infty} x \cdot f_{X|Y}(x|y) dx$

条件期望 $E[X|Y]$ 可以看作 Y 的函数, 因此本身是一个随机变量。这个随机变量的期望等于 X 的无条件期望: $E[E[X|Y]] = E[X]$ 。这称为迭代期望定律 (Law of Iterated Expectations), 在推导许多统计量时非常有用。

3.2 方差

3.2.1 方差的定义

方差 (Variance) 描述随机变量取值与其期望的偏离程度, 是随机变量最重要的散布度量之一。

随机变量 X 的方差定义为:

$$\text{Var}(X) = E[(X - E[X])^2]$$

方差的非负平方根 $\sqrt{\text{Var}(X)}$ 称为标准差 (Standard Deviation), 记作 σ 。

由定义可以直接推导方差的计算公式:

$$\text{Var}(X) = E[X^2] - (E[X])^2$$

这个公式在计算方差时更常用, 因为它避免了在公式中显式写出期望。

3.2.2 方差的性质

方差具有以下性质：

1. 常数的方差为零： $\text{Var}(c) = 0$
2. 标量乘法： $\text{Var}(aX) = a^2\text{Var}(X)$
3. 加法（独立情况）：如果 X 和 Y 独立，则 $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$
4. 一般情况的加法公式： $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$ ，其中协方差 Cov 衡量两个随机变量的共同变化趋势

3.2.3 常见分布的期望与方差

分布	参数	期望	方差
伯努利分布	p	p	$p(1-p)$
二项分布	n, p	np	$np(1-p)$
泊松分布	λ	λ	λ
均匀分布	a, b	$(a+b)/2$	$(b-a)^2/12$
正态分布	μ, σ^2	μ	σ^2
指数分布	λ	$1/\lambda$	$1/\lambda^2$
拉普拉斯分布	μ, b	μ	$2b^2$

3.3 协方差与相关系数

3.3.1 协方差的定义

协方差（Covariance）衡量两个随机变量共同变化的趋势：

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

当 $\text{Cov}(X, Y) > 0$ 时， X 和 Y 有同时偏离其期望的趋势（正相关）；当 $\text{Cov}(X, Y) < 0$ 时， X 和 Y 有向相反方向偏离的趋势（负相关）；当 $\text{Cov}(X, Y) = 0$ 时，称 X 和 Y 不相关。

需要注意的是，不相关只是说明两个随机变量没有线性相关性，它们之间仍可能有非线性关系。

3.3.2 相关系数

相关系数（Correlation Coefficient）或皮尔逊相关系数是对协方差的标准化，消除了量纲影响：

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)}\sqrt{\text{Var}(Y)}} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

相关系数满足 $-1 \leq \rho_{XY} \leq 1$ 。当 $|\rho_{XY}| = 1$ 时， X 和 Y 有完全的线性关系；当 $\rho_{XY} = 0$ 时， X 和 Y 不相关。

3.4 大数定律

3.4.1 大数定律的表述

大数定律（Law of Large Numbers, LLN）是概率论中最重要的定理之一，它描述了随着样本数量的增加，样本均值收敛于期望值。

弱大数定律（Weak Law of Large Numbers）：设 $X_1, X_2, \dots, X_n$ 是独立同分布的随机变量， $E[X_i] = \mu$ ， $\text{Var}(X_i) = \sigma^2$ 有限，则对于任意 $\epsilon > 0$ ，有：

$$\lim_{n \rightarrow \infty} P\left(\left|\frac{1}{n} \sum_{i=1}^n X_i - \mu\right| < \epsilon\right) = 1$$

也就是说，当样本数量 n 趋于无穷时，样本均值 $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ 依概率收敛于期望 μ 。

强大数定律 (Strong Law of Large Numbers)：在相同条件下，样本均值几乎必然 (almost surely) 收敛于期望值，即：

$$P\left(\lim_{n \rightarrow \infty} \bar{X}_n = \mu\right) = 1$$

强大数定律比弱大数定律更强，它表明样本均值不仅依概率收敛，而且以概率1收敛（几乎必然收敛）。

3.4.2 大数定律的意义

大数定律有着深刻的理论和实践意义：

1. 频率解释的理论基础：大数定律为概率的频率解释提供了数学支持。当我们说某事件发生的概率是 p 时，大数定律告诉我们，如果独立重复试验 n 次，事件发生的频率将趋近于 p 。
2. 蒙特卡洛方法的基础：大数定律是蒙特卡洛方法 (Monte Carlo Method) 的理论依据。蒙特卡洛方法通过大量随机采样来估计期望值。
3. 机器学习中的意义：在机器学习中，训练数据上的损失函数值通常被用作真实风险的估计。大数定律保证了当训练数据量增加时，训练损失会收敛于期望损失。

3.5 中心极限定理

3.5.1 中心极限定理的表述

中心极限定理 (Central Limit Theorem, CLT) 是概率论中最令人惊叹的结果之一，它表明大量独立随机变量的和趋向于正态分布。

中心极限定理：设 $X_1, X_2, \dots, X_n$ 是独立同分布的随机变量， $E[X_i] = \mu$ ， $\text{Var}(X_i) = \sigma^2$ 有限 ($\sigma^2 > 0$)。令 $S_n = \sum_{i=1}^n X_i$ ，则：

$$\lim_{n \rightarrow \infty} P\left(\frac{S_n - n\mu}{\sigma\sqrt{n}} \leq x\right) = \Phi(x)$$

其中 $\Phi(x)$ 是标准正态分布的累积分布函数。

换句话说，标准化后的和 $\frac{S_n - n\mu}{\sigma\sqrt{n}}$ 收敛于标准正态分布 $N(0, 1)$ 。

3.5.2 中心极限定理的意义

中心极限定理的意义深远：

1. 正态分布的普适性：中心极限定理解释了为什么正态分布在自然界如此常见。任何受大量微小、独立的随机因素影响的现象，其分布都会趋近于正态分布。
2. 统计推断的基础：中心极限定理为许多统计推断方法（如置信区间、假设检验）提供了理论依据。即使总体分布未知，只要样本量足够大，样本均值的分布就可以近似为正态分布。
3. 机器学习中的应用：在深度学习中，梯度下降的随机扰动、权重的初始化等经常假设为正态分布。中心极限定理为这些假设提供了理论支持。

3.5.3 大数定律与中心极限定理的比较

大数定律和中心极限定理都描述了大量随机变量和的性质，但侧重点不同：

- 大数定律 告诉我们和的均值（除以 n ）收敛于期望值，即 $\bar{X}_n \rightarrow \mu$
- 中心极限定理 告诉我们和的标准化量 $\frac{S_n - n\mu}{\sigma\sqrt{n}}$ 收敛于标准正态分布

两者并不矛盾：大数定律描述了收敛的目标 (μ)，中心极限定理描述了收敛的速度和分布（误差以 $1/\sqrt{n}$ 的速度围绕 μ 分布，渐近正态）。

3.6 统计在机器学习中的应用

3.6.1 最大似然估计

最大似然估计 (Maximum Likelihood Estimation, MLE) 是统计学中最基本的参数估计方法之一。

给定观测数据 $D = \{x_1, x_2, \dots, x_n\}$ 和参数化模型 $P(x|\theta)$ ，似然函数定义为：

$$\mathcal{L}(\theta; D) = P(D|\theta) = \prod_{i=1}^n P(x_i|\theta)$$

最大似然估计选择使似然函数最大化的参数值：

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta; D) = \arg \max_{\theta} \sum_{i=1}^n \log P(x_i | \theta)$$

在机器学习中，许多模型的学习可以归结为最大似然估计：逻辑回归、线性回归（高斯噪声假设）、神经网络等都可以通

3.6.2 偏差-方差分解

在机器学习中，预测误差可以分解为偏差、方差和不可约误差三部分。

设真实函数为 $f(x)$ ，模型在输入 x 上的预测为 $\hat{f}(x)$ ，则均方误差可分解为：

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2$$

其中：

- 偏差 (Bias) : $\text{Bias}[\hat{f}(x)] = E[\hat{f}(x)] - f(x)$ ，度量了模型预测值的平均值与真实值的偏离
- 方差 (Variance) : $\text{Var}[\hat{f}(x)] = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$ ，度量了模型预测值的变化范围
- 不可约误差 (Irreducible Error) : σ^2 ，由标签的固有噪声引起，无法通过改进模型来消除

偏差-方差分解帮助我们理解模型复杂度的选择：简单模型（如线性回归）通常偏差高、方差低，容易欠拟合；复杂模型

3.6.3 贝叶斯统计与正则化

贝叶斯统计提供了一种不同于频率学派的方法来估计参数和量化不确定性。

在贝叶斯框架下，我们不仅估计一个参数值，而是估计参数的完整后验分布：

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

其中 $P(\theta)$ 是先验分布， $P(D|\theta)$ 是似然， $P(\theta|D)$ 是后验分布。

贝叶斯方法自然地提供了不确定性量化，并且在先验分布的选择中隐含了正则化。例如，在贝叶斯线性回归中，如果选择

在深度学习中，Dropout 可以被解释为贝叶斯神经网络的近似。通过在训练时随机丢弃神经元，Dropout 实际上是在对大量的子网络进行集成，并在推理时对预测进行边缘化。

本章小结

本章系统介绍了概率论与统计学的基础知识，这些知识是理解机器学习算法的另一核心数学基础。

在概率基础部分，我们学习了概率的公理化定义，了解了条件概率的概念和计算方法，并深入探讨了贝叶斯定理。贝叶斯随机变量部分介绍了离散和连续随机变量的概念，以及概率质量函数和概率密度函数。我们详细讨论了几种常见的概率分布。统计基础部分介绍了期望和方差的定义与性质，以及协方差和相关系数。我们重点讨论了大数定律和中心极限定理：前者最后，我们简要介绍了统计在机器学习中的应用，包括最大似然估计、偏差-方差分解和贝叶斯统计方法。这些内容为理解后续章节的机器学习算法奠定了基础。

概率论与统计学是数据驱动方法的核心数学基础。在后续的学习中，我们将看到概率论的概念将反复出现在模型定义、推

思考与练习

1. 贝叶斯定理应用：假设某种癌症的发病率为 0.5%（即每 10000 人中有 50 人患病）。有一种早期筛查方法：

- 对于真正患病者，筛查呈阳性的概率（灵敏度）为 99%
- 对于健康人，筛查呈阴性的概率（特异度）为 98%

如果某人筛查结果呈阳性，请计算他真正患病的概率。如果发病率变为 5%（提高了 10 倍），这个概率如何变化？从中你能得到什么启示？

2. 概率分布计算：假设某网站一天的访问量 X 服从泊松分布，均值为 1000 次。
 - 求 $P(X = 1000)$ 和 $P(X = 0)$
 - 求访问量超过 1050 次的概率（使用近似）
 - 解释为什么泊松分布适用于描述这类现象
3. 期望与方差的计算：设随机变量 X 服从参数为 p 的伯努利分布，即 $P(X = 1) = p$, $P(X = 0) = 1 - p$ 。
 - 计算 $E[X]$ 和 $\text{Var}(X)$
 - 如果 $X_1, X_2, \dots, X_n$ 是 n 个独立同分布的伯努利随机变量， $S_n = X_1 + \dots + X_n$ 是什么分布？求 $E[S_n]$ 和 $\text{Var}(S_n)$
 - 大数定律如何描述 S_n/n 的行为？
4. 中心极限定理验证：通过编程模拟，验证中心极限定理。
 - 假设原始分布为均匀分布 $U(0, 1)$ ，取 $n = 30$ 个样本，计算其和 S_n
 - 重复试验 10000 次，绘制 S_n 的直方图
 - 与相同均值和方差的正态分布进行比较，验证中心极限定理
5. 协方差与相关系数：设二维随机变量 (X, Y) 服从以下分布（单位：米）：
 - X 表示某城市居民的身高，假设 $E[X] = 1.70$, $\text{Var}(X) = 0.01$
 - Y 表示同一城市居民的体重，假设 $E[Y] = 65$, $\text{Var}(Y) = 100$
 - X 和 Y 的协方差 $\text{Cov}(X, Y) = 0.5$
 - 计算 X 和 Y 的相关系数 ρ
 - 这个相关系数说明两者之间有什么关系？
 - 如果协方差变为 -0.5，相关系数会如何变化？
6. 偏差-方差分析：考虑一个简单的回归问题，假设真实函数为 $y = \sin(2\pi x)$ ，噪声为加性高斯噪声 $\epsilon \sim N(0, 0.1^2)$ 。

比较以下两种模型的预测误差：

 - 简单模型：常数预测 $\hat{y} = \bar{y}$ （训练集的均值）
 - 复杂模型：5 阶多项式回归

分析两种模型的偏差和方差，解释为什么简单模型可能泛化能力更好（考虑噪声和过拟合）。
7. 似然与后验：在贝叶斯框架下重新考虑一个问题：假设硬币是不均匀的，抛正面朝上的概率为 θ 。进行 $n = 10$ 次独立试验，观察到 $k = 7$ 次正面。
 - 写出似然函数 $P(D|\theta)$
 - 如果使用 $\text{Beta}(\alpha, \beta)$ 作为先验分布（Beta 先验是二项分布的共轭先验），写出后验分布 $P(\theta|D)$
 - 如果选择先验 $\text{Beta}(1, 1)$ （即均匀先验），后验分布是什么？后验均值是多少？
 - 如果选择先验 $\text{Beta}(0.5, 0.5)$ （Jeffreys 先验），后验分布是什么？后验均值是多少？
 - 比较两种先验下的后验均值，哪一个更接近频率学派的最大似然估计 $\hat{\theta} = k/n = 0.7$ ？

优化基础

学习目标

1. 理解优化的基本概念，理解目标函数、梯度等核心概念
2. 掌握梯度下降法的原理、实现和收敛性分析
3. 了解随机梯度下降法及其在大规模机器学习中的应用
4. 熟悉 Momentum、Adam、RMSprop 等高级优化算法

5. 理解约束优化问题，掌握 Lagrange 乘数法和 KKT 条件

引言

优化（Optimization）是数学的一个核心分支，也是机器学习和人工智能最重要的技术基础之一。在机器学习中，“学习”本质上是一个优化过程。例如，在线性回归中，我们希望找到一条直线，使得预测值与真实值之间的均方误差最小；在逻辑回归中，我们最大化似然函数。可以说，没有优化算法，就没有现代机器学习和深度学习。本章将系统介绍优化理论的基础知识，从最简单的梯度下降法等高级优化算法，最后介绍约束优化问题及其解决方法。

理解优化的数学原理对于设计新算法、诊断训练问题（如梯度消失、梯度爆炸）、选择合适的优化策略等都至关重要。

1 优化基础

1.1 优化问题的定义

优化问题（Optimization Problem）的一般形式是：

$$\min_{x \in \mathbb{R}^n} f(x)$$

或者更一般地：

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m$$

其中：

- $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是目标函数（Objective Function）或损失函数（Loss Function）
- $g_i(x) \leq 0$ 是不等式约束（Inequality Constraint）
- 如果还有等式约束 $h_j(x) = 0$ ，则构成更一般的约束优化问题

在机器学习中，优化问题通常没有约束（或约束非常简单），因此无约束优化是最常见的形式。我们将在本章最后一节专门讨论最小化与最大化：由于 $\max f(x) = -\min(-f(x))$ ，最小化和最大化问题是等价的。在机器学习中，我们通常习惯于最小化。

1.2 优化问题的分类

优化问题可以从多个角度进行分类：

根据目标函数的性质：

- 线性规划（Linear Programming）：目标函数和约束都是线性的
- 二次规划（Quadratic Programming）：目标函数是二次的，约束是线性的
- 非线性规划（Nonlinear Programming）：目标函数或约束是非线性的

根据约束情况：

- 无约束优化（Unconstrained Optimization）：没有约束条件
- 约束优化（Constrained Optimization）：有约束条件

根据目标函数的凸性：

- 凸优化（Convex Optimization）：目标函数是凸函数
- 非凸优化（Non-convex Optimization）：目标函数是非凸函数

这种分类非常重要。凸优化问题具有很好的性质：任何局部最优解都是全局最优解。而非凸优化问题可能存在多个局部最优解。

1.3 梯度与海森矩阵

梯度（Gradient）是多元函数导数的推广。设函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，则梯度定义为：

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

梯度是一个向量，指向函数值增长最快的方向。在优化中，我们通常使用梯度的反方向（负梯度）来下降。

海森矩阵（Hessian Matrix）是梯度的雅可比矩阵，定义为：

$$H(x) = \nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

海森矩阵是目标函数的二阶导数，描述了函数的曲率信息。在牛顿法等二阶优化算法中，海森矩阵起着核心作用。

泰勒展开：函数在某点附近的局部行为可以用泰勒级数来近似：

$$f(x + \Delta x) \approx f(x) + \nabla f(x)^\top \Delta x + \frac{1}{2} \Delta x^\top H(x) \Delta x + O(\|\Delta x\|^3)$$

这个展开式是许多优化算法推导的基础。

1.4 局部最优与全局最优

局部最优（Local Optimum）：如果存在 $\epsilon > 0$ ，使得对于所有满足 $\|x - x^*\| < \epsilon$ 的 x ，都有 $f(x) \geq f(x^*)$ ，则称 x^* 是 f 的一个局部最小值点。

全局最优（Global Optimum）：如果对于所有 $x \in \mathbb{R}^n$ ，都有 $f(x) \geq f(x^*)$ ，则称 x^* 是 f 的全局最小值点。

对于凸函数，局部最优就是全局最优。但对于非凸函数（如深度神经网络的目标函数），可能存在多个局部最优解，这些

在深度学习中，损失函数 landscape（损失 landscape，即损失函数在参数空间中的形状）非常复杂，充满了鞍点（saddle points）和局部最优。理解这些性质对于理解为什么深度学习能够成功至关重要——

研究表明，在高维空间中，局部最优可能并不像我们想象的那么可怕，很多局部最优的函数值接近全局最优，而真正的全局

1.5 优化算法概述

优化算法可以大致分为以下几类：

一阶方法（First-order Methods）：只使用梯度（一阶导数）信息。代表算法：梯度下降法、随机梯度下降法。

二阶方法（Second-order Methods）：使用梯度信息和海森矩阵（或其近似）。代表算法：牛顿法、拟牛顿法（如 L-BFGS）。

自适应方法（Adaptive Methods）：自动调整学习率等超参数。代表算法：AdaGrad、RMSprop、Adam。

零阶方法（Zero-order Methods）：不使用梯度信息，只使用函数值。代表算法：网格搜索、贝叶斯优化、进化算法。

在实际应用中，梯度下降法及其变体是机器学习中使用最广泛的优化算法，原因在于：

1. 计算效率高：每次迭代只需要计算梯度
2. 易于实现：算法逻辑清晰
3. 可扩展性强：可以处理大规模数据和高维参数
4. 收敛性有理论保证（在适当条件下）

2 梯度下降法

2.1 梯度下降法的原理

梯度下降法（Gradient Descent）是最基本、最广泛使用的优化算法之一。其核心思想来自一个直观的观察：沿函数梯度的负方向，即 $-\nabla f(x)$ ，梯度下降法的更新规则为：

$$x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)})$$

其中 $\alpha > 0$ 是学习率（Learning Rate）或步长（Step Size），控制每一步移动的距离。

几何解释：梯度 $\nabla f(x)$ 指向函数增长最快的方向，因此负梯度 $-\nabla f(x)$ 指向函数下降最快的方向。梯度下降法就是沿着这个方向迈一小步，然后重新计算梯度，重复这个过程。

一维示例：考虑函数 $f(x) = x^2$ 。梯度为 $\nabla f(x) = 2x$ 。如果当前点在 $x^{(t)} = 2$ ，梯度为4，负梯度方向是-4。更新后 $x^{(t+1)} = 2 - \alpha \cdot 4 = 2 - 4\alpha$ 。如果选择 $\alpha = 0.1$ ，则 $x^{(t+1)} = 2 - 0.4 = 1.6$ ，函数值从 $f(2) = 4$ 下降到 $f(1.6) = 2.56$ ，确实下降了。

泰勒展开解释：由泰勒展开：

$$f(x^{(t+1)}) \approx f(x^{(t)}) + \nabla f(x^{(t)})^\top (x^{(t+1)} - x^{(t)}) = f(x^{(t)}) - \alpha \|\nabla f(x^{(t)})\|^2$$

只要 $\alpha > 0$ ，函数值必然下降（近似意义上）。

2.2 收敛性分析

梯度下降法的收敛性取决于多个因素，包括目标函数的性质和学习率的选择。

凸函数的情况：对于凸函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ，如果梯度是Lipschitz连续的（Lipschitz常数为L），即：

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y$$

并且学习率满足 $\alpha \leq 1/L$ ，则梯度下降法具有线性收敛速度：

$$\|x^{(t)} - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^t \|x^{(0)} - x^*\|^2$$

其中 μ 是函数的强凸参数（strongly convex parameter）， x^* 是全局最优解。

Lipschitz连续梯度是一个常见且合理的假设。它保证函数变化不会太快，梯度不会剧烈波动。

收敛速度分类：

- 线性收敛（Linear Convergence）：误差以几何级数减少，收敛速度用收敛率衡量
- 次线性收敛（Sublinear Convergence）：收敛越来越慢，如 $O(1/t)$
- 超线性收敛（Superlinear Convergence）：收敛越来越快，如 $O(1/t^2)$ （牛顿法）

对于一般非凸函数，梯度下降法只能保证收敛到驻点（stationary point，即梯度为零的点），但不能保证收敛到全局甚至局部最优解。

2.3 学习率的影响

学习率是梯度下降法最重要的超参数，它直接影响算法的收敛性和稳定性。

学习率过大的问题：

1. 震荡：参数在极小值附近来回跳动，无法收敛
2. 发散：函数值不降反升，完全无法收敛
3. 越过极小值：可能越过局部最优或鞍点

学习率过小的问题：

1. 收敛缓慢：需要大量迭代才能接近最优解
2. 容易陷入局部最优：探索能力不足

3. 效率低下：计算资源和时间浪费

学习率选择的艺术：在实际应用中，学习率通常需要通过实验来调整。常见策略包括：

- 固定学习率：简单但可能不是最优的
- 学习率衰减（Learning Rate Decay）：随着训练进行逐渐减小学习率
- 学习率调度（Learning Rate Scheduling）：根据预设策略动态调整学习率

2.4 学习率衰减策略

学习率衰减（Learning Rate Decay）是一种经验上有效且广泛使用的策略，它允许算法在训练初期快速探索，在后期精细调整。常见的衰减策略包括：

步衰减（Step Decay）：每隔固定 epoch 数将学习率乘以一个因子（如 0.5）：

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/T \rfloor}$$

其中 γ 是衰减率（通常为 0.5 或 0.9）， T 是衰减周期。

指数衰减（Exponential Decay）：

$$\alpha_t = \alpha_0 \cdot e^{-\gamma t}$$

余弦衰减（Cosine Decay）：学习率按余弦函数曲线衰减：

$$\alpha_t = \alpha_0 \cdot \frac{1 + \cos(\pi t/T)}{2}$$

余弦衰减在深度学习中有广泛应用，PyTorch 的 CosineAnnealingLR 等调度器实现了这种策略。

线性热身（Linear Warmup）：在训练初期逐渐增大学习率，然后再衰减：

$$\alpha_t = \begin{cases} \alpha_0 \cdot t/T_w & \text{if } t < T_w \\ \text{按策略衰减} & \text{if } t \geq T_w \end{cases}$$

热身策略有助于训练初期参数的稳定性，防止因参数初始化不当导致的大梯度。

2.5 批量梯度下降、随机梯度下降与小批量梯度下降

根据每次迭代使用的样本数量，梯度下降法可以分为三种形式：

批量梯度下降（Batch Gradient Descent, BGD）：每次迭代使用全部训练样本计算梯度：

$$x^{(t+1)} = x^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^n \nabla f_i(x^{(t)})$$

其中 n 是训练样本总数。

- 优点：梯度估计准确，收敛稳定
- 缺点：每次迭代计算量大，不适合大规模数据；容易陷入局部最优（泛化能力可能受限）

随机梯度下降（Stochastic Gradient Descent, SGD）：每次迭代只使用一个样本计算梯度：

$$x^{(t+1)} = x^{(t)} - \alpha \nabla f_i(x^{(t)})$$

其中 i 是随机选择的一个样本。

- 优点：每次迭代计算量小，可以逃离局部最优
- 缺点：梯度估计噪声大，收敛不稳定，可能在极小值附近震荡

小批量梯度下降（Mini-batch Gradient Descent）：每次迭代使用一小批（batch）样本计算梯度：

$$x^{(t+1)} = x^{(t)} - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla f_i(x^{(t)})$$

其中 $\mathcal{B}$ 是大小为 b 的小批量。

这是深度学习中最常用的形式，它结合了批量梯度下降的稳定性和随机梯度下降的效率。小批量大小（batch size）是一个重要超参数，通常在 32 到 256 之间选择。

小批量梯度下降的理论解释：使用小批量而非单个样本，可以获得更准确的梯度估计，减少噪声；而与全量数据相比，小批量（配合 GPU）的效率，同时保持足够的随机性来逃离局部最优。

3 随机梯度下降

3.1 随机梯度下降的原理

随机梯度下降（SGD）是机器学习最重要的优化算法之一，尤其在大规模机器学习中扮演核心角色。其基本思想是用单个样本的梯度来近似整体梯度。在机器学习中，目标函数通常可以写成经验风险的形式：

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(x_i, y_i, \theta)$$

其中 ℓ 是单个样本的损失， n 是样本数量。

计算 $\nabla L(\theta)$ 需要遍历所有 n 个样本，当 n 非常大时（如 ImageNet 有上百万张图片），这变得不可接受。随机梯度下降用单个样本 i_t 的梯度 $\nabla \ell(x_{i_t}, y_{i_t}, \theta)$ 来近似整体梯度：

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla \ell(x_{i_t}, y_{i_t}, \theta^{(t)})$$

3.2 随机梯度的统计性质

随机梯度是真实梯度的无偏估计：

$$E[\nabla \ell(x_{i_t}, y_{i_t}, \theta)] = \nabla L(\theta)$$

这是因为每个样本被选中的概率相等，期望等于整体平均。

然而，随机梯度有方差（variance）：

$$\text{Var}(\nabla \ell(x_i, y_i, \theta)) = \sigma^2$$

这个方差导致随机梯度下降的收敛路径是曲折的，而不是平滑地直奔极小值。

方差与收敛的关系：在优化后期，当参数接近最优解时，梯度的模长趋近于零，但随机梯度的方差可能仍然较大，导致收敛速度变慢。

3.3 收敛性分析

随机梯度下降的收敛性分析比批量梯度下降更复杂，主要使用随机优化的分析工具。

基本假设：

1. 目标函数 $L(\theta)$ 是凸的（或满足其他正则条件）
2. 随机梯度是真实梯度的无偏估计
3. 梯度的二阶矩（variance）有界： $E[\|\nabla \ell(x_i, y_i, \theta)\|^2] \leq G^2$

收敛率：在适当的学习率调度下（如学习率递减），随机梯度下降可以收敛到全局最优，收敛率为 $O(1/\sqrt{T})$ ，其中 T 是迭代次数。

与批量梯度下降的比较：批量梯度下降的收敛率是 $O(1/T)$ （线性收敛），比随机梯度下降快。但批量梯度下降每次迭代需要计算所有样本的梯度，复杂度为 $O(n)$ ，而随机梯度下降是 $O(1)$ 。综合考虑计算成本，对于大规模问题，随机梯度下降往往更有效。

非凸情况：在深度学习中，目标函数是非凸的。理论研究表明，在适当的条件下（如学习率满足 $\sum \alpha_t = \infty$ 和 $\sum \alpha_t^2 < \infty$ ），随机梯度下降可以收敛到驻点（梯度为零的点）。实践中，随机梯度下降通常能够找到不错的局部最优解。

3.4 随机梯度下降在深度学习中的应用

随机梯度下降及其变体是深度学习训练的基础。几乎所有深度学习框架（如 PyTorch、TensorFlow）都内置了 SGD 优化器。

批量大小的影响：在深度学习中，批量大小是一个重要超参数，它影响训练动态和最终性能：

- 大批量（ ≥ 512 ）：梯度估计更准确，训练更稳定，但泛化性能可能下降（sharp minima 问题）；需要更大的学习率
- 小批量（32-128）：梯度有噪声，有助于逃离局部最优和鞍点，泛化性能通常更好；需要较小的学习率

学习率与批量大小的关系：研究表明，学习率应该与批量大小成比例，即大批量使用更大的学习率。Linear Scaling Rule 指出，当批量大小增加 k 倍时，学习率也应该增加 k 倍（需要足够的 warm-up）。

3.5 梯度裁剪

梯度裁剪（Gradient Clipping）是一种防止梯度爆炸的技术，特别适用于训练循环神经网络（RNN）等容易出现梯度爆炸的模型。梯度裁剪的公式：

$$g \leftarrow g \cdot \min \left(1, \frac{c}{\|g\|} \right)$$

或者按范数裁剪：

$$\text{if } \|g\| > c: \quad g \leftarrow \frac{c}{\|g\|} \cdot g$$

其中 c 是阈值（通常为 1-5）， g 是梯度。

何时使用：当梯度范数超过阈值时进行裁剪。这可以防止参数更新过大，避免数值不稳定。梯度裁剪在训练 RNN、Transformer 等模型时几乎是必不可少的。

4 高级优化算法

4.1 Momentum

4.1.1 动量法的原理

动量法（Momentum）是最古老且最有效的优化算法加速技术之一，由 Polyak 在 1964 年提出。其核心思想是模拟物理中动量的概念，让参数更新具有一定的惯性。

标准 SGD 的更新规则是：

$$v^{(t+1)} = \alpha \nabla f(x^{(t)})$$

$$x^{(t+1)} = x^{(t)} - v^{(t+1)}$$

动量法的更新规则是：

$$v^{(t+1)} = \beta v^{(t)} + \alpha \nabla f(x^{(t)})$$

$$x^{(t+1)} = x^{(t)} - v^{(t+1)}$$

其中 $v^{(t)}$ 是速度（velocity）， $\beta \in [0, 1)$ 是动量系数（通常取 0.9 或 0.99）。

物理直观：动量法可以理解为一个球在损失函数曲面上滚动。由于具有动量，球不会在第一个平坦点就停下来，而是会继续滚动。等效形式：动量法还有另一种等效的表达形式：

$$x^{(t+1)} = x^{(t)} - \alpha \nabla f(x^{(t)}) + \beta (x^{(t)} - x^{(t-1)})$$

这表明当前更新方向是当前梯度和上一步更新的组合。

4.1.2 动量法的收敛性

动量法的收敛性分析比标准 SGD 更复杂。在凸函数情况下，动量法可以加速收敛。

收敛速度：对于凸函数，动量法可以达到 $O(1/t)$ 的收敛率（与批量梯度下降相同），但实际收敛速度更快，尤其是对于 ill-conditioned（条件数较大）的函数。

Condition Number 的影响：函数的 condition number（条件数）定义为最大曲率与最小曲率之比。当 condition number 很大时（即函数在不同方向上曲率差异很大），标准梯度下降会走“之”字形路线，收敛缓慢。动量法通过累

4.1.3 动量法在深度学习中的应用

在深度学习中，动量法几乎是训练神经网络的标配。它被内置于所有主流深度学习框架的优化器中（如 PyTorch 的 SGD with momentum）。

学习率与动量的关系：

- 较大的动量系数（ $\beta \rightarrow 1$ ）允许更大的有效学习率
- 当 $\beta = 0.99$ 时，有效学习率可以高达单步学习率的 100 倍
- 但过大的动量可能导致不稳定

Nesterov 动量：Nesterov 动量是动量法的一个变体，它使用“前瞻”梯度来计算更新：

$$\begin{aligned} v^{(t+1)} &= \beta v^{(t)} + \alpha \nabla f(x^{(t)} - \beta v^{(t)}) \\ x^{(t+1)} &= x^{(t)} - v^{(t+1)} \end{aligned}$$

这相当于先按照之前的速度方向走一步，然后再计算梯度校正。Nesterov 动量在某些情况下比标准动量收敛更快。

4.2 AdaGrad

4.2.1 AdaGrad 的原理

AdaGrad (Adaptive Gradient) 是第一个自适应学习率算法，由 Duchi 等人在 2011 年提出。它的核心思想是对每个参数使用不同的学习率，频繁更新的参数学习率较小，稀疏更新的参数学习率较大。

AdaGrad 的更新规则为：

$$\begin{aligned} g^{(t)} &= \nabla f(x^{(t)}) \\ r^{(t)} &= r^{(t-1)} + g^{(t)} \odot g^{(t)} \\ x^{(t+1)} &= x^{(t)} - \frac{\alpha}{\sqrt{r^{(t)}} + \epsilon} \odot g^{(t)} \end{aligned}$$

其中：

- $r^{(t)}$ 是梯度平方的累积和（按元素）
- $\odot$ 是逐元素乘法
- ϵ 是防止除零的小常数（如 10^{-8} ）

直观理解：AdaGrad 对每个维度维护了一个累积的梯度平方和。如果某个维度在过去经常有大的梯度（即更新频繁）， $r^{(t)}$ 会较大，该维度的学习率会被自动调低。反之，如果某个维度稀疏更新（即很少被更新）， $r^{(t)}$ 较小，学习率相对较高。

这对于处理稀疏数据（如自然语言处理中的词嵌入）特别有效，因为词频差异很大，频繁出现的词不需要大学习率，而稀

4.2.2 AdaGrad 的问题

AdaGrad 的一个主要问题是学习率会单调递减，可能导致训练过早停止。随着 $r^{(t)}$ 不断累积，学习率会越来越小，最终趋近于零，参数停止更新。

这在处理凸函数时是可以接受的（学习率递减有助于收敛到最优解），但在非凸的深度学习场景中可能有问题——我们可能还没有找到好的解，学习率就已经太小了。

4.3 RMSprop

4.3.1 RMSprop 的原理

RMSprop (Root Mean Square Propagation) 是 AdaGrad 的改进版本, 由 Hinton 在其 Coursera 课程中提出, 它使用指数移动平均来替代累积和, 从而避免学习率持续下降的问题。

RMSprop 的更新规则:

$$\begin{aligned} g^{(t)} &= \nabla f(x^{(t)}) \\ r^{(t)} &= \rho r^{(t-1)} + (1 - \rho) g^{(t)} \odot g^{(t)} \\ x^{(t+1)} &= x^{(t)} - \frac{\alpha}{\sqrt{r^{(t)}} + \epsilon} \odot g^{(t)} \end{aligned}$$

其中 ρ 是衰减率 (通常取 0.9), 控制指数移动平均的窗口大小。

与 AdaGrad 的区别: AdaGrad 使用所有历史梯度平方的累积和, 而 RMSprop 只使用最近一段时间的梯度 (通过指数移动平均实现)。这使得学习率可以自适应调整, 而不会无限递减。

等效形式: RMSprop 可以与动量结合使用:

$$\begin{aligned} g^{(t)} &= \nabla f(x^{(t)}) \\ r^{(t)} &= \rho r^{(t-1)} + (1 - \rho) g^{(t)} \odot g^{(t)} \\ v^{(t+1)} &= \beta v^{(t)} + \frac{\alpha}{\sqrt{r^{(t)}} + \epsilon} \odot g^{(t)} \\ x^{(t+1)} &= x^{(t)} - v^{(t+1)} \end{aligned}$$

这称为 RMSprop with Momentum, 是深度学习中的常用优化器之一。

4.4 Adam

4.4.1 Adam 的原理

Adam (Adaptive Moment Estimation) 是目前深度学习中使用最广泛的优化器之一, 由 Kingma 和 Ba 在 2015 年提出。Adam 结合了动量法和 RMSprop 的优点, 同时使用了一阶矩估计 (动量) 和二阶矩估计 (梯度平方的移动平均)。

Adam 的更新规则:

参数初始化: $m^{(0)} = 0, v^{(0)} = 0, t = 0$

每次迭代: $t = t + 1, g^{(t)} = \nabla f(x^{(t-1)})$
 $m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1) g^{(t)}$ (一阶矩, 动量)
 $v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2) g^{(t)} \odot g^{(t)}$ (二阶矩)

偏差校正: $\hat{m}^{(t)} = \frac{m^{(t)}}{1 - \beta_1^t}$ (校正一阶矩偏差) $\hat{v}^{(t)} = \frac{v^{(t)}}{1 - \beta_2^t}$ (校正二阶矩偏差)

参数更新: $x^{(t)} = x^{(t-1)} - \frac{\alpha}{\sqrt{\hat{v}^{(t)}} + \epsilon} \odot \hat{m}^{(t)}$

参数说明:

- β_1 : 一阶矩 (动量) 的衰减率, 通常取 0.9
- β_2 : 二阶矩 (梯度平方) 的衰减率, 通常取 0.999
- α : 学习率, 通常取 0.001 (与上述默认值配合使用)
- ϵ : 防止除零的小常数, 通常取 10^{-8}

偏差校正的必要性: 在训练初期, 由于 $m^{(0)} = 0$ 和 $v^{(0)} = 0$, 一阶矩和二阶矩的估计值会有偏差。例如, $m^{(1)} = \beta_1 \cdot 0 + (1 - \beta_1) g^{(1)} = (1 - \beta_1) g^{(1)}$, 而真实值应该是 $g^{(1)}$ 。通过除以 $(1 - \beta_1^t)$ 可以校正这个偏差。随着 t 增大, β_1^t 趋近于零, 校正项趋近于 1, 偏差逐渐消失。

4.4.2 Adam 的优点

Adam 有以下主要优点：

1. 自适应学习率：结合了一阶矩和二阶矩估计，可以自动调整每个参数的学习率
2. 稀疏梯度友好：对稀疏梯度有较好的处理
3. 内存效率高：只存储一阶和二阶矩的估计，不需要存储历史梯度
4. 鲁棒性强：对超参数的选择相对不敏感，默认参数在很多场景下都能工作
5. 收敛速度快：通常比 SGD 收敛更快，适合大规模数据和参数

4.4.3 Adam 的问题与改进

Adam 的问题：

1. 泛化性能：在一些任务上，Adam 的泛化性能不如 SGD（带有动量的随机梯度下降）。这可能是因为自适应学习率 sharp minima。
2. 理论收敛性：在某些情况下，Adam 不能收敛到最优解（虽然实践中通常工作良好）。
3. 方差问题：在后期，Adam 的方差可能较大，导致泛化性能下降。

AdamW：Adam 的一个改进是权重衰减（Weight Decay）方式的改进。在标准 Adam 中，权重衰减是通过 L2 正则化实现的，但 Adam 的自适应学习率会与 L2 正则化产生干扰。AdamW 将权重衰减与优化步骤分离，取得了更好的效果。PyTorch 的 AdamW 优化器实现了这一改进。

Adamax：Adam 的一个变体，使用无穷范数来代替二阶矩估计，在某些场景下更稳定。

RAAdam（Rectified Adam）：在 Adam 中引入学习率预热（warm-up）来改进收敛性。

4.5 学习率总结对比

算法	学习率特点	动量	适用场景
SGD	固定或衰减	可选	收敛稳定的任务，泛化性能好
SGD + Momentum	固定或衰减	有	通用场景
AdaGrad	自适应递减	无	稀疏数据
RMSprop	自适应调整	可选	非稳态问题
Adam	自适应调整	有	默认选择，大规模问题

实践建议：对于大多数深度学习任务，Adam 是一个不错的默认选择，它可以快速收敛。对于追求最优泛化性能的任务，Adam 快速收敛，然后再用 SGD 精调。对于稀疏数据或自然语言处理任务，Adam 特别适合。

5 约束优化

5.1 约束优化问题的定义

约束优化（Constrained Optimization）问题是指在满足一定约束条件的情况下，寻找使目标函数最小化的解。约束优化问题

$$\begin{aligned}
 & \min_x f(x) \\
 & \text{s.t. } g_i(x) \leq 0, \quad i = 1, \dots, m \\
 & \quad h_j(x) = 0, \quad j = 1, \dots, p
 \end{aligned}$$

其中 $g_i(x) \leq 0$ 是不等式约束， $h_j(x) = 0$ 是等式约束。

可行域（Feasible Region）：满足所有约束条件的点 x 的集合称为可行域，记作：

$$\mathcal{F} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, i = 1, \dots, m; h_j(x) = 0, j = 1, \dots, p\}$$

约束优化的目标是找到 $x^* \in \mathcal{F}$ 使得 $f(x^*) = \min_{x \in \mathcal{F}} f(x)$ 。

5.2 约束优化的分类

根据约束的类型，约束优化问题可以分为：

仅含不等式约束： $g_i(x) \leq 0$ ，这是最一般的情况

仅含等式约束： $h_j(x) = 0$ ，等式约束定义了一个流形（manifold）

混合约束：同时含有不等式约束和等式约束

约束的性质：

- 线性/非线性约束：约束函数是线性还是非线性
- 凸/非凸约束：约束定义的可行域是凸集还是非凸集
- 稀疏/稠密约束：约束是稀疏分布还是稠密分布

凸优化问题：如果目标函数是凸函数，不等式约束函数是凸函数，等式约束函数是仿射函数（即线性的），则该问题称为凸优化问题。

5.3 可视化理解约束

二维例子：考虑以下约束优化问题：

$$\begin{aligned} \min_{x_1, x_2} f(x_1, x_2) &= x_1^2 + x_2^2 \\ \text{s.t.} \quad x_1 + x_2 &= 1 \end{aligned}$$

这是一个在直线 $x_1 + x_2 = 1$ 上寻找离原点最近点的问题。几何直观告诉我们，最优解应该是 $(0.5, 0.5)$ ，距离原点 $\sqrt{0.5}$ 。

不等式约束的例子：

$$\begin{aligned} \min_{x_1, x_2} f(x_1, x_2) &= x_1^2 + x_2^2 \\ \text{s.t.} \quad x_1 + x_2 &\leq 1, \quad x_1 \geq 0, \quad x_2 \geq 0 \end{aligned}$$

这个问题的可行域是第一象限中位于直线 $x_1 + x_2 = 1$ 下方的三角形区域。最小值点仍然是 $(0.5, 0.5)$ ，但这次它是一个内部可行点（不是边界约束的交点）。

5.4 拉格朗日乘数法

拉格朗日乘数法（Lagrange Multiplier Method）是求解等式约束优化问题的基本方法。

基本思想：对于等式约束问题 $\min_x f(x) \quad \text{s.t.} \quad h(x) = 0$ ，我们引入拉格朗日乘子 λ 构造拉格朗日函数：

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^\top h(x)$$

其中 $\lambda \in \mathbb{R}^p$ 是拉格朗日乘子向量。

最优性条件：在约束下极小化 f 等价于在无约束下极小化拉格朗日函数。一阶必要条件为：

$$\frac{\partial \mathcal{L}}{\partial x} = \nabla f(x) + J_h(x)^\top \lambda = 0$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = h(x) = 0$$

其中 $J_h(x)$ 是等式约束函数 h 的雅可比矩阵（梯度按行排列）。

几何解释：在最优解处，目标函数的梯度 $\nabla f(x^*)$ 一定位于约束曲面法向量张成的空间中，即 $\nabla f(x^*)$ 是约束梯度 $h'(x^*)$ 的线性组合。这意味着在最优解处，目标函数的梯度与约束曲面正交，约束曲面是目标函数梯度方向的切平面。

二维例子回顾：对于问题 $\min x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 + x_2 = 1$ ：

构造拉格朗日函数： $\mathcal{L} = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 1)$

求偏导并设为零：

- $\frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 + \lambda = 0 \Rightarrow x_1 = -\lambda/2$
- $\frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 + \lambda = 0 \Rightarrow x_2 = -\lambda/2$
- $\frac{\partial \mathcal{L}}{\partial \lambda} = x_1 + x_2 - 1 = 0 \Rightarrow x_1 + x_2 = 1$

由前两式得 $x_1 = x_2$ ，代入第三式得 $2x_1 = 1$ ，即 $x_1 = x_2 = 0.5$ ， $\lambda = -1$ 。

5.5 KKT 条件

KKT 条件（Karush-Kuhn-Tucker Conditions）是拉格朗日乘数法在不等式约束问题上的推广，是约束优化问题最优解的——考虑一般约束优化问题：

$$\begin{aligned} \min_x & f(x) \\ \text{s.t.} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

KKT 条件：

1. Stationarity（平稳性）： $\nabla f(x^*) + \sum_{i=1}^m \mu_i \nabla g_i(x^*) + \sum_{j=1}^p \lambda_j \nabla h_j(x^*) = 0$
2. Primal Feasibility（原始可行性）： $g_i(x^*) \leq 0, \quad i = 1, \dots, m, h_j(x^*) = 0, \quad j = 1, \dots, p$
3. Dual Feasibility（对偶可行性）： $\mu_i \geq 0, \quad i = 1, \dots, m$
4. Complementary Slackness（互补松弛）： $\mu_i g_i(x^*) = 0, \quad i = 1, \dots, m$

互补松弛条件的直观理解：对于每个不等式约束 $g_i(x) \leq 0$ ：

- 如果 $g_i(x^*) < 0$ （约束未绑紧），则 $\mu_i = 0$ （对应的乘子为零）
- 如果 $g_i(x^*) = 0$ （约束绑紧，在可行域边界上），则 $\mu_i \geq 0$ 可以非零

这意味着只有“活跃的”（active）不等式约束才会对对偶变量有贡献。

例子：考虑约束优化问题：

$$\begin{aligned} \min_{x_1, x_2} & (x_1 - 2)^2 + (x_2 - 1)^2 \\ \text{s.t.} & x_1^2 - x_2 \leq 0 \end{aligned}$$

这是一个抛物线约束——可行域是抛物线 $x_2 = x_1^2$ 下方的区域（包括边界）。目标函数是一个二次函数，在 $(2, 1)$ 处取得最小值（无约束情况下）。

检查 $(2, 1)$ 是否在可行域内： $2^2 - 1 = 3 > 0$ ，不满足约束。所以约束是绑紧的（active），最优解在边界 $x_2 = x_1^2$ 上。

设 $x_1 = t, x_2 = t^2$ ，代入目标函数： $f(t) = (t - 2)^2 + (t^2 - 1)^2$ 。求导： $f'(t) = 2(t - 2) + 4t(t^2 - 1) = 2t - 4 + 4t^3 - 4t = 4t^3 - 2t - 4$ 。解 $f'(t) = 0$ 可以得到最优解。

5.6 约束优化在机器学习中的应用

5.6.1 支持向量机（SVM）

支持向量机是一个经典的约束优化问题：

$$\begin{aligned} \min_{w, b} & \frac{1}{2} \|w\|^2 \\ \text{s.t.} & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

这是一个凸二次规划问题，目标是最大化间隔（等价于最小化权重范数），约束是保证所有样本被正确分类且有间隔。

通过引入拉格朗日乘子和求解对偶问题，我们可以得到支持向量的稀疏解——只有少数样本（支持向量）的乘子非零。

5.6.2 神经网络的不确定性量化

在贝叶斯深度学习中，我们经常需要对网络权重施加约束（如非负性、正则化等），这可以通过约束优化来实现。

5.6.3 权重归一化

某些模型架构要求权重满足特定约束（如正交性、归一性等）。权重归一化（Weight Normalization）将权重向量分解为模长和方向两部分：

$$w = \frac{g}{\|v\|} v$$

其中 v 是方向参数， g 是模长参数。通过参数化，我们可以将模长约束转化为无约束参数优化。

5.7 约束优化算法概述

实际求解约束优化问题的算法包括：

惩罚函数法（Penalty Method）：将约束加入目标函数，形成惩罚项：

$$\min_x f(x) + \sigma \sum_{i=1}^m \max(0, g_i(x))^2 + \sigma \sum_{j=1}^p h_j(x)^2$$

当 x 违反约束时，惩罚项增加。增大惩罚系数 σ 可以迫使解进入可行域。

障碍函数法（Barrier Method）：只在可行域内部进行优化，通过障碍函数防止接近边界：

$$\min_x f(x) - \frac{1}{t} \sum_{i=1}^m \log(-g_i(x))$$

当 x 接近边界（ $g_i(x) \rightarrow 0$ ）时，障碍函数值趋近无穷，阻止 x 离开可行域。

增广拉格朗日法（Augmented Lagrangian Method）：结合了惩罚函数和拉格朗日乘数的优点：

$$\mathcal{L}_\rho(x, \lambda) = f(x) + \lambda^\top h(x) + \frac{\rho}{2} \|h(x)\|^2 + \sum_{i=1}^m \mu_i g_i(x) + \frac{\rho}{2} \sum_{i=1}^m \max(0, g_i(x))^2$$

序列最小二乘规划（SQP）：将约束优化问题转化为一系列二次规划子问题来求解，是求解非线性约束优化问题最有效的

本章小结

本章系统介绍了优化的基础知识及其在机器学习中的应用。

我们首先定义了优化问题，理解了优化问题的分类、梯度与海森矩阵等基本概念。梯度下降法是最基本的优化算法，我们介绍了随机梯度下降法是处理大规模机器学习问题的关键算法，它用单个样本或小批量样本的梯度来近似整体梯度，大大提高了效率。高级优化算法部分介绍了动量法、AdaGrad、RMSprop 和 Adam 等算法。动量法通过引入“惯性”来加速收敛并减少震荡；RMSprop 通过自适应调整学习率来适应不同参数的需求；Adam 结合了动量法和 RMSprop 的优点，是目前最广泛使用的优化器。

最后，我们介绍了约束优化问题及其解决方法，包括拉格朗日乘数法和 KKT 条件。这些理论工具在支持向量机、条件随机场等机器学习模型中有重要应用。

优化算法是机器学习的核心，掌握这些基础知识对于理解模型训练过程、诊断训练问题、设计新算法都至关重要。

思考与练习

1. 梯度下降法验证：设目标函数 $f(x) = x^4 - 3x^3 + 2$ ，使用梯度下降法求解最小值：

- 计算梯度 $\nabla f(x)$
- 编写程序实现梯度下降法，初始点 $x_0 = 3$ ，学习率 $\alpha = 0.01$
- 观察迭代过程，理解为什么学习率过大会导致不收敛

- 尝试不同的学习率（如 0.1, 0.05, 0.01），观察收敛行为的差异
2. 学习率调度：在深度学习中，为什么训练后期通常需要降低学习率？
 - 设计一个学习率调度策略：初始学习率 0.1，每 10 个 epoch 降低 10%
 - 思考余弦衰减和线性 warmup 的适用场景
 - 如果学习率一直保持较高，会发生什么？
 3. 随机梯度下降 vs 批量梯度下降：
 - 对于一个有 100 万样本的训练集，比较批量梯度下降和随机梯度下降每次迭代的计算复杂度
 - 假设批量梯度下降每 epoch 需要 1000 秒，使用随机梯度下降大约需要多少时间？
 - 为什么说随机梯度下降的“噪声”有时候是有益的？
 4. 动量法分析：考虑函数 $f(x, y) = x^2 + 10y^2$ （椭圆函数，condition number = 10）。
 - 使用标准梯度下降（学习率适当）迭代 100 步，观察收敛路径
 - 使用动量法（ $\beta = 0.9$ ）迭代 100 步，观察收敛路径有何不同
 - 解释为什么动量法在椭圆函数上表现更好
 5. Adam 算法实现：手动实现 Adam 优化器，包括偏差校正步骤。
 - 初始化 $m_0 = 0, v_0 = 0, t = 0$
 - 给定梯度 g_t ，更新规则是什么？
 - 解释为什么需要偏差校正
 - 使用 Adam 训练一个简单模型（如线性回归），观察收敛过程
 6. AdaGrad 与 RMSprop 比较：AdaGrad 的学习率会随时间单调递减，这可能导致什么问题？
 - RMSprop 如何解决这个问题？
 - 在什么情况下 AdaGrad 仍然可能是更好的选择？
 7. 约束优化 - 拉格朗日乘数法：求解以下约束优化问题： $\min_{x,y} x^2 + y^2 \text{ s.t. } x + y = 1$
 - 构造拉格朗日函数
 - 求偏导并设方程组
 - 解出 x, y 的最优值
 - 验证该点满足约束且是全局最小值
 8. KKT 条件应用：考虑 SVM 的优化问题（简化为二维情况）： $\min_{w,b} \frac{1}{2} \|w\|^2 \text{ s.t. } y_i(w^\top x_i + b) \geq 1$
 - 写出 KKT 条件中的 complementary slackness 条件
 - 解释为什么只有支持向量（落在间隔边界上的样本）的拉格朗日乘子可能非零
 - 为什么 SVM 的解是稀疏的？
 9. 优化算法选择：假设你正在训练一个深度神经网络：
 - 如果训练 loss 一直在震荡但没有明显下降，可能是什么原因？应该怎么调整？
 - 如果训练 loss 下降正常但验证 loss 很高（过拟合），优化算法能解决这个问题吗？
 - 如果模型出现梯度消失/梯度爆炸，应该使用什么技术？
 10. Batch Size 的影响：在深度学习中，批量大小会影响训练动态：
 - 大批量（1024+）和小批量（32-128）的收敛性有何不同？
 - 大批量为什么需要更大的学习率（Linear Scaling Rule）？
 - 为什么小批量通常有更好的泛化性能？
 - 如果 GPU 内存有限，如何在保持小批量优势的同时增大批量？

本章小结

本章系统介绍了人工智能数学基础的三大核心领域：线性代数、概率论与统计学、以及优化理论。

线性代数基础章节涵盖向量与矩阵的基本概念与运算、行列式与逆矩阵的计算、特征值与特征向量的意义，以及矩阵分解。

概率与统计章节从概率的公理化定义出发，深入讲解了条件概率、贝叶斯定理等核心理论，介绍了离散和连续随机变量及分布。

优化基础章节介绍了优化问题的基本概念，重点讲解了梯度下降法和随机梯度下降法的原理与收敛性分析，并详细介绍了牛顿法。

思考与练习

线性代数

- 向量运算验证：设 $\vec{a} = (1, 2, 3)$, $\vec{b} = (4, 5, 6)$ ，计算 $\vec{a} + \vec{b}$ 、 $3\vec{a}$ 、 $\vec{a} \cdot \vec{b}$ ，并解释每种运算的几何意义。
- 矩阵乘法维度分析：如果 $A \in \mathbb{R}^{3 \times 4}$, $B \in \mathbb{R}^{4 \times 5}$, $C \in \mathbb{R}^{5 \times 2}$ ，那么 ABC 的维度是什么？计算 AB 和 $(AB)C$ 的复杂度分别是多少？
- 特征值与特征向量计算：求矩阵 $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$ 的特征值和对应的特征向量，并验证 $A\vec{v} = \lambda\vec{v}$ 对每个特征对成立。
- SVD 应用思考：考虑一个 100×100 的图像矩阵，如果对其进行 SVD 分解并保留前 10 个奇异值，重构后的矩阵秩是多少？
- 范数的比较：对于向量 $\vec{x} = (1, -2, 3, -4)$ ，计算其 L1 范数、L2 范数和 L_∞ 范数。

概率与统计

- 贝叶斯定理应用：假设某种癌症的发病率为 0.5%，检测方法的灵敏度为 99%，特异度为 98%。如果某人筛查结果呈阳性，计算他真正患病的概率。
- 概率分布计算：假设某网站一天的访问量 X 服从泊松分布，均值为 1000 次。求 $P(X = 1000)$ 和 $P(X > 1050)$ 。
- 期望与方差的计算：设随机变量 X 服从参数为 p 的伯努利分布，计算 $E[X]$ 和 $\text{Var}(X)$ 。如果 $X_1, X_2, \dots, X_n$ 是 n 个独立同分布的伯努利随机变量， $S_n = X_1 + \dots + X_n$ 是什么分布？
- 协方差与相关系数：设 X 和 Y 的协方差 $\text{Cov}(X, Y) = 0.5$, $\text{Var}(X) = 0.01$, $\text{Var}(Y) = 100$ ，计算相关系数并解释其含义。
- 偏差-方差分析：考虑回归问题，真实函数为 $y = \sin(2\pi x)$ 。比较简单模型（常数预测）和复杂模型（5 阶多项式回归）的偏差和方差。

优化基础

- 梯度下降法验证：设目标函数 $f(x) = x^4 - 3x^3 + 2$ ，使用梯度下降法求解最小值，观察不同学习率下的收敛行为。
- 随机梯度下降分析：对于一个有 100 万样本的训练集，比较批量梯度下降和随机梯度下降每次迭代的计算复杂度。
- 动量法分析：考虑函数 $f(x, y) = x^2 + 10y^2$ ，使用标准梯度下降和动量法迭代 100 步，比较收敛路径。
- Adam 算法实现：实现 Adam 优化器，包括偏差校正步骤，并用其训练一个简单模型。
- 约束优化：求解约束优化问题 $\min_{x,y} x^2 + y^2$ s.t. $x + y = 1$ ，使用拉格朗日乘数法验证结果。

1.4.4 Chap04 AI系统概述

本章学习目标

- 理解AI系统的基本概念和定义
- 掌握AI系统的整体架构
- 了解AI系统的设计目标

4. 认识AI系统的生态组成

前置知识

- 第一阶段：预备知识（Chap01-03）

内容导航

AI系统基本概念

本节学习目标

1. 理解AI系统的定义和基本概念
2. 掌握AI系统在整个AI技术栈中的定位和作用
3. 了解为什么要学习AI系统
4. 认识AI系统与人工智能、机器学习、深度学习之间的关系

前置知识

- 了解基本的编程概念
- 具备高中数学基础（矩阵、导数基础概念）
- 对计算机系统有初步认识

1.1 什么是AI系统

1.1.1 AI系统的定义

****AI系统（AI System）****是连接硬件和上层应用的中间层软硬件基础设施，是人工智能时代承担类似传统软件栈中数据库如果用建筑来类比，我们可以更好地理解AI系统的位置和作用：

- 地基（硬件层）：类似于建筑的地基，提供了最底层的算力支撑，包括GPU、NPU、TPU等AI专用芯片，以及通用
- 中间层（AI系统层）：类似于建筑的主体结构，连接地基和屋顶，是整个AI系统的核心。这一层包括AI框架（如Py
- 屋顶（应用层）：类似于建筑的屋顶，代表各种AI应用，如图像识别、自然语言处理、推荐系统等。



1.1.2 为什么需要AI系统

在AI时代，算法工程师和研究人员主要通过高级编程语言（如Python）和AI框架提供的API来编写AI算法。那么问题来了，答案在于效率和生产力。

让我们通过一个简单的对比来理解。假设我们要实现一个卷积神经网络LeNet：

方式一：直接使用CUDA + cuDNN API

这种方式需要编写数千行代码，管理内存分配释放，手动拼接模型计算图。以cudnn书写的卷积神经网络LeNet实例为例，

方式二：使用AI框架（如PyTorch）

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        out = F.relu(self.conv1(x))
        out = F.max_pool2d(out, 2)
        out = F.relu(self.conv2(out))
        out = F.max_pool2d(out, 2)
        out = out.view(out.size(0), -1)
        out = F.relu(self.fc1(out))
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

使用AI框架只需要不到20行代码，而直接使用底层API则需要上千行。

AI系统帮助开发者解决了以下核心问题：

- 1. 以Python API供读者编写网络模型计算图结构
- 2. 提供调用基本算子实现，大幅降低开发代码量
- 3. 自动化内存管理，不暴露指针和内存管理给用户
- 4. 实现自动微分功能，自动构建反向传播计算图
- 5. 调用或生成运行时优化代码，调度算子在指定硬件上执行
- 6. 在运行期应用并行算子，提升硬件利用率等动态优化

1.1.3 AI系统与传统软件栈的类比

理解AI系统在整个技术栈中的位置，我们可以将其与传统软件栈进行类比：

传统软件时代	AI时代
数据库	AI框架（训练/推理）
操作系统	AI编译器
中间件	AI芯片_runtime
底层硬件	AI芯片硬件
应用软件	AI应用

传统的本地部署时代，三大基础软件（数据库、操作系统、中间件）实现控制硬件交互、存储管理数据、网络通信调度等云时代同理，形成了IaaS、PaaS、SaaS三层架构，其中PaaS层提供应用开发环境和基础的数据分析管理服务。类比来看，

1.2 AI系统与相关概念的关系

1.2.1 人工智能、机器学习、深度学习与神经网络

在深入学习AI系统之前，我们需要厘清几个常被混淆的概念：人工智能、机器学习、深度学习与神经网络。

可以用如下包含关系来理解：

- 人工智能（AI）：最广泛的概念，是指让计算机系统具有模拟人类智能的能力，包括推理、学习、感知、理解等。
- 机器学习（ML）：实现人工智能的一种方法，通过让计算机从数据中学习规律来完成特定任务，而不需要明确编程。
- 深度学习（DL）：实现机器学习的一种技术，基于深层神经网络，能够自动学习数据的层次化表示。
- 神经网络（NN）：深度学习的具体实现形态，使用神经网络模型来表示各种复杂的函数关系。

简单来说：机器学习是实现AI的一种方法，深度学习是一种实现机器学习的技术，神经网络是深度学习的具体实现形态。后续的章节中，我们将会以“AI”一词代指神经网络这一具体实现形态。

1.2.2 AI系统的边界

需要特别说明的是，AI系统（AI System）和AI基础设施（AI Infra）虽然经常被混用，但严格来说是有区别的：

- AI Infra（AI基础设施）：连接AI训练推理全流程的软件应用及AI中间件基础设施。主要包含大模型从设计、生产到部署的全流程基础设施。
- AI System（AI系统）：AI时代连接硬件和上层应用的中间层基础设施。主要包含底层系统架构、编译器、AI框架等。

本书聚焦于AI System，侧重于纵向的基础软硬件基础设施层面的知识体系。

1.3 AI发展历程

了解AI的发展历程对于理解AI系统的设计目标和演化方向至关重要。AI的发展可以大致分为以下几个阶段：

1.3.1 萌芽期（1950s-1980s）

1950年：图灵测试被提出，用于测试机器是否能表现出与人无法区分的智能。

1956年：约翰·麦卡锡提出“人工智能”概念，掀起了AI发展的第一个高潮。

1957年：Frank Rosenblatt发明感知机（Perceptron），奠定了之后AI的基本结构，其计算以矩阵乘加运算为主。这影响了（Core）。

1960年：Bernard Widrow和Hoff发明了感知器Adaline/Madaline，首次尝试把线性层叠加整合为多层感知器网络。

1969年：Marvin Minsky和Seymour Papert证明了单层感知器无法解决线性不可分问题（例如异或问题），神经网络的研究陷入低谷。

1974年：Paul Werbos在博士论文中提出用误差反向传播来训练人工神经网络，使得训练多层神经网络成为可能，有效解决了非线性问题。

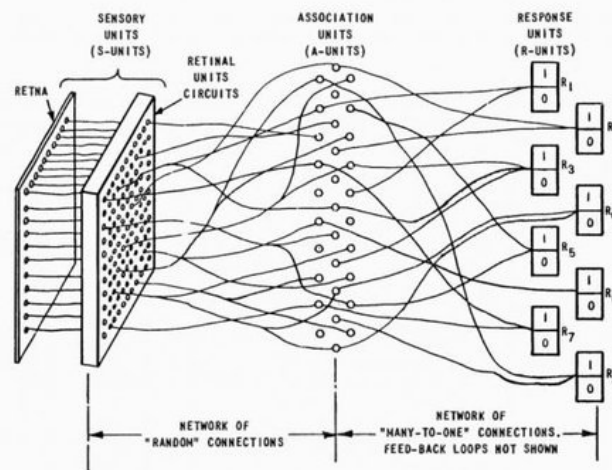
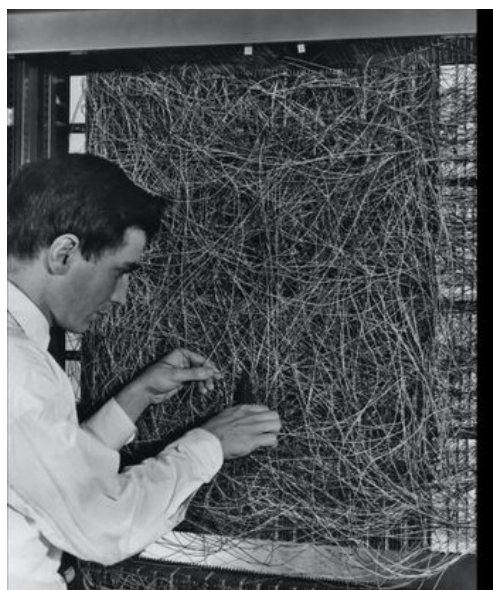


Figure 1 ORGANIZATION OF THE MARK I PERCEPTRON

1.3.2 繁荣期（1980s-2010s）

1986年：“深度学习”（Deep Learning）一词由Rina Dechter引入机器学习社区。

1989年：Yann LeCun提出用反向传导进行更新的卷积神经网络LeNet，启发了后续卷积神经网络的研究与发展。

1998年：现代卷积神经网络CNN的基本结构LeNet-5诞生，机器学习方法由早期基于浅层机器学习的模型变为了基于深度

2006年：Geoff Hinton等人表明多层前馈神经网络可以一次有效地预训练一层，提出了深度信念网络（Deep Belief Nets）的学习方法。

2009年：李飞飞教授团队展示了ImageNet数据库，之后大量计算机视觉领域的经典模型在此数据库上进行验证和评测。

1.3.3 突破期（2012-2020）

2012年9月：AlexNet赢得ImageNet竞赛，神经网络开始再次流行。AlexNet首次采用ReLU激活函数，添加Dropout层减小过拟合，使用2 580 3GB GPU进行加速，形成了AI系统以GPU等加速器为主要计算单元的架构。

2013年：自然语言处理模型Word2Vec诞生，首次提出将单词转换为向量的“词向量模型”。

2014年：GAN（对抗式生成网络）诞生，标志着深度学习进入了生成模型研究的新阶段。

2017年：谷歌提出基于自注意力机制的神经网络结构Transformer架构，奠定了大模型预训练算法架构的基础。

2018年：OpenAI和谷歌分别发布了GPT-1与BERT大模型，Transformer架构开始主导NLP领域。

1.3.4 大模型时代（2020-至今）

2020年：OpenAI推出GPT-3，模型参数规模达到1750亿（175B），成为当时最大的语言模型。

2021年8月：李飞飞等100多位学者发表《On the Opportunities and Risk of Foundation Models》报告，将这种基于神经网络和自监督学习技术，在大规模、广泛来源数据集上训练的AI模型称为“大模型”或“基础模型”。

2022年11月：搭载GPT3.5的ChatGPT横空出世，凭借逼真的自然语言交互与多场景内容生成能力引爆互联网。

2023年3月：GPT-4发布，具备多模态理解与多类型内容生成能力。

大模型时代的特点：

- 参数规模爆发：从亿级到百万亿级
- Transformer架构主流：形成了GPT和BERT两条主要技术路线
- 多模态融合：支持文本、图片、语音等多种模态
- 训练成本激增：GPT-4训练一次的成本超过5000万美金

1.4 AI学习基础

1.4.1 前向传播与反向传播

理解神经网络的工作原理是学习AI系统的基础。神经网络的工作过程可以分为前向传播和反向传播两个阶段。

前向传播 (Forward Propagation)

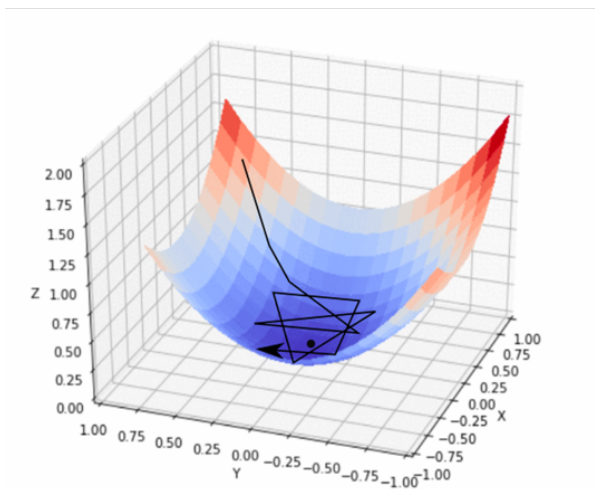
前向传播是指数据从输入层经过隐藏层向输出层流动的过程。输入数据经过每一层的加权求和，再加上激活函数，最终在输出层产生结果。以一个简单例子说明：假设输入是一张手写数字图片，输出是10个类别的概率分布（0-9数字）。数据依次通过：

1. 输入层：接收原始图像像素值
2. 卷积层：提取图像特征
3. 池化层：降低特征维度
4. 全连接层：综合特征进行分类
5. 输出层：输出各类别的概率

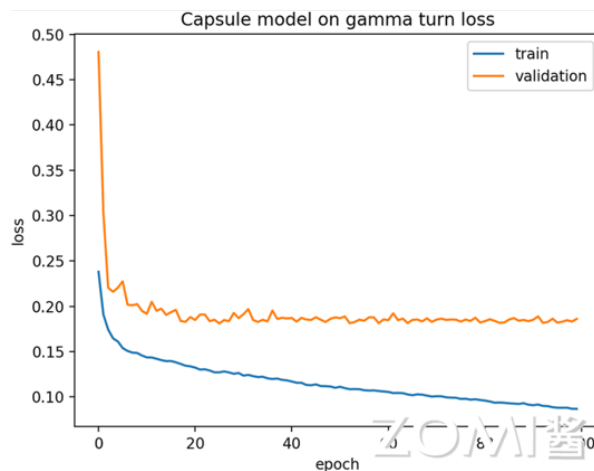
反向传播 (Back Propagation)

反向传播是训练神经网络的核心机制。当前向传播产生输出后，计算预测值与真实标签之间的误差（损失函数Loss），然后将误差反向传播回输入层，用于调整权重和偏差。

梯度下降算法寻找数据鞍点



损失值在下降



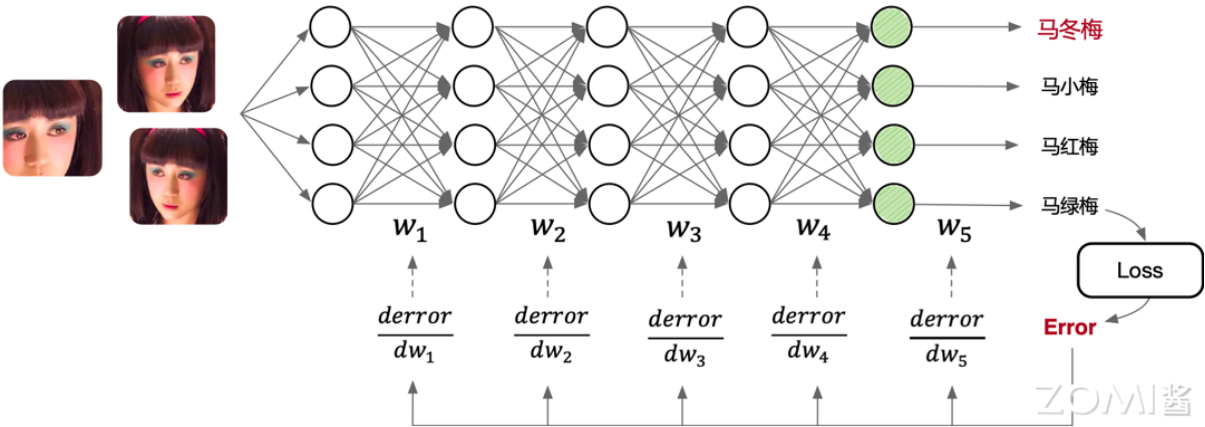
训练过程的本质是通过网络模型中的连接逐层向后传播总误差，计算每个层中每个权重和偏差对总误差的贡献（梯度），并据此更新参数。一句话概括：训练AI模型的过程，就是通过不断的迭代计算，使用梯度下降的优化算法，使得损失值Loss越来越小。损失值越小，模型的性能越好。

1.4.2 训练与推理的区别

训练（Training）和推理（Inference）是AI模型生命周期的两个核心阶段，它们有着显著的区别：

特征	训练	推理
目的	学习数据中的规律，更新模型权重	使用已训练好的模型进行预测
是否需要梯度	是	否
计算资源需求	大	相对较小
延迟要求	相对宽松	通常要求低延迟
精度要求	通常使用FP32	可使用FP16/INT8等低精度
模型状态	可变（weights updating）	固定（frozen）

训练过程需要反向传播计算梯度、更新权重，通常使用32位浮点数（FP32）以保证精度。训练一块GPU可能在数天到数月。推理过程只需要执行前向传播，不需要计算梯度，可以使用量化、剪枝等技术优化，可以使用16位浮点数（FP16）或8位。



1.4.3 主流AI算法

目前AI模型有很多种类，从影响AI系统设计的视角，可以将代表性的AI网络结构归为以下几类：

卷积神经网络（CNN）

以卷积层、池化层、全连接层等算子的组合形成的AI网络模型，在计算机视觉领域取得显著效果和广泛应用。典型应用：

- 图像分类（ResNet、VGG等）
- 目标检测（YOLO、Faster R-CNN等）
- 语义分割（U-Net、Mask R-CNN等）

循环神经网络（RNN）

以循环神经网络、长短时记忆（LSTM）等基本单元组合形成的适合时序数据预测的模型结构。典型应用：

- 自然语言处理
- 语音识别
- 时间序列预测

生成对抗网络（GAN）

训练两个神经网络相互竞争，从给定的训练数据集生成更真实的新数据。例如，从现有图像数据库生成新图像，或从歌曲生成新歌曲。

Transformer架构

通过自注意力机制（Self-Attention）实现序列到序列的转换，成为大语言模型（LLM）的基础架构。

典型代表：

- BERT：基于Transformer Encoder的双向编码器
- GPT：基于Transformer Decoder的自回归模型
- T5：基于Transformer Encoder-Decoder的文本到文本转换模型

图神经网络（GNN）

使用神经网络来学习图结构数据，提取和发掘图结构数据中的特征和模式，满足聚类、分类、预测、分割、生成等图学习

混合结构网络

组合以上不同结构类型的网络模型，如结合卷积神经网络和循环神经网络，解决如OCR等复杂应用场景的预测任务。

1.5 AI发展驱动力

催生AI热潮的原因有三个重要因素：大数据的积累、超大规模的计算能力支撑、机器学习尤其是深度学习算法取得了突破

1.5.1 大规模数据驱动

随着数字化发展，信息系统和平台不断沉淀了大量数据。AI算法利用数据驱动的方式解决问题，从数据中不断学习出规律。互联网公司因为有海量的用户，这些用户不断使用互联网服务，上传文字、图片、音频等数据，积累了丰富的数据。这些以图像分类问题为例：

- MNIST手写数字识别数据集：6万样本，10个分类
- ImageNet：1600万样本，1000个分类
- 互联网Web服务中：数亿量级的图像数据

海量的数据让人工智能问题变得愈发挑战的同时，也实质性地促进了神经网络模型效果的提升。

数据对AI系统的影响：

1. 推动AI算法不断产生更高准确度与更低的误差
2. 让AI有更广泛的应用，产生商业价值
3. 传统的机器学习库不能满足相应需求，产生分布式训练和AI集群的需求
4. 多样的数据格式和任务，驱动AI框架需要有更灵活的表达能力
5. 数据安全与模型安全问题挑战日益突出

1.5.2 AI算法的进步

算法研究员和工程师不断设计新的AI算法和AI模型提升预测效果。新的算法和模型结构需要AI框架提供便于对AI范式的封装。以MNIST手写数字识别任务为例：

- 1998年：简单的CNN可以接近SVM最好效果
- 2012年：CNN可以将错误率降低到0.23%，与人所达到的错误率0.2%非常接近

新的模型不断在以下方面演化进而提升效果：

1. 更好的激活函数和层（如ReLU、Batch Norm等）
2. 更深更大的网络结构和更多的模型权重
3. 更好的训练技巧：正则化、初始化、学习方法

1.5.3 算力与体系结构进步

从1960年以来，计算机性能的增长主要来自摩尔定律。单纯靠工艺尺寸的进步，无法满足各种应用对性能的要求。

于是，人们开始为应用定制专用芯片，通过消除通用处理器中冗余的功能部分，来进一步提高对特定应用的计算性能：

GPU（图形处理器）：对图像类算法进行专用硬件加速，后来出现GPGPU（通用GPU），对适合于抽象为SIMD或SIMT的

TPU（张量处理单元）：谷歌设计的专用AI加速器，通过对神经网络模型中的算子进行抽象，转换为矩阵乘法或非线性变换（Array）进一步提高计算密度。

NPU（神经网络处理器）：华为针对神经网络运算专门优化设计，可解决传统芯片在神经网络运算时效率低下的问题。在Cube设计，每时钟周期可进行4096次MAC运算。

算力的提升让AI取得了突破，但我们也看到，算力还是可能在短期内成为瓶颈。因此，除了单独芯片的不断迭代进行性能提升（Up），系统工程师不断设计更好的分布式计算系统将计算并行，提出了AI集群来达到向外扩展（Scale Out），同时发掘深度学习的作业特点（如稀疏性等）通过算法、系统、硬件协同设计，进一步提升计算效率和性能。

1.6 大模型时代的AI系统

1.6.1 大模型发展现状

从参数规模上看，AI大模型先后经历了预训练模型、大规模预训练模型、超大规模预训练模型三个阶段，每年网络模型自

从技术架构上看，Transformer架构是当前大模型领域主流的算法架构基础，由此形成了GPT和BERT两条主要的技术路线。

从支持模态上看，AI大模型可分为：

- 大语言模型（LLM）：如GPT、LLAMA等
- 视觉大模型（LVM）：如CLIP、DALL·E等
- 多模态大模型（LMM）：如GPT-4等
- 图网络大模型（GLM）
- 科学计算大模型（LSM）

1.6.2 AI系统对大模型的影响

大模型进入爆发期后，在LLM进化树中，BERT（Encoder-Only）、T5（Encoder-Decoder）、GPT（Decoder-Only）分别代表了不同的架构方向。

那为什么在大模型时代，曾经风光无限的BERT家族和T5家族会逐渐没落？

从纯算法模型结构上，T5的模型结构不是线性的，因为在Decoder和Encoder之间有复杂的连接关系，导致T5在真正大规模

针对直接基于Decoder-Only实现的GPT模型，在工程领域实现分布式并行优化，会比T5、BERT等网络模型更加容易实现。在up的大模型时代，工程上更容易实现分布式并行、更容易扩展、训练Token效率更高的模型，一定是更具备优势的。

这就是AI系统反过来影响算法发展的一个典型例子。

1.6.3 大模型时代的系统挑战

大模型训练的成本极高，GPT-4训练一次的成本超过5000万美金。百亿级别和千亿级别的MoE（Mixture of Experts）架构开始成为大模型时代考虑的下一个主流方向。

在大模型时代，系统研究员需要考虑以下问题：

- 网络模型结构算法如何实现各种并行时所需的通信量更低？
- 网络模型结构算法如何优化实现更少计算量、更少的显存需求、更容易进行推理？
- 如何设计编译器使得网络模型计算自动优化、实现通信量、计算量、显存占用更少？
- 如何设计编译器使得网络模型结构算法能够适配到不同的AI加速器上执行计算？
- 如何设计AI计算集群的网络拓扑，减少网络拥塞，降低网络传输耗时，提升线性度？

本节小结

1. AI系统的定义：AI系统是连接硬件和上层应用的中间层软硬件基础设施，包括AI框架、AI编译器、底层算子库等组件。
2. AI系统的重要性：AI系统将复杂的底层硬件抽象隐藏起来，让算法工程师能够专注于算法本身的设计和创新，大幅降低开发门槛。
3. 概念关系：机器学习是实现AI的方法，深度学习是实现机器学习的技术，神经网络是深度学习的具体实现形态。
4. AI发展历程：经历了萌芽期（1950s-1980s）、繁荣期（1980s-2010s）、突破期（2012-2020）和大模型时代（2020-至今）。
5. AI学习基础：包括前向传播、反向传播、训练与推理的区别，以及CNN、RNN、GAN、Transformer等主流算法。
6. 发展驱动力：大数据积累、超大规模计算能力支撑、深度学习算法取得突破性进展。
7. 大模型时代挑战：稀疏化、分布式并行、硬件协同设计成为新的研究方向。

思考与练习

1. 请解释AI系统在整个AI技术栈中的定位和作用，为什么需要AI系统？
2. 论述人工智能、机器学习、深度学习与神经网络之间的关系。
3. 描述AI训练的基本流程，包括前向传播、反向传播和梯度更新的过程。
4. 训练与推理有什么区别？为什么推理可以使用低精度计算？
5. 哪些因素驱动了AI从2012年开始的快速发展？
6. 在大模型时代，AI系统如何反过来影响算法的发展方向？请结合具体例子说明。
7. 思考：如果你要设计一个AI系统，需要考虑哪些核心问题？

AI系统全栈架构

本节学习目标

1. 理解AI系统全栈架构的层次划分
2. 掌握各层的主要职责和交互方式
3. 了解AI训练与推理框架的作用
4. 认识AI编译器与计算架构的功能
5. 熟悉AI硬件与体系结构的基本概念
6. 了解AI生态的组成

前置知识

- 理解1.1节AI系统基本概念
- 具备基本的计算机系统知识
- 了解操作系统和编译器的基本原理

2.1 AI系统全栈架构概述

2.1.1 全栈架构的层次划分

AI系统全栈架构是一个复杂的软硬件协同系统，从上到下可以分为四个主要层次：

层次	主要组件	核心职责
应用层	AI应用、模型服务	业务逻辑、接口封装
AI框架层	PyTorch、MindSpore等	编程接口、自动微分、计算图构建
AI编译器层	TVM、XLA、Glow等	代码生成、优化调度
硬件层	GPU、NPU、TPU等	计算执行、加速运算

应用算法 (Algorithm)	领域算法	视觉、自然语言处理、语音语义：CNN、RNN、LSTM、GAN、Transformer、Stable Diffusion、Reinforce Learning			
开发编程 (Development)	集成开发环境	编程环境：VS Code, Jupyter Notebook, PyCharm			
	编程语言	与主流语言集成：PyTorch and Tensorflow etc. inside Python			
框架 (Frameworks)	AI框架、推理引擎	中间表达 (IR)、前端 API	编译优化 (Compilation)	图优化与图执行调度	推理优化
		基本数据结构：张量 (Tensor)	词法、语法、语义分析	计算图动静统一	常量折叠&冗余节点消除等
		基本计算单元：有向无环图 (DAG)	自动微分	计算图控制流	模型压缩：蒸馏剪枝与量化
异构与编译 (Compiler)	编程编译	前端优化	数据格式布局转换	内存分配	公共表达式消除
		后端优化	代码优化、代码生成	算子循环优化	令和内存优化
		编程模型：CUDA/Ascend C/Band C		AI编译器：TVM、XLA、TC	
		LLVM/GCC			
	底层通信	HCCL/NCCL、通信原语、集合通信算法			
体系结构 (Architecture)	底层硬件	AI芯片：CPU/GPU/AISC/FPGA/TPU/NPU			

2.1.2 各层的职责与交互

从上到下看，每一层都有其独特的职责：

应用层位于最顶层，直接面向最终用户和业务场景。这一层包括各类AI应用（如图像识别应用、自然语言处理应用）、和

AI框架层承上启下，是整个AI系统的核心中间层。这一层负责：

- 提供高级编程语言（如Python）的API供开发者编写神经网络模型
- 实现自动微分（Automatic Differentiation）功能，支持反向传播训练
- 构建和优化计算图（静态图或动态图）
- 管理模型的训练和推理流程
- 提供数据加载、模型管理等工具链

主流的AI框架包括PyTorch、MindSpore、TensorFlow、JAX等。

AI编译器层负责将计算图转换为底层硬件可执行的代码。这一层：

- 解析神经网络模型的高级表达
- 进行图优化、算子融合等编译优化
- 生成针对特定硬件的优化代码
- 管理运行时调度和资源分配

典型的AI编译器包括TVM（由Apache TVM开源）、XLA（谷歌的加速线性代数编译器）、Glow（Facebook的编译器）等

硬件层是整个栈的底层基础，提供实际的计算能力。这一层包括：

- AI专用芯片（GPU、NPU、TPU等）

- 通用CPU
- 内存和存储系统
- 网络互联设备（RDMA、NVLink等）

各层之间通过精心设计的接口进行交互。上层调用下层提供的服务，下层为上层提供抽象后的能力支撑。这种分层架构使

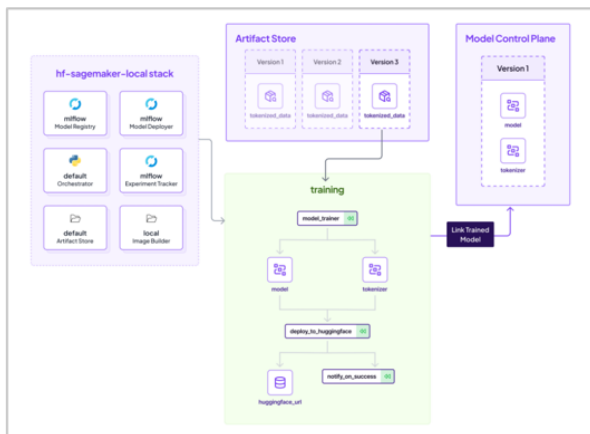
2.2 AI训练与推理框架

2.2.1 AI框架的核心作用

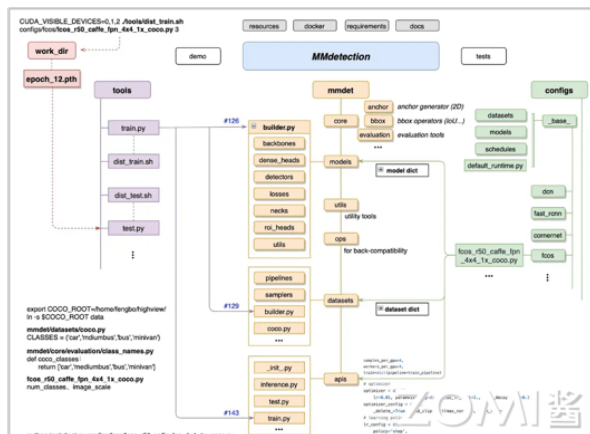
AI框架不仅仅是指如PyTorch等训练框架，还包括推理框架。其核心职责是：

1. 提供编程接口：通过Python API供用户编写网络模型计算图结构
2. 自动微分：高效地对网络模型自动求导等。由于网络模型中大部分算子较为通用，AI框架提前封装好算子的自动求导
3. 计算图构建：静态计算图、动态计算图构建等
4. 中间表达构建：通过构建网络模型的中间表达及多层中间表达，让模型本身可以更好地被下层AI编译器编译生成高

Huggingface 组件架构



MMDetection 套件架构



2.2.2 AI框架的核心功能模块

AI框架作为一个复杂的软件系统，包含多个核心功能模块：

网络模型构建模块

网络模型构建模块负责提供卷积神经网络CNN、循环神经网络RNN、Transformer结构等基本结构和算子支持的API。语言

模型算法实现模块

模型算法实现模块封装了算法层面的配置和API供用户选择。算法一般被封装为AI框架的配置或API供用户选择，有些AI

计算图构建模块

不同的AI框架类型决定了其使用静态还是动态图进行构建：

- 静态计算图：在执行前完整地定义计算图，有利于获取更多信息做全图优化，但调试相对困难
- 动态计算图：在实际执行时才动态构建计算图，有利于调试，但优化机会有限

目前实际处于一个融合的状态，如PyTorch 2.X版本后推出Dynamo特性支持原生静态图。

自动求导模块

自动求导（Automatic Differentiation, AutoGrad）是深度学习框架的核心功能之一。由于网络模型中大部分算子较为通用

中间表达构建模块

多层次中间表达（Intermediate Representation, IR）是连接AI框架和AI编译器的桥梁。通过构建网络模型的中间表达及多

工具链模块

工具链模块提供了如模型在不同硬件的迁移、在不同框架的迁移、模型转换、调试、可视化、类型系统等功能。就像传统

生命周期管理模块

生命周期管理模块负责数据读取、训练与推理等流程开发与管理。机器学习领域的DevOps也就是MLOps的基础工具支持

2.2.3 主流AI框架介绍

PyTorch

PyTorch由Meta（原Facebook）开发，是目前最流行的深度学习框架之一。PyTorch采用命令式编程风格，动态图执行，调2.0引入了torch.compile特性，支持将动态图转换为静态图进行优化。

TensorFlow

TensorFlow由谷歌于2015年开源，是工业界应用最广泛的深度学习框架之一。TensorFlow提供静态图（Graph mode）和动态图（Eager mode）两种执行模式。TensorFlow 2.0默认启用动态图执行，大幅提升了开发体验。TensorFlow自

MindSpore

MindSpore由华为于2020年开源，是国产AI框架的代表。MindSpore支持端云统一架构，可以在终端和云端无缝切换。Min

JAX

JAX由谷歌于2018年开源，定位为高性能数值计算框架。JAX的核心设计理念是函数转换，通过grad、jit等函数转换实现自

2.2.4 推理框架的特殊需求

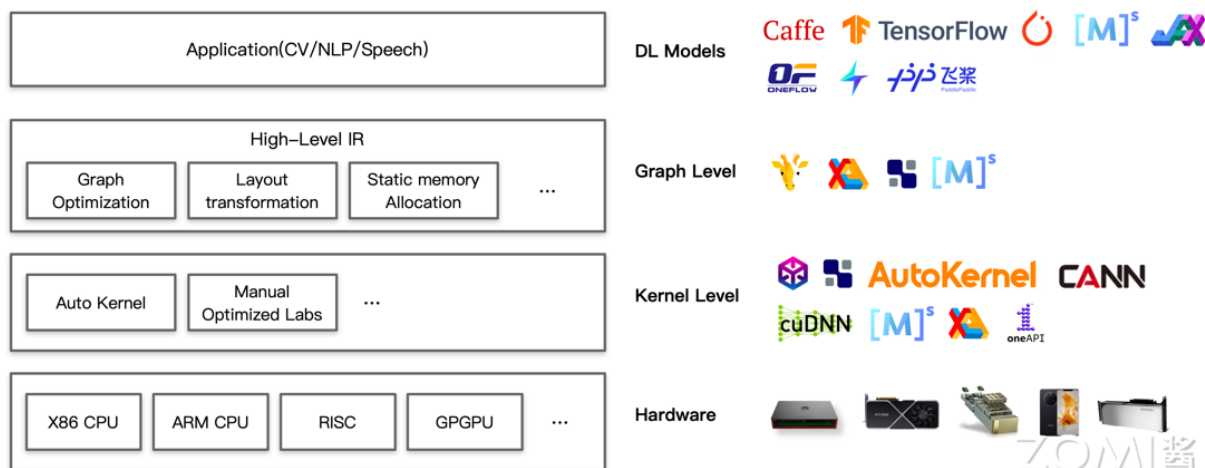
推理框架与训练框架既有联系又有区别。推理框架需要关注：

1. 低延迟：推理通常要求较低的响应延迟，需要针对延迟进行优化
2. 资源高效利用：推理部署环境通常资源有限，需要高效利用计算资源
3. 低精度计算：推理可以使用FP16、INT8等低精度格式来提升性能
4. 模型压缩：量化、剪枝、知识蒸馏等模型压缩技术在推理中广泛应用
5. 跨平台部署：模型可能需要部署到多种硬件平台，需要跨平台支持

2.3 AI编译与计算架构

2.3.1 AI编译器的定位

AI框架充分赋能深度学习领域，为AI算法的开发者提供了极大便利。但随着AI技术应用的全面发展，各厂家根据自身业务为了提供不同框架和硬件体系结构之间的迁移性，ONNX等中间IR被提出，其定义了表示神经网络模型的统一格式，以便



2.3.2 为什么需要AI编译器

在通用硬件上，高度优化的线性代数库为神经网络模型计算提供了基础加速库。此外，大多数硬件供应商还发布了专属的DNN和cuDNN等。但基于基础加速库的优化往往落后于深度学习算法模型的更新，且大多数情况下需要针对不同的平台。为了解决多硬件平台上的性能优化的问题，多种AI编译器被提出并得到了普及和应用。

2.3.3 AI编译器的核心特征

与传统编译器相比，AI编译器是一个领域特定的编译器，有四个明显的特征：

主前端语言

与传统编译器不同，AI编译器通常不需要Lexer/Parser，而是基于前端高级编程语言（如Python）的AST将神经网络模型编译成中间表示。

多层IR设计

多层IR设计，为的是满足易用性与高性能这两种类型需求：

1. 为了让开发者使用方便，AI框架会尽量对张量的计算进行抽象封装成具体的API或者函数，算法开发者只要关注神经网络模型的计算图即可。
2. 在底层算子性能优化时，可以打破算子的边界，从更细粒度的循环调度等维度，结合不同的硬件特点完成优化。

面向神经网络优化

面向神经网络模型特殊的数据类型进行定义。AI领域，网络模型层的具体计算被抽象成张量的计算，这就意味着AI编译器需要对张量进行特殊的数据类型定义。

DSA芯片架构支持

AI训练和推理对性能和时延都非常敏感，所以大量使用专用的AI加速芯片进行计算，而AI编译器其实是以DSA架构的AI加速芯片为目标的。

2.3.4 主流AI编译器介绍

TVM

TVM（Tensor Virtual Machine）由Apache开源，是一个端到端的深度学习编译器。TVM提供了从高层模型描述到硬件指令的编译。

XLA

XLA（Accelerated Linear Algebra）是谷歌为TensorFlow开发的加速线性代数编译器。XLA将TensorFlow计算图编译成优化的线性代数计算图。

Glow

Glow由Meta（原Facebook）开发，是一个基于LLVM的深度学习编译器。Glow采用低级中间表示（LLVM IR）来进行图优化和代码生成，可以支持多种硬件后端。

Jittor

Jittor是清华大学开发的深度学习框架，采用了元算子统一计算图表达的技术。Jittor的编译器可以自动将计算图优化并生成

2.3.5 编译优化的核心内容

AI编译器的主要优化内容包括：

图优化

图优化是在计算图层面进行的优化，包括：

- 算子融合：将多个相邻的算子合并为一个算子，减少内存访问和内核启动开销
- 常量折叠：将可以在编译时计算的张量提前计算出来
- 公共子表达式消除：消除重复的计算

算子内循环优化

在算子内部层面，编译器可以进行循环优化：

- 循环展开：减少循环控制开销
- 循环分块：优化数据局部性，提高缓存命中率
- 循环重排：优化内存访问模式

自动并行化

AI编译器可以根据硬件拓扑结构自动进行并行化优化，包括：

- 算子间并行：多个独立的算子可以并行执行
- 算子内并行：利用SIMD或SIMT指令进行数据级并行
- 分布式并行：支持多卡、多节点的分布式计算

量化与低精度优化

AI编译器支持将FP32模型量化到FP16、INT8等低精度格式：

- 量化感知训练（Quantization Aware Training）
- 训练后量化（Post-Training Quantization）
- 混合精度计算

2.4 AI硬件与体系结构

2.4.1 AI硬件概述

AI硬件是整个系统栈的底层基础，负责程序的真正执行、互联与加速。AI硬件的发展经历了从通用CPU到专用AI芯片的演

通用CPU

通用CPU是最基础的计算单元，具有高度的通用性，可以执行任何计算任务。但在AI场景下，CPU的并行计算能力有限，

GPU

GPU（图形处理单元）最初为图形渲染设计，但其高度并行的架构非常适合矩阵运算，在深度学习领域得到广泛应用。

英伟达（NVIDIA）是GPU领域的领导者，其推出的Tesla系列、Quadro系列等专业GPU，以及消费级的GeForce系列，都（Tesla Core），专门针对深度学习中的矩阵运算进行优化。

TPU

TPU（张量处理单元）是谷歌专门为机器学习设计的专用芯片。TPU使用脉动阵列（Systolic Array）架构，专门优化矩阵乘法和向量运算。谷歌的TPU已经经历了多代迭代，从TPU v1用于推理，到TPU v2/v3同时支持训练和推理。

NPU

NPU（神经网络处理器）是华为针对神经网络运算专门优化设计的专用芯片。华为昇腾达芬奇架构面向AI计算设计，通过Cube设计，每时钟周期可进行4096次MAC运算，为AI提供强大算力支持。

其他AI加速器

除了上述主流AI加速器，还有多种专用AI芯片：

- 英特尔（Intel）的Nervana和Myriad系列
- 亚马逊（Amazon）的AWS Inferentia
- 特斯拉（Tesla）的Dojo芯片
- 苹果（Apple）的Neural Engine

2.4.2 硬件互联技术

在AI训练中，单个AI加速器的算力往往不够，需要多卡甚至多节点协同工作。硬件互联技术为这种协同提供了高带宽的通信

NVLink

NVLink是英伟达开发的高速互联技术，可以在GPU之间提供高带宽的点对点通信。NVLink 4.0提供900GB/s的双向带宽，显著高于传统的PCIe连接。

NVSwitch

NVSwitch是英伟达的交换芯片，可以连接多个GPU形成高速互连网络。一个基于NVSwitch的服务器最多可以连接8个GPU

Infinity Fabric

Infinity Fabric是AMD的GPU互联技术，用于连接同一服务器内的多个GPU。

RDMA

RDMA（远程直接内存访问）是一种网络技术，允许直接访问远程服务器的内存，无需操作系统介入，可以大幅降低网络

InfiniBand

InfiniBand是一种高性能网络技术，提供高带宽和低延迟的互连，广泛应用于AI超级计算机和数据中心。

2.4.3 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是AI系统中的重要层次，它将底层硬件的复杂性抽象出来，为上层软件提供统一的硬件接口抽象可以：

- 复用编译优化策略
- 让优化方案与具体底层的AI硬件设备和AI体系结构适当解耦
- 支持上层软件在不同硬件平台间移植

AI芯片厂商通常会提供自家的运行时库和驱动，如NVIDIA的CUDA、cuDNN、cuBLAS等，华为的CANN（Compute Architecture for Neural Networks）等。

2.5 AI生态组成

2.5.1 核心系统软硬件

核心系统软硬件构成了AI系统的基础设施层。通过核心系统软硬件，底层的基础架构已经可以给上层提供算力、存储、网络

深度学习任务运行和优化环境

提供更高的运行时性能，资源隔离与调度。当深度学习作业启动，AI框架或运行时提供更好的算子与任务调度、内存管理

通用资源管理和调度系统

提供更公平、高效率和稳定的平台支持。性能并不是系统设计本身的唯一考虑因素，在多租环境，还要兼顾公平、效率和资源利用率。Kubernetes等容器编排系统被广泛应用于AI集群的资源管理和调度。

新型硬件及相关高性能网络 and 计算栈

随着加速器技术不断发展，网络互连技术提供更高的带宽，硬件层提供更高的算力与带宽支持模型训练与推理。系统需要

2.5.2 AI算法和框架

通过深度学习算法与框架，用户可以表达模型设计和训练配置等需求，就像提供了一套特定领域的“编程语言”，并且提供

广泛用途的高效新型通用AI算法

提供更多样的模型支持，推进和支持模型效果的提升。支持新的算子（如控制流等）、更加灵活的模型结构（如图模型等）

多种AI框架的支持与进化

由于多种框架与工具的存在，如何为用户提供更多样的框架的统一支持与优化对提升用户体验、复用已有代码有很强的实

神经网络编译架构及优化

在编译期，通过静态分析与优化的方法，提供更优化的编译支持，提升模型的性能、正确性等。类似传统编译器，网络框

2.5.3 更广泛的AI生态

随着深度学习高速发展，更大的搜索空间、运行时才能获取的数据、模型安全与隐私、部署推理的多样化需求变得日益迫

机器学习新模式

强化学习等新训练范式需要通过与环境交互的过程中获取训练数据，需要设计新的系统以支持灵活的训练范式执行、部署

自动机器学习 (AutoML)

当用户想试错的搜索空间达到一定量级，通过自动化机器学习工具与算法可以更高效地进行模型的探索与训练。自动化机

安全与隐私

数据与模型的安全与隐私保护变得越来越重要。神经网络模型类似传统程序，功能上接受输入、处理后产生输出，但相比

模型推理、压缩与优化

如果不需要训练，只需要执行前向传播过程，则是用户开始使用模型进行推理。基于深度学习特有性质进行高效的模型部署。同时网络模型本身可以通过模型压缩、量化等手段精简计算量与内存消耗，加速模型的部署。

2.6 AI系统的设计目标

深度学习系统的设计目标可以总结为以下几个部分：

2.6.1 高效编程语言、开发框架和工具链

设计更具表达能力和简洁的神经网络计算原语

设计更具表达能力和简洁的神经网络计算原语以及高级编程语言。让用户能够提升AI应用程序的开发效率，屏蔽底层硬件。当前神经网络模型除了特定领域模型的算子和流程可以复用（如大语言模型Transformer架构在自然语言处理NLP领域被广

更直观的编辑、调试和实验工具

让用户可以完整地进行神经网络模型的开发、测试、调整诊断与修复和优化程序，提升所开发AI应用程序的性能与鲁棒性。训练过程不是一蹴而就，其中伴随着损失函数Loss曲线不收敛、Loss值出现NaN无效值、内存溢出等算法问题与算法设计

支持AI生命周期中的各个环节

数据处理、模型开发与训练、模型压缩与推理、安全和隐私保护等。不仅能构建AI模型，能够支持全生命周期的AI程序开

2.6.2 AI任务系统级支持

支持强化学习、自动化机器学习等新的训练范式

除了对深度学习训练与推理的支持，还能支持强化学习、自动化机器学习等新的训练范式。例如，需要不断和环境或模型

提供更强大和可扩展的计算能力

让用户的AI程序可扩展并部署于可以并行计算的节点或者集群，应对大数据和大模型的挑战。

因为当前AI模型不断通过大模型、多模态大模型以产生更好的算法效果，促使AI系统需要支持更大的模型、更多模态的输

自动编译优化算法

对计算图自动推导：尽可能的通过符号执行或即时编译JIT技术，获取更多的计算图信息，让AI框架或者AI编译器自动执。根据不同体系结构自动并行化：面对部署场景的多样化体系结构、训练阶段异构硬件的趋势，AI框架让用户透明地进行任

云原生自动分布式化

自动分布式并行扩展到多个计算节点，面对云与集群场景，自动将AI任务扩展与部署，进而支撑分布式计算、弹性计算，

2.6.3 探索并解决新挑战下系统设计、实现和演化

在AI系统中会随着AI算法的发展，出现对动态图、动态Shape的支持需求，利用网络模型结构的稀疏性进行压缩加速优化提供在更大规模的企业级环境的部署需求。如云环境、多租环境的训练部署需求：面对多组织、多研究员和工程师共享集跨平台的推理部署需求。面对割裂的边缘侧硬件与软件栈，如何让模型训练一次，跨平台部署到不同软硬件平台，也是推最后是安全与隐私的需求。由于网络模型类似传统程序的功能，接受输入，处理后产生输出，但是相比传统程序，其解释

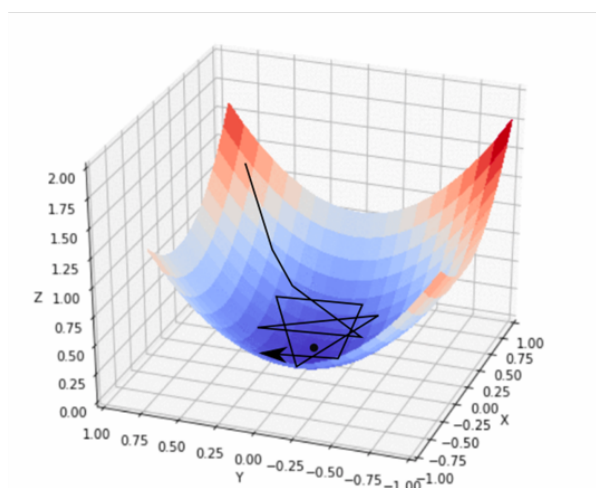
2.7 AI训练与推理流程

2.7.1 AI训练全流程

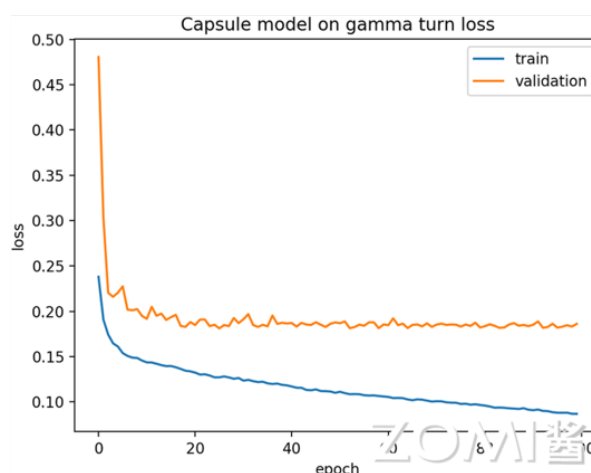
AI训练是将模型从随机初始化的状态，通过学习数据中的规律，逐渐优化到能够较好完成目标任务的过程。一个完整的AI

1. 数据准备：收集、清洗、标注训练数据，设计合适的数据增强策略
2. 模型设计：选择合适的网络结构，设计损失函数
3. 训练循环：重复执行前向传播、损失计算、反向传播、权重更新
4. 评估与调优：使用验证集评估模型性能，调整超参数
5. 模型保存：将训练好的模型保存为特定格式

梯度下降算法寻找数据鞍点



损失值在下降



2.7.2 AI推理全流程

AI推理是使用训练好的模型对新数据进行预测的过程。相比训练，推理流程更加简单直接：

1. 模型加载：加载训练好的模型权重
2. 输入处理：对待预测的数据进行预处理
3. 前向传播：执行推理计算
4. 输出后处理：将模型输出转换为可读的结果

推理相比训练有以下几个显著区别：

- 不需要计算梯度，资源消耗更低
- 通常有更严格的延迟要求
- 可以使用模型压缩技术（如量化、剪枝）进行优化
- 常需要支持跨平台部署

2.7.3 分布式训练

当单个AI加速器的算力和显存无法满足训练需求时，需要使用分布式训练。分布式训练主要有两种策略：

数据并行 (Data Parallelism)

将训练数据分成多个批次 (batch)，每个计算设备持有完整的模型副本，分别处理不同的数据批次，然后通过通信聚合。优点是实现简单，每个设备需要存储完整的模型；缺点是通信压力大，批大小受单设备显存限制。

模型并行 (Model Parallelism)

将模型的不同部分分配到不同的计算设备上。适用于模型过大，单个设备无法完整存储模型参数的情况。模型并行可以分为：

- 流水线并行 (Pipeline Parallelism)：将模型的不同层分配到不同设备
- 张量并行 (Tensor Parallelism)：将同一层的参数分割到不同设备
- 专家并行 (Expert Parallelism)：MoE架构中不同专家分配到不同设备

混合并行

实际训练大模型时，通常会组合多种并行策略，形成3D并行或更复杂的并行策略。

本节小结

1. AI系统全栈架构分为四个层次：应用层、AI框架层、AI编译器层、硬件层，各层有明确的职责分工和交互接口。
2. AI框架层是核心中间层：提供高级编程API、自动微分、计算图构建、生命周期管理等核心功能，是AI系统中最接近应用层的中间件。
3. AI编译器负责代码生成与优化：将计算图转换为硬件可执行代码，进行图优化、算子融合、自动并行化等优化工作。
4. AI硬件提供计算基础：包括GPU、NPU、TPU等专用AI加速器，以及高带宽互联技术如NVLink、RDMA等。
5. AI生态包含核心系统和更广泛的应用：核心系统软硬件、AI算法框架、更广泛的机器学习新模式和AutoML等共同构成生态。
6. AI系统的设计目标：高效编程语言与工具链、对多种AI任务的系统级支持、解决新挑战下的系统设计与演化。
7. 分布式训练是处理大模型的主要手段：包括数据并行、模型并行、流水线并行等多种策略，实际中常采用混合并行。

思考与练习

1. 请描述AI系统全栈架构的层次划分，并说明每一层的核心职责。
2. AI框架在AI系统中扮演什么角色？请列举AI框架的核心功能模块。
3. AI编译器相比传统编译器有哪些特征？为什么AI领域需要专门的编译器？
4. 主流的AI硬件有哪些？它们各自有什么特点？硬件互联技术的作用是什么？
5. 请解释数据并行和模型并行的区别，以及各自的优缺点。
6. AI系统的设计目标有哪些？这些目标分别针对哪些具体的挑战？
7. 在大模型时代，AI系统需要支持哪些新的需求和挑战？
8. 思考：如果你要设计一个支持大模型训练的分布式系统，需要考虑哪些核心问题？

本章小结

AI系统基本概念

AI系统（AI System）是连接硬件和上层应用的中间层软硬件基础设施，是人工智能时代承担类似传统软件栈中数据库、操作系统等核心要点：

- AI系统将复杂的底层硬件抽象隐藏起来，让算法工程师能够专注于算法本身的设计和创新
- 机器学习是实现AI的方法，深度学习是实现机器学习的技术，神经网络是深度学习的具体实现形态
- AI发展经历了萌芽期（1950s-1980s）、繁荣期（1980s-2010s）、突破期（2012-2020）和大模型时代（2020-至今）

AI学习基础

- 前向传播：数据从输入层经过隐藏层向输出层流动，产生预测结果
- 反向传播：计算预测值与真实标签之间的误差，将误差反向传播并更新权重
- 训练：通过迭代计算使损失值Loss越来越小
- 推理：使用训练好的模型进行预测，只需执行前向传播

主流AI算法

- CNN（卷积神经网络）：计算机视觉领域的基础模型结构
- RNN（循环神经网络）：适合时序数据和自然语言处理
- GAN（生成对抗网络）：生成模型，用于生成新数据
- Transformer：大语言模型的基础架构

AI发展驱动力

1. 大数据积累：数据驱动方式推动AI算法效果提升
2. 算力提升：从CPU到GPU再到专用AI芯片，硬件算力大幅提升
3. 算法突破：从AlexNet到Transformer，模型结构不断创新

AI系统全栈架构

AI系统全栈架构从上到下分为四个层次：应用层 → AI框架层 → AI编译器层 → 硬件层

各层职责

层次	主要组件	核心职责
应用层	AI应用、模型服务	业务逻辑、接口封装
AI框架层	PyTorch、MindSpore等	编程接口、自动微分、计算图构建
AI编译器层	TVM、XLA、Glow等	代码生成、优化调度
硬件层	GPU、NPU、TPU等	计算执行、加速运算

AI训练与推理框架

- AI框架提供Python API供开发者编写神经网络模型
- 实现自动微分功能，支持反向传播训练
- 支持静态图和动态图两种计算图构建方式

- 提供数据加载、模型管理等工具链

AI编译器

AI编译器是领域特定编译器，具有四个特征：

- 基于前端高级语言（如Python）的AST
- 多层IR设计
- 面向神经网络优化
- DSA芯片架构支持

AI硬件

- GPU：英伟达主导，广泛应用于AI训练
- TPU：谷歌专用AI加速器
- NPU：华为昇腾系列
- 互联技术：NVLink、RDMA、InfiniBand等

分布式训练

- 数据并行：每个设备持有完整模型，处理不同数据
- 模型并行：将模型分配到不同设备
- 混合并行：组合多种并行策略

思考与练习

第一部分：AI系统基本概念

1. 请解释AI系统在整个AI技术栈中的定位和作用，为什么需要AI系统？
2. 论述人工智能、机器学习、深度学习与神经网络之间的关系。
3. 描述AI训练的基本流程，包括前向传播、反向传播和梯度更新的过程。
4. 训练与推理有什么区别？为什么推理可以使用低精度计算？
5. 哪些因素驱动了AI从2012年开始的快速发展？
6. 在大模型时代，AI系统如何反过来影响算法的发展方向？请结合具体例子说明。
7. 思考：如果你要设计一个AI系统，需要考虑哪些核心问题？

第二部分：AI系统全栈架构

8. 请描述AI系统全栈架构的层次划分，并说明每一层的核心职责。
9. AI框架在AI系统中扮演什么角色？请列举AI框架的核心功能模块。
10. AI编译器相比传统编译器有哪些特征？为什么AI领域需要专门的编译器？
11. 主流的AI硬件有哪些？它们各自有什么特点？硬件互联技术的作用是什么？
12. 请解释数据并行和模型并行的区别，以及各自的优缺点。
13. AI系统的设计目标有哪些？这些目标分别针对哪些具体的挑战？
14. 在大模型时代，AI系统需要支持哪些新的需求和挑战？
15. 思考：如果你要设计一个支持大模型训练的分布式系统，需要考虑哪些核心问题？

1.4.5 Chap05 神经网络基础

本章学习目标

1. 理解神经元模型和工作原理
2. 掌握主流网络结构（CNN/RNN/Transformer）
3. 了解训练和推理的区别
4. 理解模型量化和压缩的基本概念

前置知识

- Chap03 数学基础
- Chap04 AI系统概述

内容导航

神经元与网络模型

本章学习目标

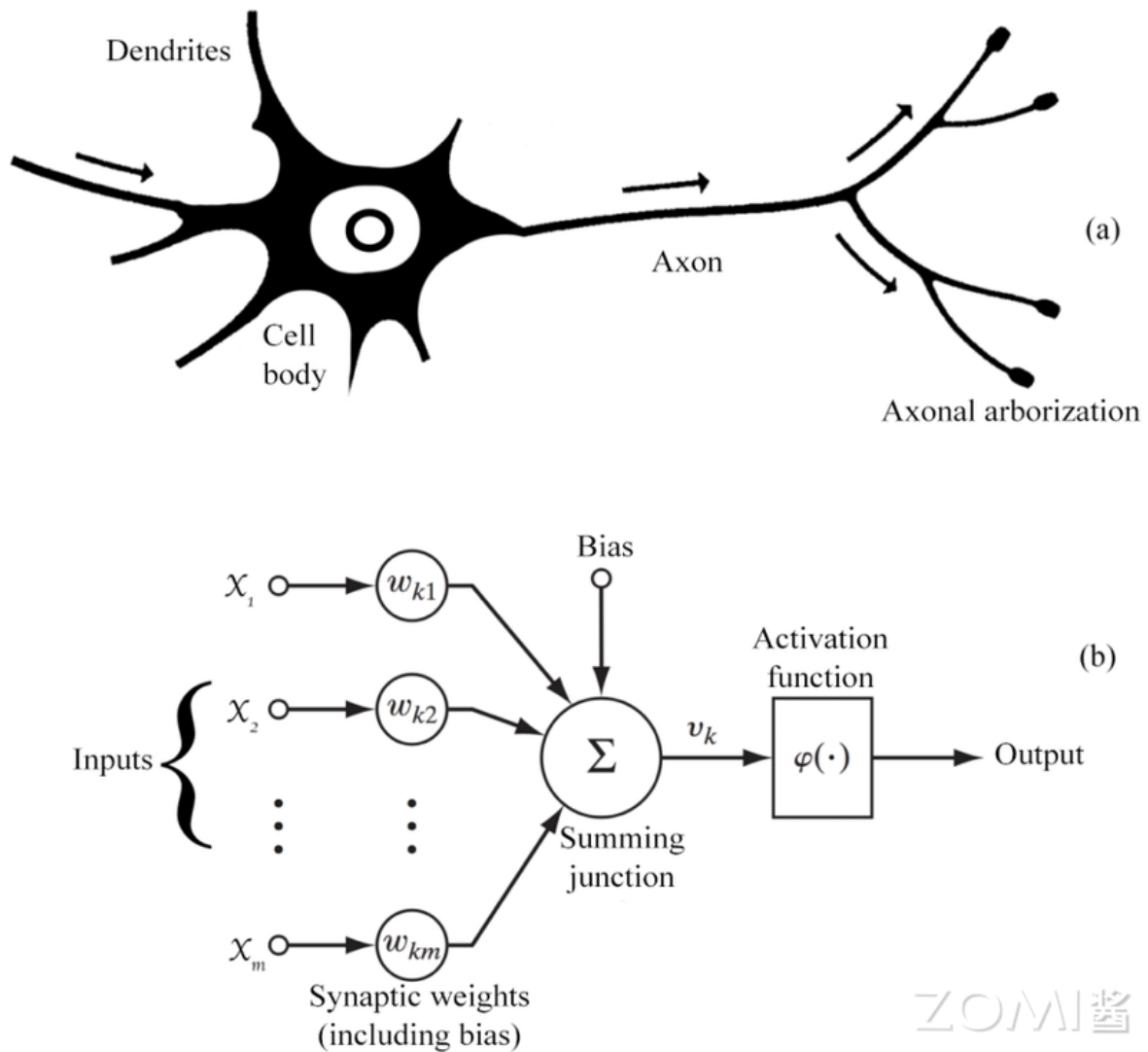
1. 理解神经元模型和工作原理
2. 掌握主流网络结构（CNN/RNN/Transformer）
3. 了解训练和推理的区别
4. 理解模型量化和压缩的基本概念

前置知识

- 数学基础（线性代数、概率论）
 - AI系统概述
-

1.1 神经网络的基本概念

神经网络是人工智能算法基础的计算模型，灵感来源于人类大脑的神经系统结构。它由大量的人工神经元组成，分布在多



对一个神经网络来说，主要包含如下几个知识点，这些是构成一个神经网络模型的基础组件。

1.1.1 神经元 (Neuron)

神经元 (Neuron) 是神经网络的基本组成单元，模拟生物神经元的功能，接收输入信号并产生输出。在神经网络中，每个神经元在生物神经元中，信息通过轴突和树突进行传递，人工神经元也模拟了类似的结构。人工神经元的输入类似于树突，权重

1.1.2 激活函数 (Activation Function)

激活函数 (Activation Function) 用于神经元输出非线性化的函数。线性函数的表达能力有限，只能表示线性关系，而现实世界中的问题往往是非线性的。常见的激活函数包括：

ReLU (Rectified Linear Unit) : $f(x) = \max(0, x)$ ，计算简单高效，是目前最广泛使用的激活函数之一。ReLU函数的优点

Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$ ，将输入压缩到(0,1)区间，常用于二分类问题的输出层。缺点是容易出现梯度消失问题，且输出

Tanh (双曲正切) : $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ，输出范围是(-1,1)，是零中心化的。但同样存在梯度消失问题。

Softmax: 常用于多分类问题的输出层，将输出转换为概率分布: $Softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ 。

1.1.3 模型层数 (Layer)

神经网络由多个层次组成，主要包括三种类型的层：

输入层 (Input Layer)：接收原始输入数据，不进行任何计算，只是将数据传递给下一层。输入层的神经元数量等于输入特征数量。

隐藏层 (Hidden Layer)：介于输入层和输出层之间的网络层，用于提取数据的不同特征。隐藏层可以有多层，网络深度取决于隐藏层的数量。

输出层 (Output Layer)：产生网络的最终输出。输出层的结构取决于具体任务，例如分类任务的输出层可能使用Softmax函数。

1.1.4 前向传播 (Forward Propagation)

前向传播 (Forward Propagation) 是指输入数据通过神经网络从输入层传递到最后输出层的过程，用于生成预测结果。在训练过程中，前向传播与反向传播交替进行。

1. 输入数据进入输入层
2. 对于第 l 层的每个神经元，计算： $z^l = W^l \cdot a^{l-1} + b^l$
3. 应用激活函数： $a^l = \sigma(z^l)$
4. 将 a^l 传递给下一层，重复步骤2-3直到输出层

1.1.5 反向传播 (Backpropagation)

反向传播 (Backpropagation) 是通过计算损失函数对网络参数进行调整的过程，以使网络的输出更接近预期输出。反向传播是神经网络训练的关键步骤。

1. 计算输出层的损失：给定预测值 $\hat{y}$ 和真实值 y ，计算损失 $L(y, \hat{y})$
2. 从输出层开始，利用链式法则逐层计算梯度：

$$\frac{\partial L}{\partial W^l} = \frac{\partial L}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \cdot \dots \cdot \frac{\partial a^l}{\partial z^l} \cdot \frac{\partial z^l}{\partial W^l}$$
3. 使用梯度下降或其他优化算法更新参数： $W^l = W^l - \alpha \frac{\partial L}{\partial W^l}$

1.1.6 损失函数 (Loss Function)

损失函数 (Loss Function) 衡量模型预测结果与实际结果之间差异的函数。神经网络训练的目标是最小化损失函数，使模型预测更准确。

均方误差 (MSE)： $L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ ，常用于回归任务。

交叉熵 (Cross-Entropy)： $L = -\sum_{i=1}^n y_i \log(\hat{y}_i)$ ，常用于分类任务，特别是多分类问题。

二元交叉熵 (Binary Cross-Entropy)： $L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$ ，用于二分类问题。

1.1.7 优化算法 (Optimization Algorithm)

优化算法用于调整神经网络参数以最小化损失函数。不同的优化算法会影响模型训练的收敛速度和能达到的性能。

梯度下降 (Gradient Descent)：最基本的优化算法，沿着梯度的负方向更新参数。缺点是收敛速度慢，且容易陷入局部最优解。

随机梯度下降 (SGD)：每次使用一个样本或少量样本来估计梯度，加快收敛速度，但会带来梯度估计的噪声。

动量 (Momentum)：引入动量概念，加速收敛并减少震荡： $v_t = \gamma v_{t-1} + \alpha \nabla L(\theta)$ ， $\theta = \theta - v_t$

Adam (Adaptive Moment Estimation)：结合动量和自适应学习率： $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta)$ ， $v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta))^2$ ， $\theta = \theta - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon}$

1.2 神经网络的训练与推理

神经网络的生产包含训练和推理两个阶段。训练阶段可以描述为如下几个过程。从下图流程中可以发现，神经网络的训练整体来说，训练阶段的目的是通过最小化损失函数来学习数据的特征和内在关系，优化模型的参数；推理阶段则是利用训练好的模型对新数据进行预测。

1.2.1 训练阶段

神经网络的训练是一个迭代优化的过程，主要包括以下步骤：

1. 数据准备：收集、清洗和预处理训练数据，将数据划分为训练集、验证集和测试集。
2. 前向传播：将训练数据输入网络，计算网络输出。
3. 计算损失：使用损失函数计算预测值与真实值之间的差异。
4. 反向传播：利用链式法则计算损失函数对每个参数的梯度。
5. 参数更新：使用优化算法根据梯度更新网络参数。
6. 迭代优化：重复步骤2-5，直到满足停止条件（如达到预设的迭代次数或验证集性能不再提升）。

训练阶段通常需要GPU或TPU等高性能计算设备，因为需要处理大量的矩阵运算。

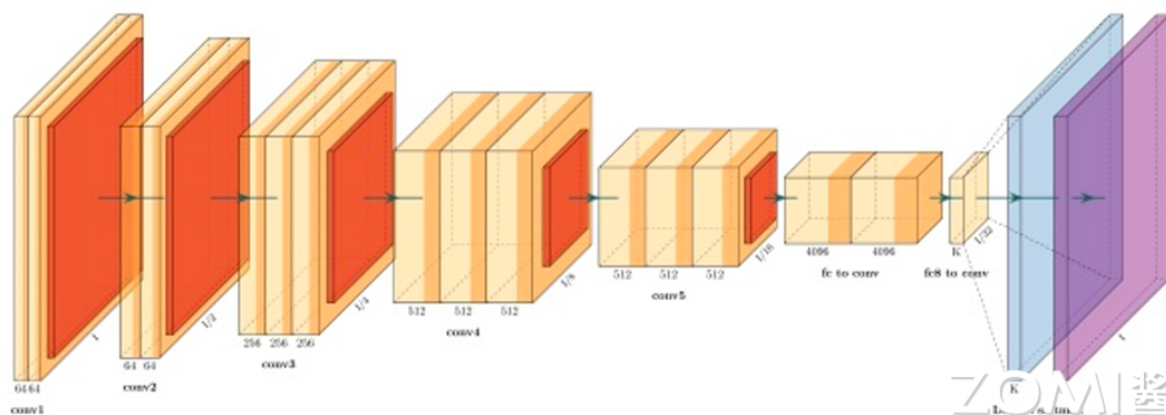
1.2.2 推理阶段

推理阶段是使用训练好的模型对新数据进行预测的过程。相比训练阶段，推理阶段的计算需求较小，主要关注点是推理速度和推理成本。

- 不需要计算梯度，节省了存储梯度的内存
- 模型权重可以是低精度的（如INT8），以提高推理效率
- 可以在各种设备上运行，从云端服务器到边缘设备

1.2.3 经典图像分类网络结构

下图是一个经典的图像分类的卷积神经网络结构，网络结构从左到右有多个网络模型层数组成，每一层都用来提取更高层次的特征。



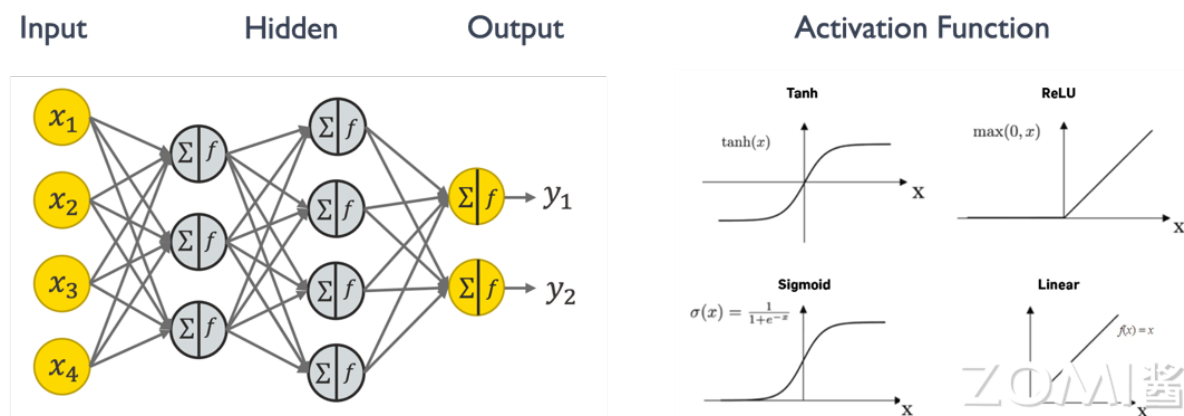
1.3 权重求和和计算范式

接下来我们来了解一下神经网络中的主要计算范式：权重求和。权重求和是神经网络中最基本也是最核心的计算操作，等

1.3.1 权重求和的基本原理

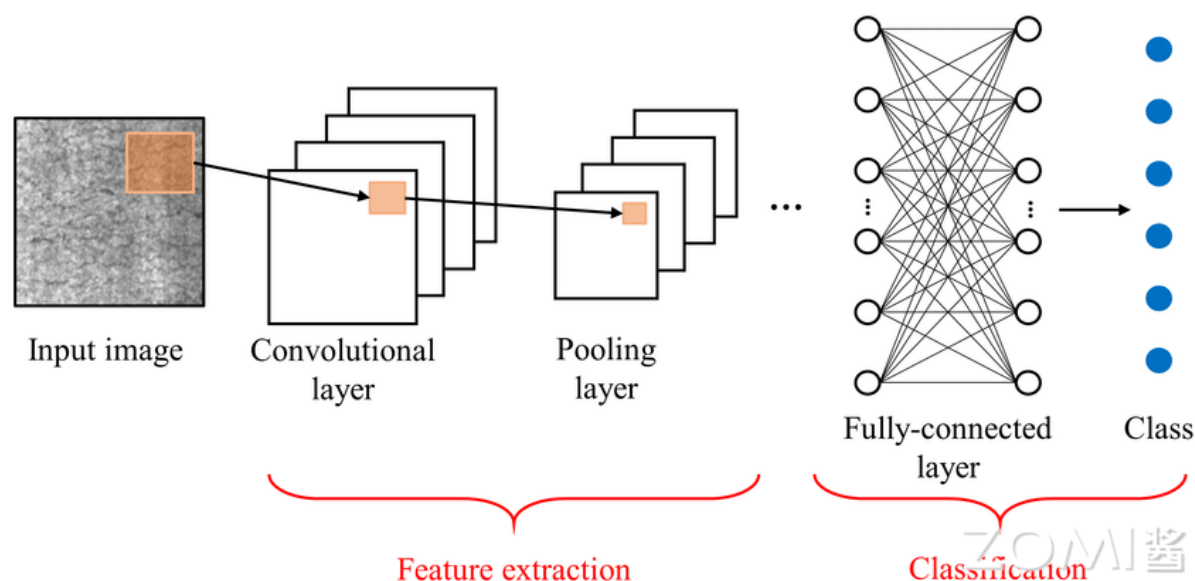
神经网络中90%的计算量都是在做乘加（multiply and accumulate, MAC）的操作，也称为权重求和。MAC操作可以表示为 $\sum_{i=1}^n w_i \cdot x_i + b$ ，其中 w_i 是权重， x_i 是输入， b 是偏置。

下图是一个简单的神经网络结构，左边中间灰色的圈圈表示一个简单的神经元，每个神经元里有求和和激活两个操作，求



1.3.2 全连接层的权重求和

全连接层（Fully Connected Layer），也称为密集连接层或仿射层，是深度学习神经网络中常见的一种层类型，通常位于网络



全连接层的基本概念和特点：

连接方式：全连接层中的每个神经元与前一层的所有神经元都有连接，这意味着每个输入特征都会影响到每个输出神经元。其计算复杂度为 $m \times m$ （包括偏置）。

参数学习：全连接层中的连接权重和偏置项是需要通过反向传播算法进行学习的模型参数，这些参数的优化过程通过最小化损失函数实现。

特征整合：全连接层的主要作用是将前面层提取到的特征进行整合和转换，以生成最终的输出。在图像分类任务中，全连接层通常位于卷积层和池化层之后。

非线性变换：在全连接层中通常会应用非线性激活函数，如ReLU（Rectified Linear Unit）、Sigmoid或Tanh等，以增加模型的表达能力和非线性拟合能力。

输出层：在分类任务中，全连接层通常作为网络的最后一层，并配合Softmax激活函数用于生成类别预测的概率分布。在回归任务中，通常使用线性激活函数。

过拟合风险：全连接层的参数数量通常较大，因此在训练过程中容易产生过拟合的问题。为了缓解过拟合，可以采用正则化技术，如L1或L2正则化。

1.3.3 矩阵乘法在神经网络中的应用

在神经网络中，权重求和通常以矩阵乘法的形式实现。给定输入向量 x 和权重矩阵 W ，偏置向量 b ，全连接层的计算可以表示为：

$$y = W \cdot x + b$$

对于批量输入，假设输入矩阵为 $X \in \mathbb{R}^{batch \times n}$ ，权重矩阵为 $W \in \mathbb{R}^{n \times m}$ ，则输出为 $Y = X \cdot W + B$ ，其中 $Y \in \mathbb{R}^{batch \times m}$ 。

这种矩阵运算形式非常适合硬件加速，因为：

- 矩阵乘法具有高度的并行性
- 矩阵乘法是计算密集型操作，适合利用专用计算单元
- 矩阵乘法的数据重用率高，适合利用缓存机制

1.3.4 AI芯片对权重求和的支持

理解权重求和的计算范式对AI芯片设计至关重要。AI芯片需要：

1. 支持高吞吐量的MAC运算：MAC是神经网络的核心操作，AI芯片需要能够高效执行大量的MAC运算。
2. 支持矩阵运算的并行化：利用并行计算单元同时处理多个MAC运算，提高计算效率。
3. 提供高带宽的数据传输：权重求和涉及大量的数据传输，AI芯片需要提供高带宽的存储和数据传输架构。
4. 支持不同精度的计算：根据应用场景，AI芯片需要支持FP32、FP16、INT8等多种精度的MAC运算。

1.4 神经网络的多样性与应用

神经网络在深度学习中被广泛应用于各种任务，包括图像分类、目标检测、语音识别和自然语言处理等领域。不同类型的神经网络具有不同的结构和应用。

1.4.1 网络结构的多样性

虽然神经网络的基本组件相似，但针对不同任务和数据类型，发展出了多种网络结构：

卷积神经网络（CNN）：主要用于处理图像数据，通过卷积操作提取空间特征。

循环神经网络（RNN）：主要用于处理序列数据，通过循环连接捕捉时序依赖。

Transformer网络：通过注意力机制处理序列数据，能够捕捉长距离依赖关系。

图神经网络（GNN）：用于处理图结构数据，能够学习节点和边的表示。

1.4.2 深度神经网络的发展趋势

随着深度学习技术的发展，神经网络呈现出以下发展趋势：

1. 网络深度不断增加：从几层到数百层甚至上千层，如ResNet的深度达到了152层。
2. 网络结构更加复杂：从简单的线性结构到残差连接、密集连接、注意力机制等复杂结构。
3. 模型规模持续增长：参数数量从百万级增长到万亿级，如GPT-3有1750亿参数。
4. 多模态融合：将文本、图像、语音等多种模态的信息融合处理。

理解这些发展趋势对于AI芯片设计至关重要，因为它们直接影响着AI芯片的架构设计和性能优化方向。

本章小结

本章介绍了神经网络的基本概念和组成组件，包括：

1. 神经元：神经网络的基本计算单元，模拟生物神经元接收输入、进行加权求和并通过激活函数产生输出。
2. 激活函数：引入非线性，使网络能够学习和表示复杂模式。常见的激活函数包括ReLU、Sigmoid、Tanh和Softmax。
3. 网络层次结构：神经网络由输入层、隐藏层和输出层组成，深度网络通过多层隐藏层提取不同层次的特征。
4. 训练与推理：训练阶段通过反向传播和优化算法学习模型参数；推理阶段使用训练好的模型进行预测。
5. 权重求和：神经网络的核心计算范式，90%的计算量来自MAC操作，以矩阵乘法的形式高效执行。
6. 全连接层：一种基础的层类型，通过密集连接整合前面层的特征，是许多网络结构的重要组成部分。

思考与练习

1. 神经元计算：假设一个神经元有3个输入 $x_1 = 0.5, x_2 = -0.3, x_3 = 0.8$ ，对应的权重 $w_1 = 0.2, w_2 = 0.5, w_3 = -0.4$ ，偏置 $b = 0.1$ 。分别计算使用ReLU和Sigmoid激活函数的输出。
2. 梯度计算：对于简单的单层网络 $\hat{y} = \sigma(Wx + b)$ ，其中 σ 是Sigmoid函数，推导损失函数 $L = (y - \hat{y})^2$ 对参数 W 和 b 的梯度。
3. 参数量计算：对于一个全连接层，输入维度为1024，输出维度为256，计算该层的参数数量，并分析参数量对模型性能的影响。
4. 训练vs推理：分析神经网络训练阶段和推理阶段在计算需求、内存使用和优化策略方面的主要区别。
5. 计算范式分析：分析权重求和（MAC）操作在典型深度学习模型（如ResNet-50、BERT）中的计算占比，并讨论这对AI芯片设计的启示。

卷积神经网络

2.1 卷积层的基本概念

卷积层是深度学习神经网络中常用的一种层类型，主要用于处理图像和序列数据。它的主要作用是通过学习可重复使用的

2.1.1 卷积操作原理

卷积层通过对输入数据应用卷积操作来提取特征。卷积操作是一种在输入数据上滑动卷积核并计算卷积核与输入数据的乘积和的操作。卷积操作的具体过程：

1. 卷积核（Kernel）：卷积核是卷积层的参数，它是一个小的可学习的权重矩阵，用于在输入数据上执行卷积操作。
2. 滑动窗口：卷积核在输入数据上按步长（Stride）滑动，通常从左上角开始，逐行或逐列移动。
3. 逐元素乘加：在每个位置，将卷积核与输入数据的对应元素相乘，然后将所有乘积相加。
4. 偏置和激活：将累加结果加上偏置，然后通过激活函数产生该位置的输出。

2.1.2 卷积层的关键特性

特征图（Feature Map）：卷积操作的输出称为特征图（Feature Map），它是通过卷积核在输入数据上滑动并应用激活函数得到的。步长和填充：卷积操作中的步长（Stride）定义了卷积核在输入数据上滑动的步长，而填充（Padding）则是在输入数据的边缘添加零，使输出Feature Map和输入的尺寸相同，Valid的输出Feature Map一般比输入小。

参数共享：卷积层的参数共享是指在整个输入数据上使用相同的卷积核进行卷积操作，这样可以大大减少模型的参数数量。

局部连接：与全连接层的全局连接不同，卷积层只连接输入数据的局部区域。这种局部连接特性使得卷积层能够捕捉局部特征。

2.1.3 卷积层的数学表示

对于输入特征图 I ，卷积核 K ，卷积操作可以表示为：

$$(I * K)_{i,j} = \sum_m \sum_n I_{i+m,j+n} \cdot K_{m,n}$$

对于具有多个输入通道和多个输出通道的卷积层：

$$Y_{c_{out},i,j} = \sum_{c_{in}} \sum_m \sum_n K_{c_{out},c_{in},m,n} \cdot X_{c_{in},i+m,j+n} + b_{c_{out}}$$

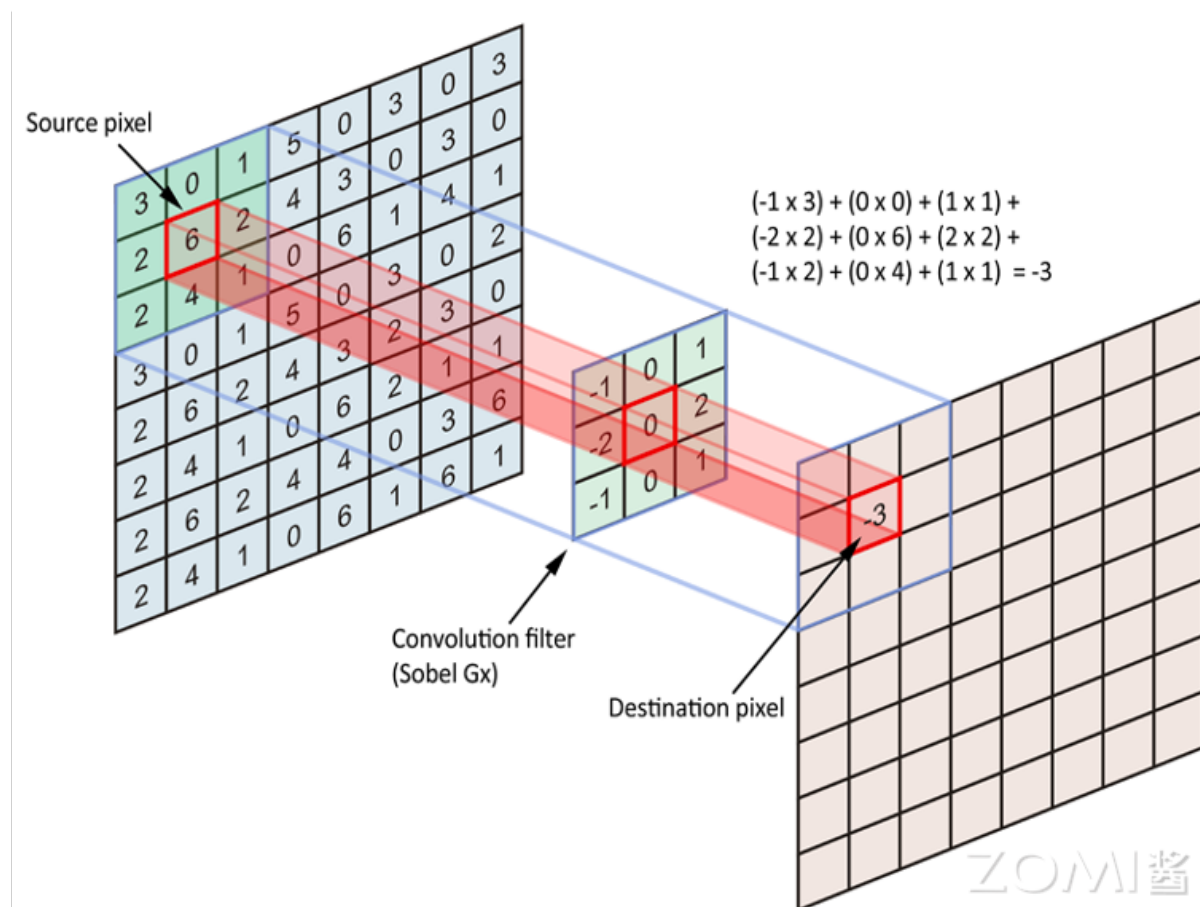
其中：

- X 是输入特征图，维度为 (C_{in}, H_{in}, W_{in})
- K 是卷积核，维度为 $(C_{out}, C_{in}, K_h, K_w)$
- Y 是输出特征图，维度为 $(C_{out}, H_{out}, W_{out})$
- b 是偏置

2.2 卷积层的详细计算过程

2.2.1 单通道卷积示例

下图是一个一层5x5的Feature Map的卷积过程，橙色表示3x3的Kernel，左侧浅色方框和Kernel相乘并累加和，得到右侧浅Map输出。



以该图为例详细说明卷积计算过程：

输入特征图：5x5的单通道图像

卷积核：3x3的Kernel

步长：1

Valid模式：

- 输出尺寸 = $\text{floor}((\text{输入尺寸} - \text{卷积核尺寸}) / \text{步长}) + 1 = \text{floor}((5-3)/1) + 1 = 3$
- 输出为3x3的特征图

Same模式：

- 在输入周围填充0，使输出尺寸与输入尺寸相同
- 输出为5x5的特征图

2.2.2 多通道卷积

实际应用中，输入图像通常是RGB三通道图像。设输入有 C_{in} 个通道，输出有 C_{out} 个通道：

1. 每个输出通道有独立的卷积核，维度为 (C_{in}, K_h, K_w)
2. 输入通道与对应卷积核进行卷积，然后所有通道结果相加
3. 加上该输出通道的偏置，通过激活函数得到最终输出

参数数量：对于卷积层，参数数量为 $(K_h \times K_w \times C_{in} + 1) \times C_{out}$

2.2.3 卷积层的FLOPs计算

卷积层的浮点运算数（FLOPs）计算公式为：

$$FLOPs = 2 \cdot H_{out} \cdot W_{out} \cdot C_{in} \cdot K_h \cdot K_w \cdot C_{out}$$

其中：

- H_{out}, W_{out} 是输出特征图的空间尺寸
 - C_{in} 是输入通道数
 - K_h, K_w 是卷积核尺寸
 - C_{out} 是输出通道数
 - 因子2是因为每次乘加运算包含一次乘法和一次加法
-

2.3 池化操作

在卷积层之后通常会使用池化层来减小特征图的尺寸并提取最重要的特征。池化操作通过对局部区域进行下采样来减少数据量。

2.3.1 最大池化 (Max Pooling)

最大池化取局部区域中的最大值：

$$Y_{i,j} = \max_{(m,n) \in \text{patch}} X_{m,n}$$

最大池化能够保留显著特征，对小的平移不敏感。

2.3.2 平均池化 (Average Pooling)

平均池化取局部区域的平均值：

$$Y_{i,j} = \frac{1}{k_h \times k_w} \sum_{(m,n) \in \text{patch}} X_{m,n}$$

平均池化能够保留区域的整体信息，产生更平滑的特征表示。

2.3.3 全局池化

全局池化将整个特征图压缩为一个值，常用于网络的最后阶段：

- 全局平均池化（Global Average Pooling, GAP）：对整个特征图求平均，输出一个标量
- 全局最大池化（Global Max Pooling）：取整个特征图的最大值

全局池化可以有效替代全连接层，大幅减少参数数量。

2.4 经典卷积神经网络架构

2.4.1 LeNet-5

LeNet-5是由Yann LeCun等人于1998年提出的经典卷积神经网络，主要用于手写数字识别。它包含：

- 2个卷积层
- 2个池化层
- 2个全连接层
- 1个输出层

LeNet-5奠定了卷积神经网络的基本架构：卷积层用于特征提取，池化层用于下采样，全连接层用于分类。

2.4.2 AlexNet

AlexNet在2012年ImageNet竞赛中取得了突破性成果，开启了深度学习在计算机视觉领域应用的新时代。主要特点：

- 5个卷积层
- 3个全连接层
- 使用ReLU激活函数
- 使用Dropout正则化
- 使用GPU进行训练

2.4.3 VGGNet

VGGNet由牛津大学的研究团队提出，以其简洁统一的设计著称。主要特点：

- 使用更小的3x3卷积核代替大的卷积核
- 网络深度更深（16-19层）
- 结构非常规整，易于理解和实现

VGGNet证明了网络深度对模型性能的重要性，但同时也带来了更大的计算量和内存需求。

2.4.4 ResNet

ResNet（Residual Network）通过引入残差连接解决了深层网络训练困难的问题。主要特点：

- 引入跳跃连接（Skip Connection）
- 使信息能够直接跨层传递
- 允许训练非常深的网络（152层甚至更多）
- 大幅提升了模型性能

残差连接的数学形式： $y = \mathcal{F}(x) + x$ ，其中 $\mathcal{F}$ 是卷积层的映射， x 是输入。
